

Universitatea POLITEHNICA din București
Facultatea de Electronică, Telecomunicații și Tehnologia Informației

**Arhitectură Agentică pentru Asistență Inteligentă în
Dezvoltarea Software**

Lucrare de licență

**Prezentată ca cerință parțială pentru obținerea
titlului de *Inginer*
în domeniul *Calculatoare și Tehnologia Informației*
programul de studii *Ingineria Informației***

Conducător științific
Ș.L. dr. ing. Mihai DOGARIU

Absolvent
Eduard-Daniel NANESCU

Anul 2026

TEMA PROIECTULUI DE DIPLOMĂ
a studentului **Eduard-Daniel NANESCU, 441A**

1. Titlul temei: Arhitectură Agentică pentru Asistență Inteligentă în Dezvoltarea Software

2. Descrierea temei:

Contribuția originală a lucrării va consta în dezvoltarea unei arhitecturi software bazată pe Modele Mari de Limbaj (LLMs - Large Language Models), capabilă să asiste utilizatorul în scrierea, depanarea, documentarea și explicarea codului sursă. Aplicațiile vizate sunt eficientizarea procesului de dezvoltare software și automatizarea sarcinilor repetitive întâlnite în programare. Sistemul propus nu vizează antrenarea de la zero a modelelor, ci implementarea unei soluții agentice care utilizează modele pre-antrenate pentru a executa acțiuni complexe. Cercetarea aferentă va include: sinteza bibliografică a realizărilor actuale din domeniu, utilizarea unui mecanism care permite modelului să utilizeze instrumente pentru a interacționa cu mediul virtual, implementarea unor tehnici de inginerie a prompturilor și management-ul contextului pentru a asigura relevanța răspunsurilor, validarea experimentală a prototipului dezvoltat prin scenarii de utilizare reale utilizând medii de dezvoltare precum PyCharm/VS Code și biblioteci specifice precum LangChain, LangGraph sau GoogleADK, concluzionarea rezultatelor obținute și a contribuției originale, cât și enunțarea perspectivelor ulterioare de cercetare.

3. Contribuția originală:

Proiectarea și implementarea arhitecturii agentului, incluzând logica de selecție și utilizare a instrumentelor, gestionarea contextului și dezvoltarea unei interfețe sau integrarea cu interfețe existente care să permită interacțiunea facilă cu asistentul.

4. Resurse și bibliografie:

- Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. "Attention is all you need." Advances in neural information processing systems 30 (2017).
- Yao, Shunyu, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. "React: Synergizing reasoning and acting in language models." In The eleventh international conference on learning representations. 2022.
- Bozkurt, Aras, and Ramesh C. Sharma. "Generative AI and prompt engineering: The art of whispering to let the genie out of the algorithmic world." Asian Journal of Distance Education 18, no. 2 (2023): i-vii.
- Alto, Valentina. Building LLM Powered Applications: Create intelligent apps and agents with large language models. Packt Publishing Ltd, 2024.
- Zhu, He, Tianrui Qin, King Zhu, Heyuan Huang, Yeyi Guan, Jinxiang Xia, Yi Yao et al. "Oagents: An empirical study of building effective agents." arXiv preprint arXiv:2506.15741 (2025).

5. Data înregistrării temei: 2025-12-11 22:54:00

Conducător(i) lucrare,
Ș.L. dr. ing. Mihai DOGARIU

Student,
Eduard-Daniel NANESCU

Director departament,
Conf. dr. ing Bogdan FLOREA

Decan,
Prof. dr. ing. Mihnea UDREA

Cod Validare: **621d9dfde3**

Declarație de onestitate academică

Prin prezenta declar că lucrarea cu titlul *Arhitectură Agentică pentru Asistență Inteligentă în Dezvoltarea Software*, prezentată în cadrul Facultății de Electronică, Telecomunicații și Tehnologia Informației a Universității “Politehnica” din București ca cerință parțială pentru obținerea titlului de *Inginer* în domeniul Inginerie Electronică și Telecomunicații/ Calculatoare și Tehnologia Informației, programul de studii *Ingineria Informației* este scrisă de mine și nu a mai fost prezentată niciodată la o facultate sau instituție de învățământ superior din țară sau străinătate. Declar că toate sursele utilizate, inclusiv cele de pe Internet, sunt indicate în lucrare, ca referințe bibliografice. Fragmentele de text din alte surse, reproduse exact, chiar și în traducere proprie din altă limbă, sunt scrise între ghilimele și fac referință la sursă. Reformularea în cuvinte proprii a textelor scrise de către alți autori face referință la sursă. Înțeleg că plagiatul constituie infracțiune și se sancționează conform legilor în vigoare. Declar că toate rezultatele simulărilor, experimentelor și măsurărilor pe care le prezint ca fiind făcute de mine, precum și metodele prin care au fost obținute, sunt reale și provin din respectivele simulări, experimente și măsurători. Înțeleg că falsificarea datelor și rezultatelor constituie fraudă și se sancționează conform regulamentelor în vigoare.

București, Iunie 2026.

Absolvent: Eduard-Daniel NANESCU

.....

Cuprins

LISTA FIGURILOR	2
LISTA TABELELOR	4
LISTA DE ACRONIME	5
INTRODUCERE	7
CONTEXT SI MOTIVATIE	7
SCOPUL SI OBIECTIVELE LUCRARI	8
1. LUCRARI ANTERIOARE	9
1.1. Evolutia catre modelele mari de limbaj (Large Language Model (LLM))	9
1.2. Maturizarea modelelor mari de limbaj si cercetari actuale	15
1.3. Solutii agentice pentru industria dezvoltarii software	21
1.4. Cerinte pentru o solutie agentica in dezvoltarea software	29
1.5. Limitari ale arhitecturilor agentice	29
1.6. Justificarea alegerilor de implementare	30
2. PRACTICI CURENTE IN INDUSTRIE	34
2.1. Software open-source	34
2.2. Flux agentic ce ruleaza local	34
2.3. Observabilitate	35
2.4. Securitate si confidentialitate	35
2.5. Cost	35
2.6. Aplicatii Open Source Software (OSS) concurente	35
3. IMPLEMENTARE	38
3.1. Mediul de dezvoltare	38
3.1.1. Cerinte hardware	38
3.1.2. Cerinte software	39
3.2. Structura	39
3.3. Configurare	39
3.4. Interfata cu utilizatorul	40
3.5. Instalare si utilizare	42
3.5.1. Cerinte preliminare	42
3.5.2. Instalare	42
3.5.3. Configurare la prima rulare	43
3.5.4. Utilizare	43
3.5.4.1. Modul headless	44
3.6. Gestionarea contextului si a memoriei	44
3.6.1. Memoria si istoricul conversatiei	44
3.6.2. Contextul	45

3.6.2.1.	Retrieval Augmented Generation (RAG)	47
3.6.2.2.	Gestionarea datelor multimodale	47
3.6.2.3.	Compactarea	48
3.6.2.4.	Tehnici pentru eficientizarea ferestrei de context	50
3.7.	Componente ale arhitecturilor agentice moderne	50
3.7.1.	Unelte	50
3.7.2.	Modul de planificare	52
3.7.3.	Skills	53
3.7.4.	Subagenti	54
3.8.	Extensibilitate	56
3.8.1.	Model Context Protocol (MCP) si adaugarea de unelte	56
3.8.2.	Adaugarea de subagenti	56
3.8.3.	Creare de skill-uri	58
3.8.4.	Suportul pentru furnizori de modele de limbaj	58
3.8.5.	Adaugarea tehnicilor de compactare	59
3.9.	Securitatea in mediul agentice	60
3.9.1.	Sandboxing	60
3.9.2.	Permisuni	61
3.9.3.	Guardrails, intentii periculoase si cereri in afara scopului (Out of Scope (OOS))	62
3.10.	Observabilitate	62
3.10.1.	Langfuse	62
3.10.2.	Panou web	64
4.	REZULTATE EXPERIMENTALE	65
4.1.	Pozitionarea fata de concurenta	65
4.1.0.1.	Functionalitati prezente in concordanta cu aplicatiile concurente proprietare si Open Source Software (OSS):	68
4.1.0.2.	Functionalitati absente fata de aplicatiile concurente:	68
4.2.	Influenta lungimii si a exactitatii prompt-ului asupra performantei. De ce un prompt prea exact nu garanteaza rezultate mai bune	69
4.3.	Contextul in mediul agentice si compactare. Consecintele metodelor de compactare	72
4.3.1.	Stadii care taie date din context	72
4.3.2.	Stadii care parafrazeaza date din context	72
4.3.3.	Stadii auxiliare	72
4.3.4.	Secventa de stadii de compactare recomandata	73
4.4.	Interactiunea dintre modele, context si unelte. Modele optime pentru sarcini de programare	73
4.4.1.	Rezultate	74
4.4.1.1.	Analiza rezultatelor	75
4.4.1.2.	Modelele open-source: evaluarea valorii si a potentialului de utilizare	76
5.	DEZVOLTARI VIITOARE	77
5.1.	Automatizari, integrari cu alte programe	77
5.2.	Integrarea cu mediile de dezvoltare	77
5.3.	Suport nativ pentru Windows	77
5.4.	Construirea unui graf de cunostinte pentru optimizarea intelegerii proiectelor	77

5.5. Suport pentru metode de compactare avansate si Software Development Kit (SDK) pentru agenti	78
6. CONCLUZII	79
BIBLIOGRAFIE	80

LISTA FIGURILOR

1.1. Modelul unui perceptron (sursa)	10
1.2. Convolutia cu un nucleu a unei imagini (sursa)	11
1.3. Recurrent Neural Network (RNN) si Long Short Term Memory (LSTM) com- parate (sursa)	12
1.4. Arhitectura transformer [1]	14
1.5. Evolutia recenta a modelelor mari de limbaj [2]	15
1.6. Evolutia modelelor Generative Pre-trained Transformer (GPT) [2]	15
1.7. Performanta tipurilor de arhitecturi de tipul "Longformer" [3]. In timp ce meca- nismul tipic de atentie utilizeaza memorie exponential, arhitectura Longformer foloseste memorie liniar odata cu cresterea contextului	16
1.8. Diferite configuratii de atentie folosite in arhitectura Longformer in comparatie cu mecanismul obisnuit de atentie [3]	16
1.9. Principiul de baza pentru Rotary Position Embedding [4]	17
1.10. Fine-tuning-ul se realizeaza doar pe A si B [5]	17
1.11. Rezultatele procesului de Simple Self Distillation [6]	18
1.12. Imbunatatiri in sarcini de programare ale modelelor dupa ce au fost supuse pro- cesului de Simple Self Distillation[6]	19
1.13. Comparatia procedeele de fine-tuning: fine-tuning clasic vs LoRA vs QLoRA [7]	19
1.14. Ilustrarea procedeeului de Reinforcement Learning (RL) [8]	20
1.15. Exemple de Chain-of-Thought (CoT) [9]	21
1.16. Diferenta intre procedee de gandire [10]	21
1.17. Paradigma ReAct [11]	22
1.18. Imbunatatirile pe care le aduce ReAct [11]	22
1.19. Ideea din spatele ReWOO [12]	23
1.20. Comparatia de performante intre paradigme pentru imbunatatirea gandirii din punctul de vedere al acuratetii vs. numarul de tokeni consumati [12]	23
1.21. Imbunatatirea adusa de ReWOO este delegarea mai multor libertati catre worker ce poate apela mai multe unelte pana sa produca un raspuns. P = Plan, E = Evidence, A = Answer, O = Observation, A = Act, T = Thought [12]	24
1.22. Modele tind sa aiba o acuratete foarte slaba contra intrebarilor ce vizeaza infor- matii care se afla la mijlocul ferestrei curente de context [13]	25
1.23. Vizualizarea problemei si al rezultatului dorit la intrebare. De multe ori, modelele intampina dificultati in accesarea acestor informatii ascunse in mijlocul ferestrei de context [13]	25
1.24. In majoritatea modelelor se observa o scadere a performantelor dupa 32K tokeni pe sarcini de gandire complexa[14]	26
1.25. Clasamentul modelelor in SWE-Bench Pro [15] la momentul actual in functie de performanta rezolvarii sarcinilor complexe in limba engleza	28
1.26. Clasamentul modelelor in SWE-Bench Pro [15] la momentul actual in functie de cost	29
1.27. Reprezentarea ferestrei de context intr-o conversatie uzuala (sursa)	30

2.1. Reprezentarea ferestrei de context intr-o conversatie in Claude Code	35
2.2. Reprezentarea utilizarii in Claude (varianta web) (sursa)	35
3.1. Structura directoarelor vitale ale proiectului	39
3.2. Prima versiune a interfetei, realizata cu ajutorul bibliotecii Textual	40
3.3. Continuarea conversatiei cu prima versiune a interfetei, realizata cu ajutorul bibliotecii Textual	40
3.4. A doua versiune a interfetei, realizata fara biblioteca Textual	41
3.5. Continuarea conversatiei cu a doua versiune a interfetei, realizata fara biblioteca Textual	41
3.6. Impartirea ferestrei de context (sursa)	46
3.7. Configurarea pipeline-ului de compactare a contextului	49
3.8. Implementarea unei functii pe care agentul o foloseste pentru a lista directoriile si fisierele la o anumita cale pe sistem	51
3.9. Configurarea gardarii executiei uneltelor	52
3.10. Implementarea unei capabilitati de Pull Request (PR) review	53
3.11. Implementarea unei functii pe care agentul o foloseste pentru a porni un subagent	55
3.12. Configurarea subagentilor	56
3.13. Inregistrarea unui nou subagent	57
3.14. Configurarea sandbox-ului Docker pentru executia izolata a comenzilor	61
3.15. Cererea de aprobare a permisiunii de a scrie un fisier	62
3.16. Interfata Langfuse	63
3.17. Imagini cu interfata web a aplicatiei	64

LISTA TABELELOR

1.1.	Comparatia catorva din modelele existente si a capabilitatile acestora in comparatie cu pretul de utilizare	27
1.2.	Comparație framework-uri pentru workflow-uri agentice	31
1.3.	Comparație biblioteci pentru interfața cu utilizatorul	32
2.2.	Top 6 asistenti de programare open source	36
2.1.	Produse Inteligență Artificială (IA) ale principalilor furnizori de modele de limbaj	37
3.1.	Hardware recomandat pentru modele open-source	38
3.2.	Comenzi disponibile in interfata	43
4.1.	Comparatie intre <code>vibe-cli</code> si principalele unelte agentice de programare pentru terminal disponibile pe piata	66
4.2.	Comparatie intre <code>vibe-cli</code> si principalii asistenti de programare Open Source Software (OSS) din ecosistem	67
4.3.	Rezultatele pentru prompt-ul detaliat: metrici per proiect si pe nivele de severitate	70
4.4.	Rezultatele pentru prompt-ul simplu: metrici per proiect si pe nivele de severitate	71
4.5.	Modelele testate: tip, numar de parametrii, marime pe disc si suport multimodal.	74
4.6.	Clasamentul modelelor dupa scor mediu, durata totala de rulare si cost. Modelele rulate local prin <code>ollama</code> au cost zero. Modelele proprietare nu au datele referitoare la marimea modelului publice	74

LISTA DE ACRONIME

AML Adversarial Machine Learning	MCP Model Context Protocol
API Application Programming Interface	MHA Multi-Head Attention
BLEU Bilingual Evaluation Understudy	ML Machine Learning
BTT Backpropagation Through Time	MLOps Machine Learning Operations
CD Continuous Deployment	MoE Mixture of Experts
CI Continuous Integration	NLP Natural Language Processing
CLI Command Line Interface	OOS Out of Scope
CNN Convolutional Neural Network	OSS Open Source Software
CoT Chain-of-Thought	PDF Portable Document Format
CPU Central Processing Unit	POSIX Portable Operating System Interface
CSS Cascading Style Sheets	PR Pull Request
DOM Document Object Model	RAG Retrieval Augmented Generation
E2E End to End	RAM Random Access Memory
FN False Negative	RCA Root Cause Analysis
FP False Positive	RL Reinforcement Learning
GPT Generative Pre-trained Transformer	RNN Recurrent Neural Network
GPU Graphics Processing Unit	ROUGE Recall-Oriented Understudy for Gisting Evaluation
GUI Graphical User Interface	SDD Spec-Driven Development
HTML Hyper-Text Markup Language	SDK Software Development Kit
HTTP Hyper-Text Transfer Protocol	SHA Secure Hash Algorithms
IA Inteligență Artificială	SSE Server-Side Events
IDE Integrated Development Environment	STDIO Standard Input-Output
JSON Java Script Object Notation	TDD Test-Driven Development
KV Key-Value	TN True Negative
LLM Large Language Model	ToT Tree-of-Thought
LSTM Long Short Term Memory	TP True Positive

TUI Terminal User Interface

URL Uniform Resource Locator

UUID Universally Unique Identifier

VRAM Video Random Access Memory

YAML YAML Ain't Markup Language

INTRODUCERE

CONTEXT SI MOTIVATIE

Motivatia acestei lucrari vine din experienta personala in industria dezvoltarii software. Atributiile unui inginer software sunt numeroase, diverse si adesea repetitive. Aceasta congestie a timpului ingreuneaza procesul de dezvoltare. A scrie cod a devenit doar o mica parte din sarcinile unui inginer software, pierzand teren in fata design-urilor, documentatiei, review-urilor, a testelor unitare de integrare si End to End (E2E), instrumentarii si monitorizarii, procesului de Continuous Integration (CI)/Continuous Deployment (CD), infrastructurii, iar in cazul problemelor a debugging-ului si a analizei cauzelor principale (Root Cause Analysis (RCA)).

Un inginer software care lucreaza intr-o companie ce deserveste milioane de clienti din intreaga lume trebuie sa fie competitiv, iar aceasta competitivitate se rezuma la a fi la curent cu cele mai noi practici din domeniu, cunoasterea notiunilor de programare, securitate cibernetica, retelistica si infrastructura, dar sa si retina simultan informatii vitale despre arhitectura si sistemele interne ale companiei, totul pe langa instruirea colegilor noi sau sedinte.

Timpul unui inginer este astfel impartit de foarte multe procese dinamice care trebuiesc adresate de un angajat capabil sa sustina acest ritm alert.

Avand in vedere cele de mai sus, cat si evolutia recenta a sistemelor bazate pe Inteligență Artificială (IA) ce folosesc un Large Language Model (LLM) pentru intelegerea si generarea de text, era natural ca directia in care dezvoltarea software urma sa se schimbe catre un proces in care multe din sarcinile pe care un inginer le-ar fi avut sunt acum delegate sistemelor automate. Astfel, atentia inginerului se muta de la a scrie cod, la supravegherea instrumentelor de programare bazate pe inteligenta artificiala si asumarea deciziilor arhitecturale. Restul grijiilor cotidiene pot fi abordate de aceste instrumente, eliberand programul operatorului.

Trecerea de la o experienta pur conversationala cu aceste modele la intrebuintari pentru cautare prin limbaj natural, iar in final integrandu-se cu mediile de dezvoltare software si cu terminalul sistemului de operare pentru a avea acces la un context mai bun dar si pentru executia de comenzi, nu a fost una dificila, avand in vedere competentele avansate de prelucrare si generare de text ale acestor modele.

Momentan, in domeniul modelelor de limbaj, exista cativa jucatori mari precum Google, Anthropic si OpenAI care controleaza o buna parte din cota de piata. Acestia sunt in stadiul in care isi fortifica ecosistemul pentru a-si maximiza retentia de utilizatori. Fiecare furnizor de astfel de servicii lanseaza produse care se folosesc de aceste modele. De la asistenti de cercetare, la asistenti de programare si la programe ce automatizeaza sarcini zilnice. Odata cu aceste produse, incearca sa restrictioneze sau sa ofere tarife diferite pentru utilizatori ce folosesc modelele pentru astfel de sarcini in afara ecosistemului lor.

Exista si modele de limbaj de tipul Open Source Software (OSS), de diferite marimi si cu diverse proprietati, dar ecosistemul uneltelor gratuite sau OSS care folosesc aceste modele pentru sarcini avansate inca este in proces de dezvoltare. Astfel, este un mediu propice pentru un produs ce livreaza un asistent inteligent pentru dezvoltarea software, independent de furnizor, construit pentru costuri reduse, performanta ridicata, abordarea problemelor complexe ce poate rula local, precum cel pe care imi propun sa il realizez in aceasta lucrare.

SCOPUL SI OBIECTIVELE LUCRARI

Scopul acestei lucrari este de a realiza un **asistent inteligent extensibil si transparent** bazat pe IA, **agnostic fata de platforma sau de furnizori de modele de limbaj**, ce ruleaza local sau in cloud si ce poate interactiona cu mediul virtual in care ruleaza (sistem de operare, mediu de dezvoltare, internet) prin intermediul uneltelor (tool-uri ¹), realizat de un workflow agentic ²

Obiectivul principal care trebuie atins este realizarea unui produs functional ce poate ajuta un inginer software in cat mai multe aspecte ale meseriei sale. Pentru a indeplini acest obiectiv, trebuie sa avem in vedere urmatoarele necesitati: interfata facila ce poate fi folosita oriunde folosind terminalul sistemului de operare (Terminal User Interface (TUI)), suficiente unelte pentru a indeplini sarcini cat mai complexe, management-ul conversatiei, contextului si al memoriei, versionare, evaluari si teste de regresie, monitorizare si observabilitate (metrici), siguranta (medii sigure pentru executarea codului, masuri de protectie pentru date de intrare malitioase, Out of Scope (OOS) detection), si cel mai important: un prompt de sistem ales cu grija.

¹In IA folosita cu ajutorul agentilor **tool**-urile sunt functii cu descrieri detaliate de folosire pe care agentul le populeaza cu parametrii, le executa si receptioneaza raspunsul pentru a-l folosi mai departe in rezolvarea problemelor complexe

²**Agentii** sunt aplicatii ce folosesc inteligenta artificiala pentru a interactiona cu mediul inconjurator

Capitolul 1

LUCRARI ANTERIOARE

O buna intelegere a tehnologiilor, standardelor si a obstacolelor actuale din industrie necesita studierea literaturii de specialitate, a lucrarilor si a arhitecturilor existente deja in domeniu. Acest pas este obligatoriu pentru alegeri informate in cadrul implementarii proiectului, evitarea problemelor deja recunoscute si confera o vedere de ansamblu asupra stadiului final al lucrarii.

1.1 Evolutia catre modelele mari de limbaj (LLM)

Domeniul inteligentei artificiale a fost studiat o perioada indelungata iar cateva publicatii au marcat industria prin concepte revolutionare si progrese importante.

Unul din conceptele de baza ale inteligentei artificiale a aparut pentru prima data publicat in anul 1958 de catre F. Rosenblatt in lucrarea [16], unde este prezentat **perceptronul**: un model inspirat din fizionomia neuronului biologic, capabil sa realizeze sarcini de clasificare binara. Perceptronul primeste un set de date de intrare, fiecare asociata unei ponderi, iar rezultatul final este determinat prin combinarea acestor valori si aplicarea unei functii de activare. Procesul de invatare are loc prin ajustarea iterativa a ponderilor in functie de eroarea obtinuta la iesire, mecanism ce reprezinta baza invatarii automate moderne. Aceasta abordare a demonstrat pentru prima data posibilitatea construirii unor sisteme capabile sa invete din experienta, fara a fi programate explicit pentru fiecare situatie (Figura 1.1).

Formal, pentru un vector de intrare $\mathbf{x} \in \mathbb{R}^n$, un vector de ponderi $\mathbf{w} \in \mathbb{R}^n$ si un termen de prag b , iesirea perceptronului este data de:

$$y = \varphi\left(\sum_{i=1}^n w_i x_i + b\right) = \varphi(\mathbf{w}^\top \mathbf{x} + b), \quad (1.1)$$

unde $\varphi(\cdot)$ este functia de activare (treapta in formularea originala). Regula de invatare consta in actualizarea ponderilor proportional cu eroarea $e = y_{\text{tinta}} - y$ si cu o rata de invatare η , $\mathbf{w} \leftarrow \mathbf{w} + \eta e \mathbf{x}$.

Cu toate acestea, desi perceptronul a reprezentat un pas revolutionar in dezvoltarea inteligentei artificiale, modelul prezenta limitari importante. In forma sa initiala, acesta putea clasifica doar date liniar separabile, ceea ce restrictiona capacitatea de rezolvare a problemelor complexe. Din acest motiv, perceptronul simplu era eficient doar pentru reprezentari de date de intrare relativ simple, intr-un singur strat de procesare. Aceste limitari au dus ulterior la dezvoltarea retelelor neuronale multistrat si a algoritmilor mai avansati de antrenare, care au stat la baza progreselor moderne din domeniul inteligentei artificiale.

Un progres major s-a ivit odata cu introducerea retelelor neurale convolutionale (Convolutional Neural Network (CNN)) de catre Yann LeCun si colaboratorii sai in lucrarea [17]. Aceasta abordare a fost conceputa pentru a depasi limitarile modelelor anterioare, precum perceptronul simplu care nu putea procesa eficient informatii complexe din imagini. Aparitia retelelor convolutionale a marcat inceputul aplicarii pe scara larga a inteligentei artificiale in domeniul procesarii si analizei imaginilor.

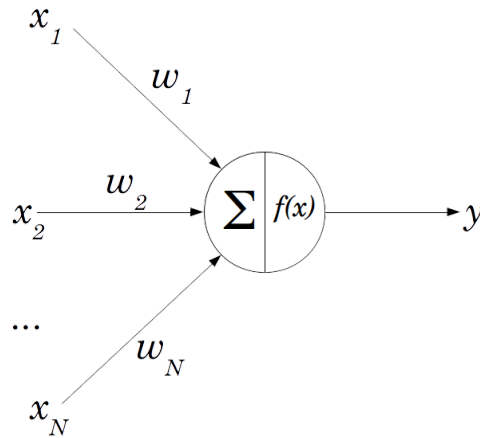


Figura 1.1: Modelul unui perceptron (sursa)

Ideea fundamentală din spatele acestor rețele era relativ rudimentară, însă extrem de eficientă: datele de intrare nu mai sunt tratate ca pe un vector unidimensional, astfel modelul analizează porțiuni locale ale imaginii prin intermediul unor "ferestre" de convoluție. Aceste filtre parcurg imaginea și extrag automat trasături relevante, precum muchii, forme sau texturi. În plus, utilizarea mai multor straturi succesive permitea rețelei să învețe reprezentări din ce în ce mai abstracte și mai complexe ale datelor vizuale.

Un alt element important introdus de rețelele convoluționale a fost utilizarea operațiilor specializate, precum convoluția (Figura 1.2) și subsampling-ul (pooling), care reduceau dimensionalitatea datelor și îmbunătățeau capacitatea de generalizare a modelului. Astfel, CNN au devenit capabile să identifice tipare spațiale complexe și să obțină performanțe superioare în sarcini precum recunoașterea facială, clasificarea imaginilor sau detectia obiectelor. Aceste inovații au reprezentat fundamentul dezvoltării ulterioare a tehnologiilor moderne de computer vision și deep learning.

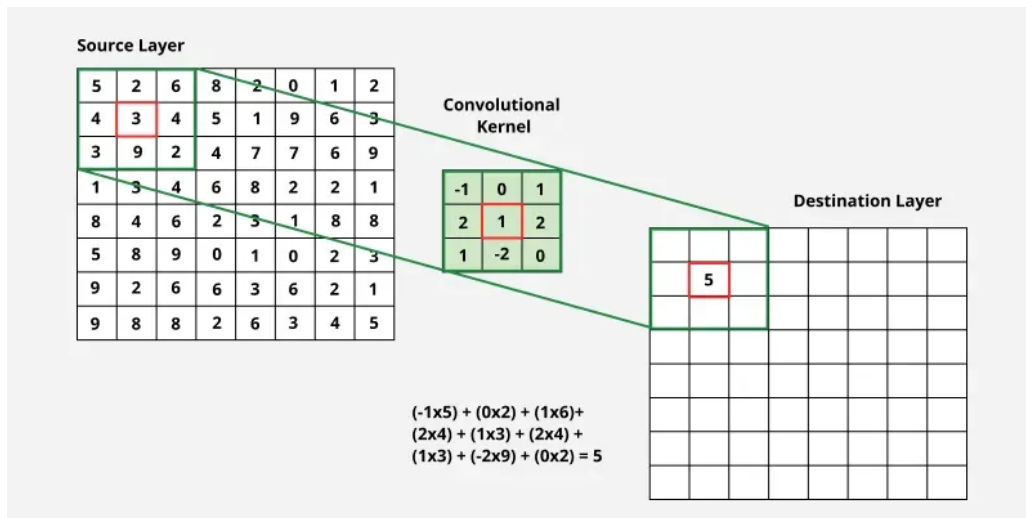


Figura 1.2: Convolutia cu un nucleu a unei imagini (sursa)

Deși rețelele neuronale convoluționale (CNN) au demonstrat performanțe remarcabile în procesarea datelor statice, precum imaginile, acestea nu erau potrivite pentru aplicații în care informația are caracter secvențial. Domenii precum procesarea limbajului natural (Natural Language Processing (NLP)), recunoașterea vorbirii sau analiza datelor în timp presupun existența unei dependențe între elementele succesive ale datelor, informația desfășurându-se în timp și nu într-o structură fixă bidimensională. Din acest motiv, utilizarea unor "ferestre" locale nu era suficientă pentru captarea relațiilor pe termen lung dintre elementele unei secvențe.

Primele încercări de rezolvare a acestei probleme au fost realizate prin intermediul rețelelor neuronale recurente (Recurrent Neural Network (RNN)), arhitecturi capabile să păstreze informația din pași anteriori prin intermediul unei stări interne. Astfel, modelul putea utiliza contextul anterior pentru a influența predicțiile curente, ceea ce reprezenta un progres important pentru procesarea datelor temporale. Cu toate acestea, antrenarea RNN clasice s-a confruntat rapid cu o problemă fundamentală a învățării profunde: dispariția gradientului.

Această problemă apărea în timpul procesului de propagare inversă prin timp (Backpropagation Through Time (BTT)), atunci când secvențele analizate deveneau foarte lungi. Gradientii calculați în timpul antrenării se diminuau exponențial pe măsura ce informația era propagată către începutul secvenței, devenind atât de mici încât actualizările ponderilor ajungeau neglijabile. În consecință, rețeaua "uita" informațiile importante de la începutul secvenței și nu putea învăța *dependente pe termen lung*.

Un progres major a fost realizat de Sepp Hochreiter și Jürgen Schmidhuber prin introducerea arhitecturii Long Short Term Memory (LSTM) în lucrarea [18]. Acest model a oferit rețelelor neuronale o formă funcțională de memorie pe termen lung, capabilă să păstreze informații relevante pentru perioade extinse de timp. Elementul central al acestei arhitecturi îl reprezintă mecanismul de "porți" care controlează fluxul informației în interiorul rețelei. Prin aceste mecanisme, modelul poate decide ce informații trebuie păstrate, ce informații trebuie actualizate și ce informații trebuie eliminate din memorie.

Datorită acestei capacități de gestionare eficientă a contextului temporal, rețelele LSTM au devenit fundamentul multor aplicații moderne de inteligență artificială, fiind utilizate cu succes în traducere automată, recunoaștere vocală, generare de text și analiză de secvențe temporale, deși problema dispariției gradientilor nu dispăruse complet. O comparație schematică între cele două arhitecturi este prezentată în Figura 1.3.

La pasul de timp t , dinamica unei celule LSTM poate fi descrisă prin trei porți (uitare,

intrare si iesire) si o stare de celula $\mathbf{c}_t$:

$$\begin{aligned}
 \mathbf{f}_t &= \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f), \\
 \mathbf{i}_t &= \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i), \\
 \mathbf{o}_t &= \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o), \\
 \mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c), \\
 \mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t),
 \end{aligned} \tag{1.2}$$

unde σ este functia sigmoid, $\odot$ produsul Hadamard, iar $[\cdot, \cdot]$ desemneaza concatenarea vectorilor. Stocarea informatiei in $\mathbf{c}_t$ printr-o cale aditiva (controlata de $\mathbf{f}_t$ si $\mathbf{i}_t$) este principalul motiv pentru care gradientii nu se diminueaza la fel de rapid ca in cazul unei RNN clasice.

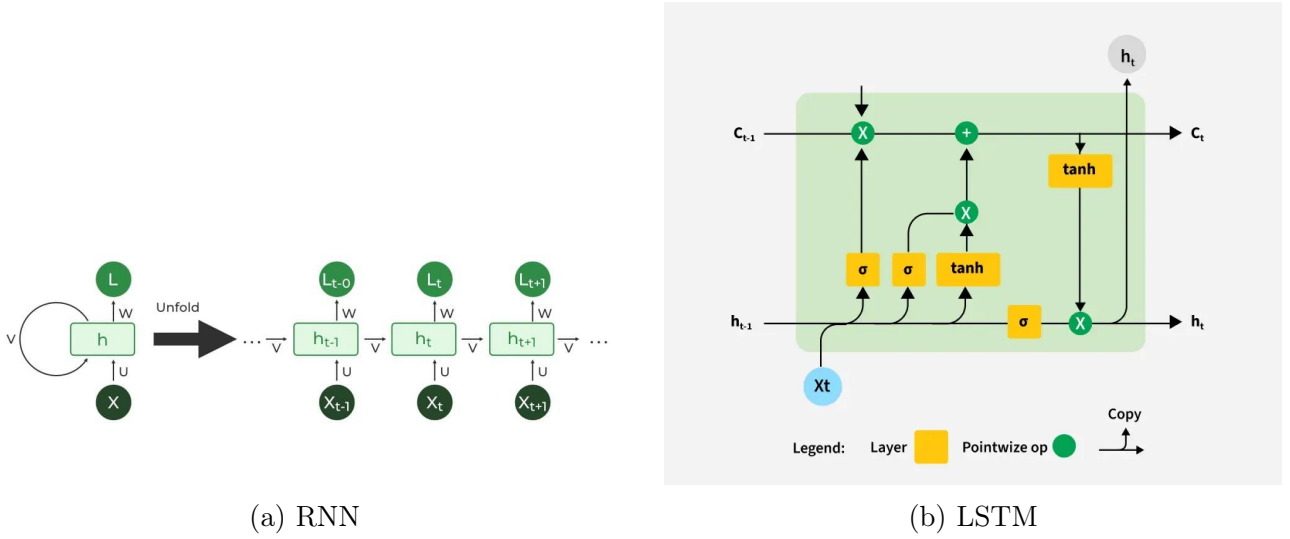


Figura 1.3: RNN si LSTM comparate (sursa)

Punctul de cotitura in evolutia inteligentei artificiale a fost reprezentat de introducerea arhitecturii "transformer", prezentata de Vaswani si colaboratorii sai in lucrarea devenita celebra [1]. Pana in acel moment, modelele bazate pe LSTM dominau procesarea limbajului natural, insa acestea prezentau o limitare importanta: datele secventiale trebuiau procesate pas cu pas, fiecare element depinzand de rezultatul precedent. Aceasta caracteristica facea antrenarea pe volume foarte mari de date sa fie lenta si dificil de paralelizat, limitand scalabilitatea modelelor.

Arhitectura transformer (Figura 1.4) a schimbat fundamental aceasta abordare prin introducerea mecanismului de **self-attention**. In loc sa proceseze secvential, modelul putea analiza simultan toate elementele unei propozitii si putea calcula relatiile dintre acestea indiferent de distanta lor in text. Astfel, fiecare cuvant primea informatii contextuale din intreaga secventa, ceea ce permitea captarea mult mai eficienta a dependentelor semantice si sintactice. In plus, eliminarea procesarii strict secventiale a permis paralelizarea operatiilor pe hardware modern precum Graphics Processing Unit (GPU), reducand semnificativ timpul de antrenare.

Mecanismul de *scaled dot-product attention* este definit, pentru matricile de interogari $\mathbf{Q} \in \mathbb{R}^{n \times d_k}$, chei $\mathbf{K} \in \mathbb{R}^{n \times d_k}$ si valori $\mathbf{V} \in \mathbb{R}^{n \times d_v}$, prin:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}, \tag{1.3}$$

unde factorul $1/\sqrt{d_k}$ stabilizeaza gradientii pentru valori mari ale dimensiunii cheilor.

Mai tarziu, mecanismul de "self-attention" s-a transformat in "multi-head attention" care

avea cate un cap de atentie pentru caracteristici specifice, putand astfel sa invete multiple tipuri de dependente intre date, fiecare cap specializandu-se pe un anumit tip mai mult sau mai putin abstract de dependenta intre date. Pe langa deblocarea intelegerii relatiilor complexe, Multi-Head Attention (MHA) a adus si o optimizare in timpul de antrenare si de inferenta, dar si la imbunatatirea generalizarii modelelor. Formal, pentru h capete de atentie:

$$\begin{aligned} \text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) &= \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O, \\ \text{head}_i &= \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V), \end{aligned} \tag{1.4}$$

cu matricile de proiectie $\mathbf{W}_i^Q, \mathbf{W}_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $\mathbf{W}_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ si $\mathbf{W}^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$.

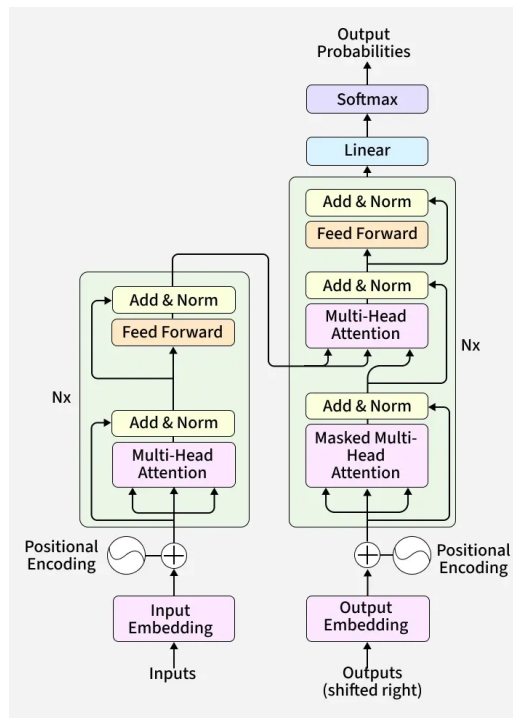


Figura 1.4: Arhitectura transformer [1]

Odata eliminata limitarea din punct de vedere computational, cercetatorii au putut antrena modele pe seturi de date de dimensiuni fara precedent. Acest progres a deschis drumul catre aparitia modelelor mari de limbaj, LLM. Acestea sunt capabile sa invete reprezentari complexe ale limbajului natural prin analiza unor cantitati uriase de text provenite din carti, articole, pagini web si alte surse disponibile pe internet. Transformerul a devenit rapid fundamentul majoritatii sistemelor moderne de inteligenta artificiala dedicate limbajului natural, influentand profund atat cercetarea academica, cat si aplicatiile comerciale.

Modelele mari de limbaj reprezinta rezultatul direct al antrenarii retelelor pe volume masive de date si folosind un numar foarte mare de parametri. Cercetatorii au observat ca performanta modelelor continua sa creasca odata cu extinderea dimensiunii acestora si a cantitatii de date utilizate la antrenare. Pornind de la aceasta idee, echipa OpenAI a dezvoltat modelul Generative Pre-trained Transformer (GPT)-3, care includea aproximativ **175 de miliarde de parametri**, descoperirile fiind prezentate in lucrarea „Language Models are Few-Shot Learners” [19]. La momentul aparitiei, GPT-3 reprezenta unul dintre cele mai mari si mai performante modele lingvistice create.

Una din cele mai importante realizari ale acestui model a fost capacitatea de a rezolva sarcini complexe fara a necesita o antrenare suplimentara specifica fiecarei probleme. Prin simpla furnizare a catorva exemple sau chiar doar a unei instructiuni formulate in limbaj natural, modelul putea genera cod sursa, realiza sumarizari de text, raspunde la intrebari, traduce texte sau crea continut creativ precum poezii si eseuri. Aceasta particularitate a fost documentata in articolul prezentat anterior a demonstrat ca modelele de mari dimensiuni pot generaliza cunostiinte intre domenii diferite si isi pot adapta comportamentul fara modificari suplimentare ale parametrilor interni.

Prin aceste progrese, inteligenta artificiala a facut tranzitia de la sisteme specializate, concepute pentru rezolvarea unei sarcini predefinite, la modele capabile sa manifeste competente generale avansate. Aceasta schimbare a reprezentat inceputul unei noi etape in dezvoltarea inteligentei artificiale moderne, in care accentul s-a mutat de la proiectarea unor solutii dedicate catre construirea unor modele inteligente universal, capabile sa interactioneze natural cu oamenii si sa rezolve probleme complexe.

1.2 Maturizarea modelelor mari de limbaj si cercetari actuale

Odata cu aparitia primelor foundation models ¹ precum GPT-4, GPT-5, Sonnet sau Opus, industria inteligentei artificiale a cunoscut o crestere exponentiala. Aceste modele prezinta o eficacitate impresionanta in rezolvarea problemelor din domenii diverse, de la generare si analiza textului, pana la programare, traduceri automate, automatizari complexe sau gandire logica. Datorita acestei flexibilitati, integrarea acestor modele a devenit o prioritate pentru majoritatea companiilor ce activau in domeniul tehnologiei.

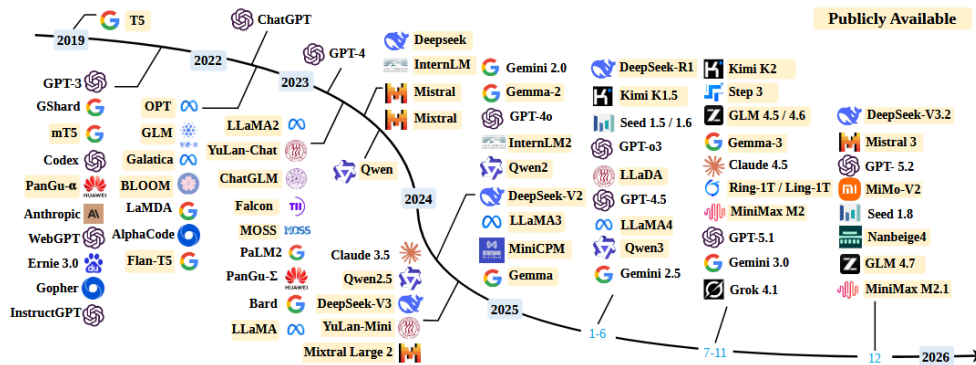


Figura 1.5: Evolutia recenta a modelelor mari de limbaj [2]

Astfel, cresterea rapida a cererii pentru sisteme bazate pe inteligenta artificiala a determinat furnizorii de astfel de modele si alte companii care ar beneficia de pe urma modelelor, sa investeasca masiv in cercetare si infrastructura. Aceste investitii au permis in final antrenarea unor modele cu capacitati din ce in ce mai sofisticate, fiecare noua generatie de modele aducand imbunatatiri semnificative in ceea ce priveste rationamentul, intelegerea limbajului si capacitatea de a rezolva probleme complexe.

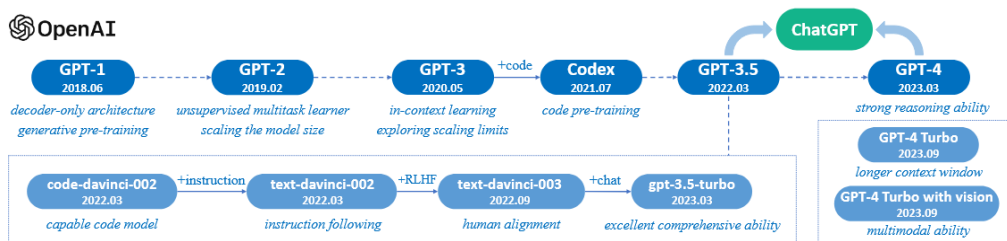


Figura 1.6: Evolutia modelelor GPT [2]

Investitiile in infrastructura nu au fost singurul motiv pentru care modelele de limbaj au avut o crestere impresionanta. Cercetarea a jucat de asemenea un rol crucial in optimizarea lor. Multe optimizari au fost aduse acestor modele de limbaj, printre care si imbunatatiri ale arhitecturii transformer care isi propun optimizeze modelele pentru procesarea secventelor mai lungi de date de intrare, precum este ilustrat in lucrarea "Longformer: The Long-Document Transformer" [3] de Beltagy si colaboratorii sai care isi propun sa optimizeze mecanismul de atentie, folosind serii trigonometrice pentru a stabili o compresie a cache-ului Key-Value (KV)

¹Un foundation model este un model mare de limbaj de uz general antrenat pe cantitati foarte mari de date diverse, ce poate fi adaptat pentru un numar mare de intrebuintari

² ³ imbunatatind astfel gandirea pe termen lung, lucru documentat in lucrarea "TriAttention: Efficient Long Reasoning with Trigonometric KV Compression" de Mao si colaboratorii sai [21].

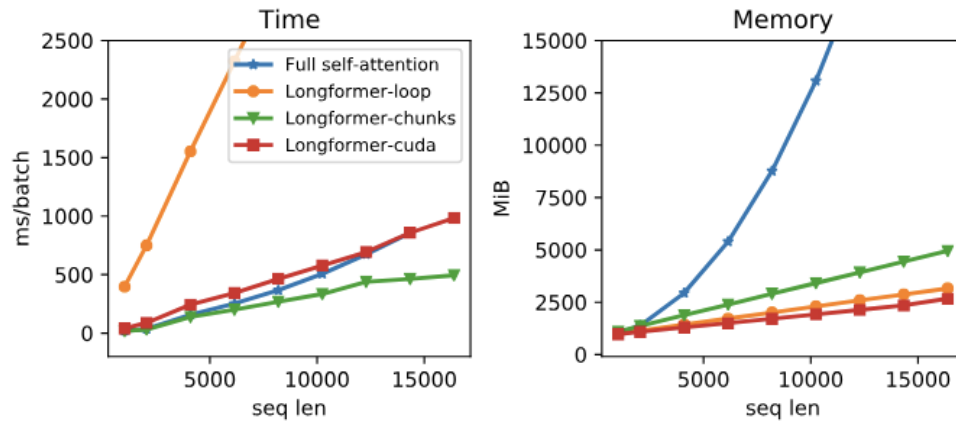


Figura 1.7: Performanta tipurilor de arhitecturi de tipul "Longformer" [3]. In timp ce mecanismul tipic de atentie utilizeaza memorie exponential, arhitectura Longformer foloseste memorie liniar odata cu cresterea contextului

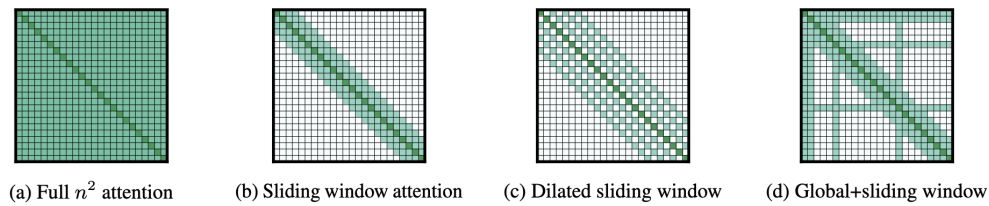


Figura 1.8: Diferite configuratii de atentie folosite in arhitectura Longformer in comparatie cu mecanismul obisnuit de atentie [3]

Inca un punct de optimizare al arhitecturii curente este imbunatatirea embedding-urilor, ⁴ precum studiul realizat in lucrarea "RoFormer: Enhanced Transformer with Rotary Position Embedding" [4] care isi propune sa optimizeze pozitia exacta a token-ului in secventa cu o matrice de rotatie in timp ce incorporeaza dependenta pozitiei relative in formula mecanismului de atentie, reusind astfel sa capteze informatii importante din secventa de intrare precum, lungimea secventei, relatii intre tokens care se diminueaza pe masura ce distanta relativa dintre ei creste, echipand astfel mecanismul de atentie cu o codare pozitionala relativa.

²Un token reprezinta datele de intrare pentru arhitectura transformer si uzual sunt cuvinte, parti din cuvinte sau caractere. Acestea permit modelului sa sparga un cuvint posibil mai rar in parti mai cunoscute.

³Cache-ul KV este o optimizare fundamentala a arhitecturii transformer ce elimina calculele redundante ale token-urilor anterioare in generarea autoregresiva, conform lucrarii "KV Cache Optimization Strategies for Scalable and Efficient LLM Inference" de Xu si colaboratorii [20]

⁴Embedding-ul este transformarea token-ului intr-un vector abstract multi-dimensional ce reprezinta informatia extrasa de model din acel token, plus un vector ce aduce informatii despre pozitia acestui in secventa.

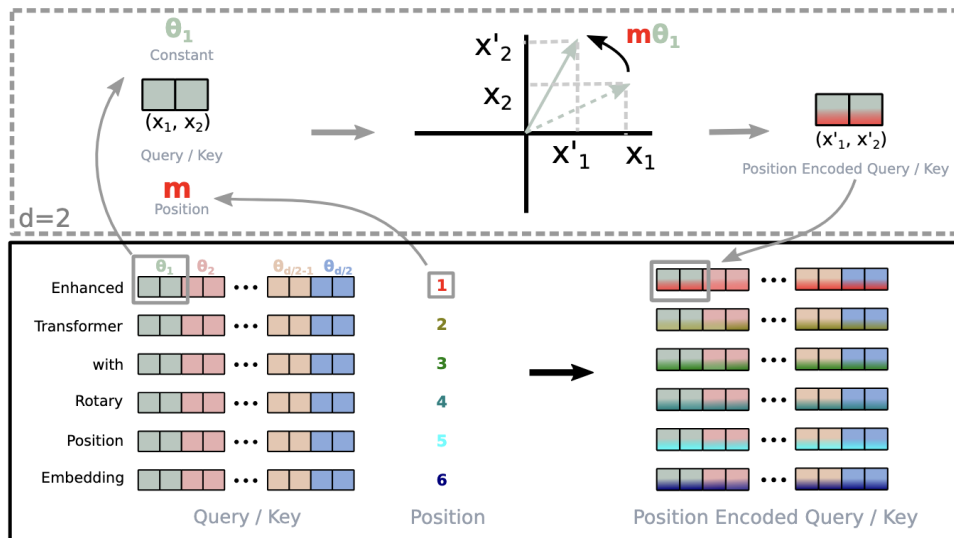


Figura 1.9: Principiul de baza pentru Rotary Position Embedding [4]

Imbunatatiri s-au adus si in partea de fine-tuning⁵ mai ales prin lucrarea LoRA: Low-Rank Adaptation of Large Language Models" [5] care aduce imbunatatiri prin inghetarea ponderilor modelului si injectarea unor matrici antrenabile de descompunere in fiecare strat al arhitecturii transformer, reducand numarul de parametrii antrenabili, reducand numarul acestor de **pana la 10.000 de ori** si puterea de calcul necesara de **3 ori** in comparatie cu fine-tuning-ul normal. Concret, pentru o matrice de ponderi pre-antrenata $\mathbf{W}_0 \in \mathbb{R}^{d \times k}$, actualizarea este aproximata printr-o descompunere de rang mic:

$$\mathbf{W} = \mathbf{W}_0 + \Delta\mathbf{W} = \mathbf{W}_0 + \mathbf{B}\mathbf{A}, \quad \mathbf{B} \in \mathbb{R}^{d \times r}, \mathbf{A} \in \mathbb{R}^{r \times k}, r \ll \min(d, k), \quad (1.5)$$

unde doar $\mathbf{A}$ si $\mathbf{B}$ sunt antrenate, in timp ce $\mathbf{W}_0$ ramane inghetata.

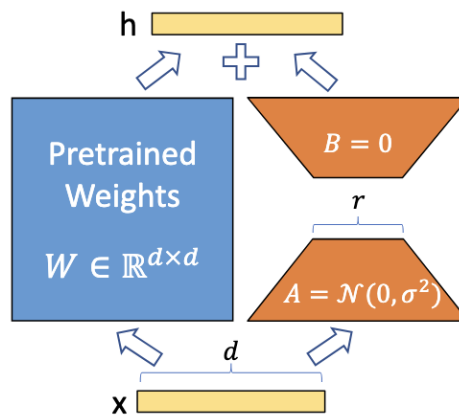


Figura 1.10: Fine-tuning-ul se realizeaza doar pe A si B [5]

Performanta a vazut de asemenea progrese, spre exemplu prin procedeul de decodare speculativa prezentata in articolul "Fast Inference from Transformers via Speculative Decoding"

⁵Fine tuning-ul este un procedeu prin care parametrii unui model sunt ajustati pentru eficienta sporita intr-un anumit domeniu.

[22] care ilustreaza problema timpului de inferenta ⁶ care continua sa creasca pe masura ce marimea modelelor creste si ea. Autorii propun un algoritm care sa permita livrarea mai multor tokeni in paralel. La baza lucrarii stau doua observatii fundamentale: (1) majoritatea sarcinilor abordate de modelele mari de limbaj pot fi scindate in obiective mai usoare care pot fi abordate de modele mai eficiente si (2) folosind executia speculativa si o noua metoda de esantionare, decodarea poate fi accelerata rulandu-le in paralel pe baza iesirilor modelele mai eficiente ce aproximeaza pe cat de bine pot, generand mai multi tokeni simultan fara a modifica distributia.

Continuand incercarile de a eficientiza modelele mari de limbaj, a aparut intrebarea: "Daca marimea modelul incetinesc antrenarea si inferenta, de ce nu cautam sa micșorăm modelul?". Astfel, au aparut procedeele de cuantizare ⁷ si distilare ⁸. Problema distilării este nevoia unui model superior care sa poarte rolul de "invatator". Zhang si echipa sa de cercetatori documenteaza o alternativa a acestei metode prin procedeul de distilare proprie in lucrarea "Embarrassingly Simple Self-Distillation Improves Code Generation" [6] prin esantionarea cu anumite valori ale temperaturii ⁹ si configuratii ale trunchierii, in final executand fine-tuning supervizat ¹⁰ pe acele esantioane.

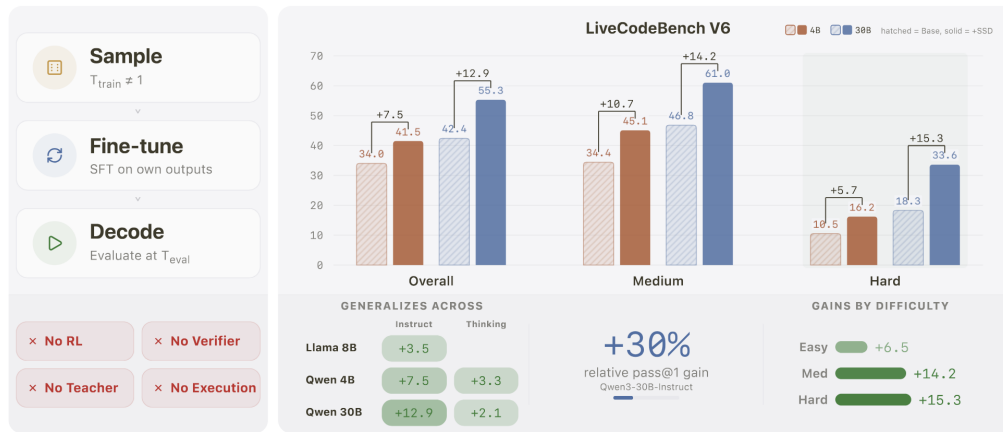


Figura 1.11: Rezultatele procesului de Simple Self Distillation [6]

⁶Inferenta este procedeul prin care un model de limbaj ia datele de intrare, aplica transformările necesare si produce un raspuns final.

⁷Cuantizarea este reducerea tipului de date care stocheaza parametrii rețelei. Spre exemplu daca in loc de 8 biti, stocam parametrii pe 4 biti, injumatatim amprenta modelului

⁸Distilarea este antrenarea unui model mai mic prin obiectivul reproducerii raspunsurilor modelului mai mare

⁹Temperatura este o setare pentru modelele de limbaj care controleaza "creativitatea" modelului. O temperatura mica (spre exemplu 0) obliga modelul sa scoata la iesirea sa cel mai probabil token posibil, pe cand o temperatura mare (spre exemplu 0.9), permite modelului sa aleaga un alt token probabil.

¹⁰Supervizarea se refera la abilitatea modelului de a folosi exemple etichetate corect de oameni in prealabil pentru antrenare. Antrenarea supervizata nu include raspunsurile corecte, iar modelul trebuie sa descopere singur tiparele ce reprezinta raspunsul corect

LiveCodeBench v6 02/2025 - 05/2025									LiveCodeBench v5 08/2024 - 02/2025								
	PASS@1				PASS@5					PASS@1				PASS@5			
	ALL	EASY	MED	HARD	ALL	EASY	MED	HARD		ALL	EASY	MED	HARD	ALL	EASY	MED	HARD
THINKING																	
Qwen3-4B-Thinking	54.5	97.4	59.2	29.7	67.5	100.0	75.4	45.8	59.6	98.0	69.5	31.4	70.3	98.9	80.1	47.5	
+ SSD	57.8 +3.3	98.7 +1.3	62.8 +3.6	33.8 +4.1	71.4 +3.9	100.0	77.3 +1.9	53.1 +7.3	63.1 +3.5	98.5 +0.5	74.1 +4.6	35.7 +4.3	74.7 +4.4	98.9	86.8 +6.7	52.8 +5.3	
Qwen3-30B-Thinking	66.1	99.8	73.0	41.5	76.7	100.0	84.7	59.7	69.1	98.6	81.7	43.9	79.8	98.9	91.4	61.1	
+ SSD	68.2 +2.1	100.0 +0.2	76.4 +3.4	46.7 +5.2	80.2 +3.5	100.0	87.1 +2.4	65.8 +6.1	69.9 +0.8	98.4 -0.2	82.9 +1.2	44.9 +1.0	80.5 +0.7	98.9	91.8 +0.4	62.4 +1.3	
INSTRUCT																	
Llama-3.1-8B-Instruct	12.7	46.8	5.4	0.0	23.0	76.7	15.1	0.8	12.7	45.1	4.1	0.7	20.8	64.7	11.6	2.8	
+ SSD	16.2 +3.5	52.6 +5.8	10.0 +4.6	1.6 +1.6	24.0 +1.0	73.3 -3.4	17.1 +2.0	3.3 +2.5	13.5 +0.8	45.2 +0.1	5.9 +1.8	1.3 +0.6	21.9 +1.1	67.1 +2.4	12.3 +0.7	3.7 +0.9	
Qwen3-4B-Instruct	34.0	79.7	34.4	10.5	41.0	90.0	40.5	16.5	34.3	85.4	28.4	10.1	45.4	96.0	45.4	17.5	
+ SSD	41.5 +7.5	86.5 +6.8	45.1 +10.7	16.2 +5.7	56.8 +15.8	98.1 +8.1	59.7 +19.2	34.1 +17.6	45.7 +11.4	98.2 +4.8	52.1 +23.7	16.6 +6.5	61.9 +16.5	98.8 +2.8	73.1 +27.7	33.6 +16.1	
Qwen3-30B-Instruct	42.4	84.5	46.8	18.3	53.5	93.3	56.8	31.1	45.8	91.2	49.8	17.7	58.7	96.1	69.2	38.6	
+ SSD	55.3 +12.9	91.0 +6.5	61.0 +14.2	33.6 +15.3	71.6 +18.1	99.9 +6.6	76.4 +19.6	54.1 +23.0	54.3 +8.5	92.9 +1.7	64.1 +14.3	25.9 +8.2	70.7 +12.0	97.6 +1.5	83.0 +13.8	47.2 +16.6	

Figura 1.12: Imbunatatiri in sarcini de programare ale modelelor dupa ce au fost supuse procesului de Simple Self Distillation[6]

Pe de alta parte, problema cuantizarii este dezbatuta in lucrarea "TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate" [23] ce isi propune sa cuantizeze vectori multi-dimensional fara distorsiuni majore ale datelor. Acest lucru este posibil prin transformari ale vectorilor de intrare, precum rotatii aleatoare inducand o distributie "Beta" si folosindu-se de independenta aproximativa ale coordonatelor distincte in vectorii multi-dimensional. Se reuseste astfel minimizarea memoriei necesare, a timpului de inferente si a latimii de banda necesare pentru rularea modelului prin reducerea preciziei numerice. Acesta este eficient si in cazul ponderilor modelului, dar este foarte important si in cache-ul KV deoarece permite rularea modelelor cu ferestre de context ¹¹ lungi, optimizand necesitatile de memorie. Odata cu aceasta descoperire, au venit adaptari ale procedeelor anterioare, precum LoRA [5] in cazul modelelor cuantizate. O astfel de lucrare este "QLoRA: Efficient Finetuning of Quantized LLMs" [7] care isi propune sa imbunatateasca fine-tuning-ul pentru modele cuantizate.

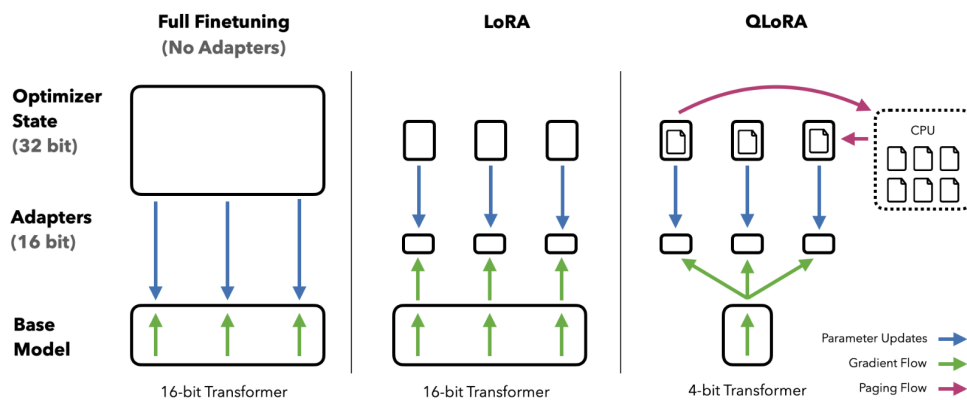


Figura 1.13: Comparatia procedeelor de fine-tuning: fine-tuning clasic vs LoRA vs QLoRA [7]

Pe langa optimizarile aduse arhitecturii transformer, o buna parte din evolutia gandirii adanci se bazeaza pe procedeul de Reinforcement Learning (RL). Pe scurt, modelul este recompensat mai mult sau mai putin in functie de raspunsurile pe care le da, iar obiectivul sau

¹¹Contextul se refera la informatia pe care modelul o are disponibila pentru a gandi un raspuns la cererea utilizatorului

este sa fie recompensat cat mai mult. Raspunsurile care prezinta gandire, mai exact Chain-of-Thought (CoT) sunt recompensate mai mult decat cele ce nu prezinta aceasta particularitate, iar raspunsurile gresite, care nu sunt fundamentate sau contin fraze nedorite sunt penalizate. In felul acesta, modelele invata sa prefere sa scoata date de iesire care prezinta particularitati ale gandirii, care duc in final la raspunsuri mai logice si capabilitatea de a rezolva probleme mai complexe, avand in vedere functionarea decodurului din transformer care genereaza pas cu pas, in functie de pasii anteriori. In lucrarea "HARDTESTGEN: A High-Quality RL Verifier Generation Pipeline for LLM Algorithmic Coding" [24], de exemplu e discutata importanta RL si a modalitatii de a avea semnale mai bune de recompensa pentru aplicatii de programare prin generarea de cazuri de test complexe, pe care verificatorul le evalueaza si recompenseaza modelul in functie de raspunsul dat.

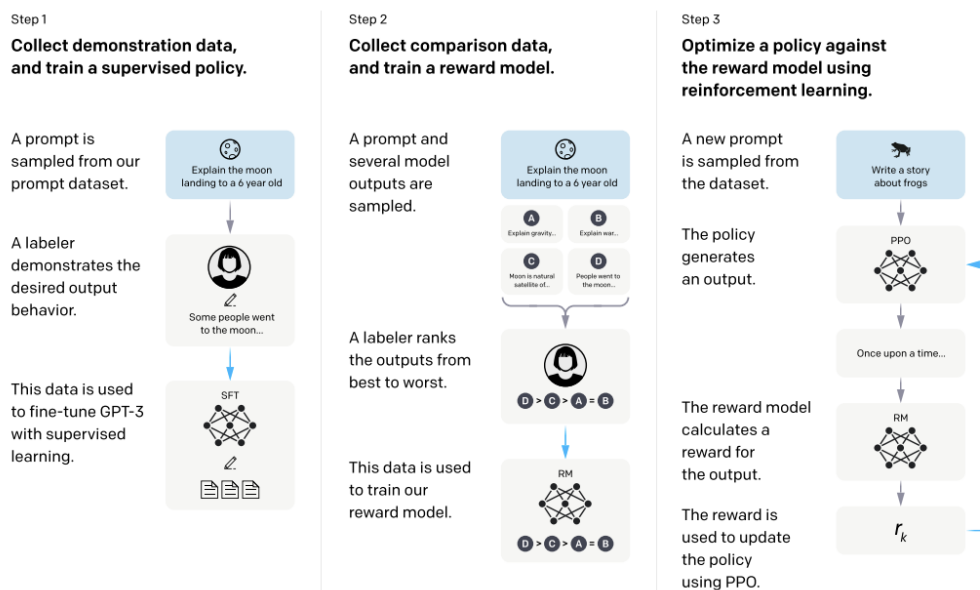


Figura 1.14: Ilustrarea procedeului de RL [8]

In final, o alta latura care a beneficiat de atentie sporita este prompt-ul ¹² pe care il primeste modelul. Modelele de limbaj, fiind autoregresive adica generand date de iesire pe baza datelor anterioare sunt dependente de un prompt bine gandit care maximizeaza contextul pe care il are modelul si care ii impune limite si directive pe care sa le respecte. Importanta acestui prompt fiind accentuata si de o noua arie de specializare, de "Prompt Engineering" ce presupune constructia unor prompt-uri eficiente, lucru ilustrat in lucrarea "The Prompt Report: A Systematic Survey of Prompt Engineering Techniques" [25].

Avand in vedere nevoia de prompt-uri ce produc rezultate consistente si versionabile, echipa condusa de Khattab a publicat lucrarea "DSPy: Compiling Declarative Language Model Calls into State-of-the-Art Pipelines" [26], ce propune o abordare sistematica pentru a crea si optimiza prompt-uri. Au introdus astfel un model de programare ce abstractizeaza aceste prompt-uri prin grafuri de transformari textuale sau grafuri computationale imperative si un compilator care functioneaza cu acest nou limbaj, rezultatele fiind instructiuni optimizate ce intrec performantele prompt-urilor scrise de mana.

¹²Prompt-ul este reprezentat de datele de intrare ale modelelor de limbaj, instructiunile pentru care trebuie sa produca un raspuns la iesire

1.3 Solutii agentice pentru industria dezvoltarii software

Arhitecturile agentice presupun inzestrarea modelelor mari de limbaj cu uneltele necesare interactionarii cu ecosistemul in care ruleaza. Fundamentul acestor arhitecturi a fost pus de procedeul CoT [9], precedat mai tarziu de Tree-of-Thought (ToT) [10] care a demonstrat ca un model de limbaj poate naviga probleme complexe prin descompunerea rationamentului in pasi intermediari expliciti. Aceasta tehnica a deschis calea catre generarea de planuri structurate, iar Wang si colaboratorii sai au extins aceasta idee prin metoda Plan-and-Solve [27], in care modelul este instruit sa conceapa mai intai un plan de actiune ce divide sarcina in sub-obiective mai mici, iar apoi sa le execute conform planului stabilit. Abordarea a reusit sa elimine erorile de pasi lipsa frecvente in CoT clasic, obtinand performante comparabile cu promptarea few-shot pe probleme de rationament matematic, fara a necesita exemple manuale.

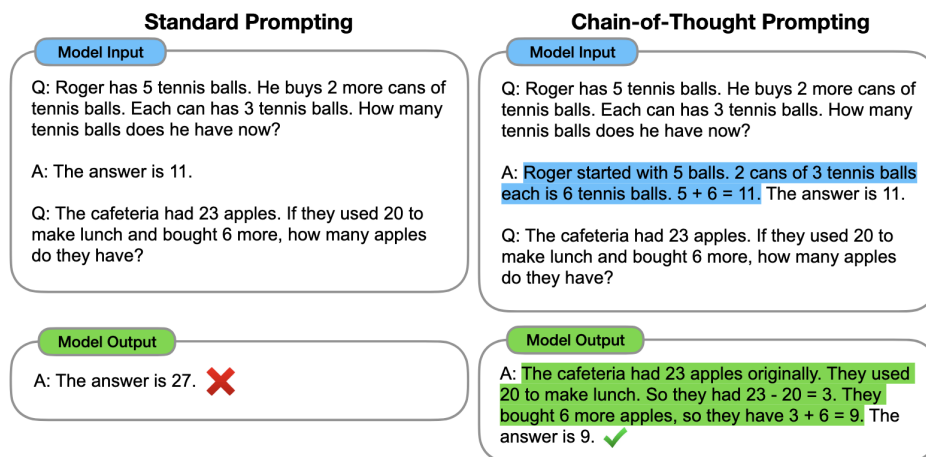


Figura 1.15: Exemple de CoT [9]

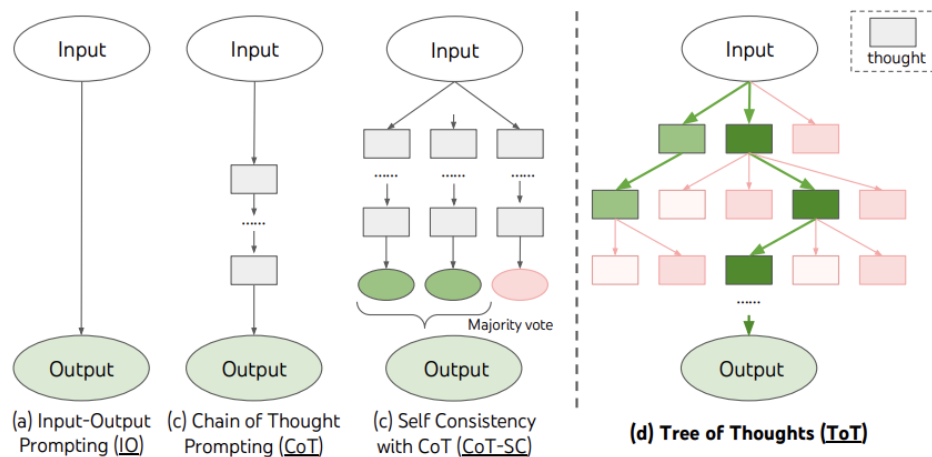


Figura 1.16: Diferenta intre procedee de gandire [10]

Mai tarziu, abordarea ReAct [11] a combinat rationamentul cu actiunea, permitand modelului sa alterneze intre generarea de ganduri intermediare si executia de actiuni concrete in mediul extern. La nivel operational, bucla ReAct urmeaza un tipar de tipul *Thought* $\rightarrow$ *Action* $\rightarrow$ *Observation*, repetat pana la producerea raspunsului final:

Question: In ce limbaj este scris proiectul X?

```

Thought: Trebuie sa inspectez fisierele din repository.
Action: list_files(repo="X")
Observation: ["main.py", "utils.py", "README.md"]
Thought: Extensia .py indica Python.
Action: finish("Python")

```

Listing 1.1: Tipar simplificat al buclei ReAct.

Aceasta sinergie intre gandire si actiune a incercat sa combata fenomenul halucinatiilor¹³ si inconsistenta in raspunsuri cu care se confruntau primele modele mari de limbaj. Totusi, paradigma ReAct presupunea o intercalare costisitoare intre rationament si observatii: modelul genera un gand, apela o unealta, astepta raspunsul, apoi decidea urmatoarea actiune pe baza tuturor token-urilor anterioare.

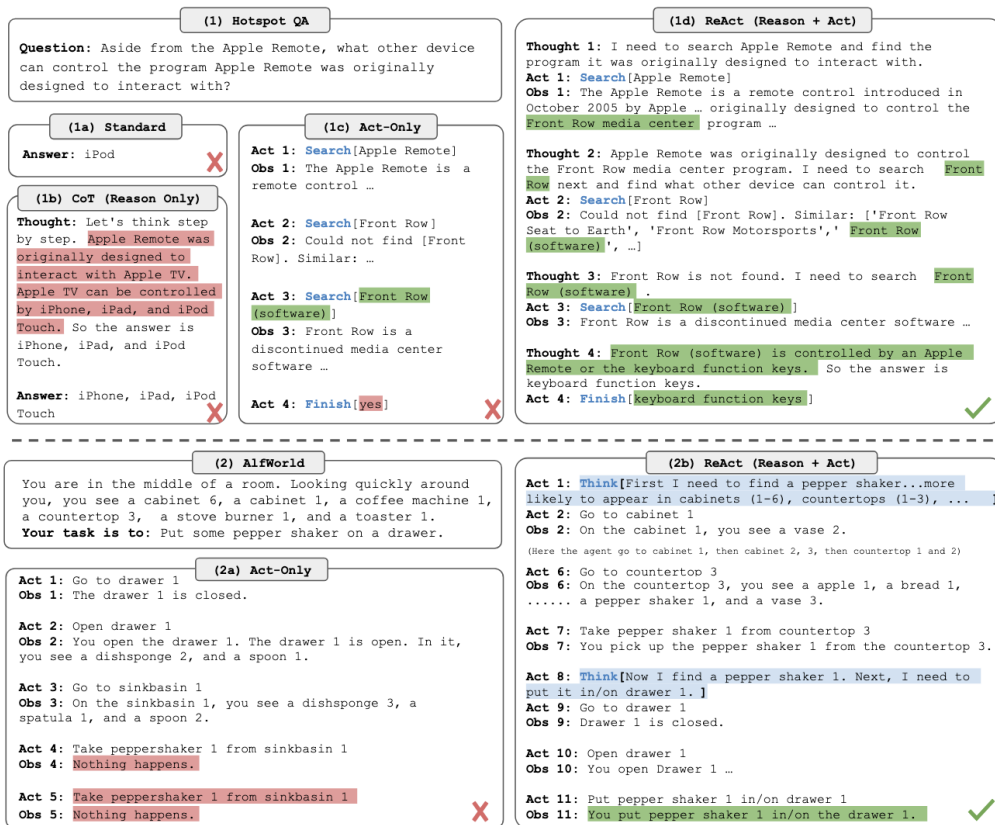


Figura 1.17: Paradigma ReAct [11]

	Type	Definition	ReAct	CoT
Success	True positive	Correct reasoning trace and facts	94%	86%
	False positive	Hallucinated reasoning trace or facts	6%	14%
Failure	Reasoning error	Wrong reasoning trace (including failing to recover from repetitive steps)	47%	16%
	Search result error	Search return empty or does not contain useful information	23%	-
	Hallucination	Hallucinated reasoning trace or facts	0%	56%
	Label ambiguity	Right prediction but did not match the label precisely	29%	28%

Figura 1.18: Imbunatatirile pe care le aduce ReAct [11]

Xu si colaboratorii sai au identificat aceasta ineficienta si au propus arhitectura ReWOO [12] care decupleaza complet procesul de rationament de observatiile externe. In loc sa astepte

¹³Fenomenul de halucinatii in modelele de limbaj se refera la generarea unor raspunsuri care nu sunt ancorate in fapte reale sau care nu au legatura cu prompt-ul de intrare

raspunsul fiecarei unelte inainte de a planifica urmatorul pas, modelul genereaza intregul plan de actiuni dintr-o singura trecere, iar observatiile sunt colectate independent si integrate ulterior. Aceasta separare a redus consumul de tokeni **de pana la 5 ori** si a imbunatatit acuratetea cu **4%** pe benchmark-ul HotpotQA, demonstrand totodata robustete crescuta in scenarii de esec al uneltelor.

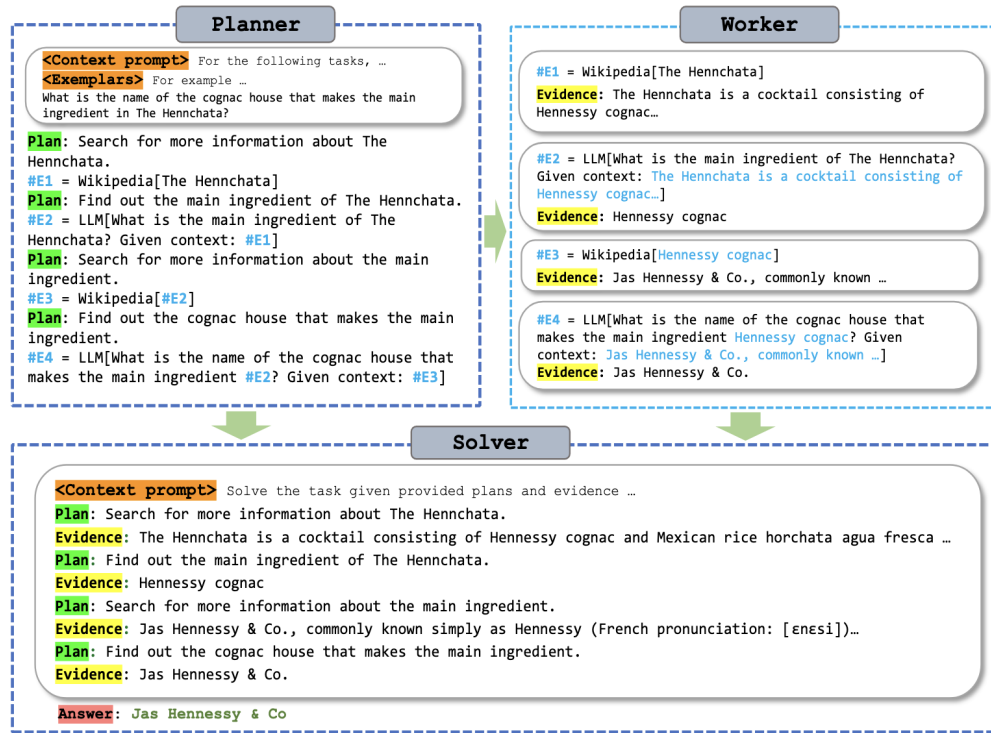


Figura 1.19: Ideea din spatele ReWOO [12]

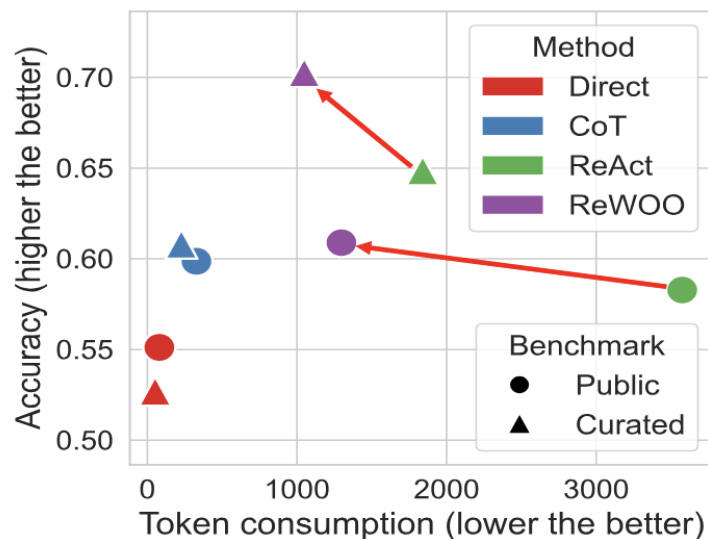


Figura 1.20: Comparatia de performante intre paradigme pentru imbunatatirea gandirii din punctul de vedere al acuratetii vs. numarul de tokeni consumati [12]

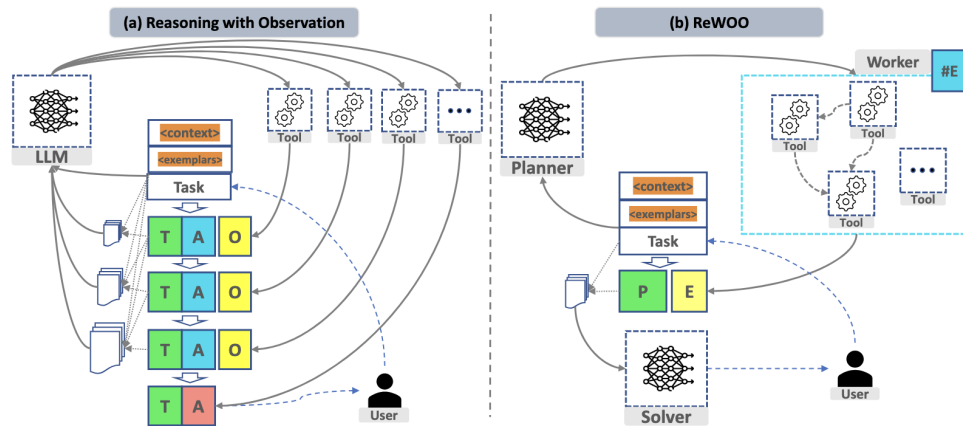


Figura 1.21: Imbunatatirea adusa de ReWOO este delegarea mai multor libertati catre worker ce poate apela mai multe unelte pana sa produca un raspuns. P = Plan, E = Evidence, A = Answer, O = Observation, T = Thought [12]

Increderea in procesul de gandire al modelelor de inteligenta artificiala s-a cimentat pe masura ce aceste tehnici au inceput sa produca rezultate consistente, iar astfel a urmat inezestrarea acestora cu unelte care sa le permita interactiunea cu mediul in care ruleaza. Numarul uneltelor ce le ofera capacitatea de a cauta pe internet pentru a gasi date actuale si a nu se limita la datele de antrenare, la posibilitatea rularii de comenzi in terminalul sistemului de operare sau chiar preluarea controlului browserului personal pentru automatizari, a crescut exponential, si odata cu el si numarul de aplicatii ale acestor modele.

Cu toate acestea, pe masura ce agentii au inceput sa proceseze cantitati din ce in ce mai mari de informatii din mediul in care opereaza, fereastra de context a devenit o limitare critica. Desi, multe modele sunt promovate ca avand o fereastra de context maxima de 1 milion de tokeni precum Opus 4.8 de la Anthropic sau GPT-5.5 de la OpenAI, modelele se lovesc de anumite limitari in a accesa tot acel context. Liu si colaboratorii sai au demonstrat in lucrarea "Lost in the Middle" [13] ca *performanta modelelor de limbaj se degradeaza semnificativ atunci cand informatia relevanta se afla in mijlocul unui context lung*. Modelele tind sa utilizeze eficient informatiile de la inceputul si de la sfarsitul secventei de intrare, dar pierd din vedere elementele plasate in zona centrala, chiar si in cazul modelelor concepute explicit pentru contexte lungi (Figura 1.22). Aceasta descoperire are implicatii directe asupra agentilor care acumuleaza istoricul conversatiei, apelurile la unelte si observatiile intr-un context tot mai voluminos.

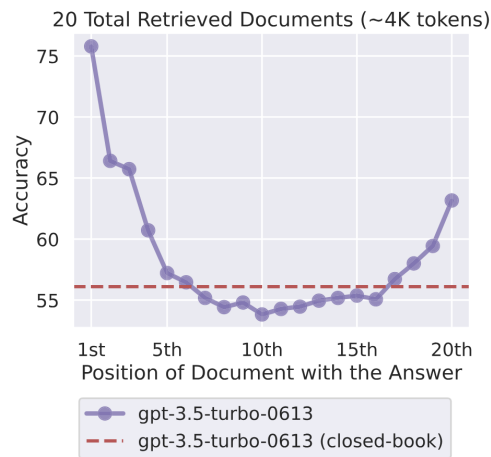


Figura 1.22: Modele tind sa aiba o acuratete foarte slaba contra intrebarelor ce vizeaza informatii care se afla la mijlocul ferestrei curente de context [13]

Input Context

Write a high-quality answer for the given question using only the provided search results (some of which might be irrelevant).

Document [1] (Title: Asian Americans in science and technology) Prize in physics for discovery of the subatomic particle J/ψ. Subrahmanyan Chandrasekhar shared...

Document [2] (Title: List of Nobel laureates in Physics) The first Nobel Prize in Physics was awarded in 1901 to Wilhelm Conrad Röntgen, of Germany, who received...

Document [3] (Title: Scientist) and pursued through a unique method, was essentially in place. Ramón y Cajal won the Nobel Prize in 1906 for his remarkable...

Question: who got the first nobel prize in physics

Answer:

Desired Answer

Wilhelm Conrad Röntgen

Figura 1.23: Vizualizarea problemei si al rezultatului dorit la intrebare. De multe ori, modelele intampina dificultati in accesarea acestor informatii ascunse in mijlocul ferestrei de context [13]

Pentru a evalua riguros aceste limitari, Vodrahalli si echipa sa au propus framework-ul Michelangelo [14], o metoda de evaluare sintetica si minimala care depaseste testele clasice de tip 'needle-in-a-haystack'. In loc sa masoare doar capacitatea de a regasi o singura informatie dintr-un context lung, Michelangelo testeaza abilitatea modelului de a sintetiza si corecta informatii dispersate, relevand o structura latentă din datele de intrare. Rezultatele au confirmat ca exista un spatiu semnificativ de imbunatatire in procesarea informatiei din contexte lungi, chiar si pentru modelele de ultima generatie.

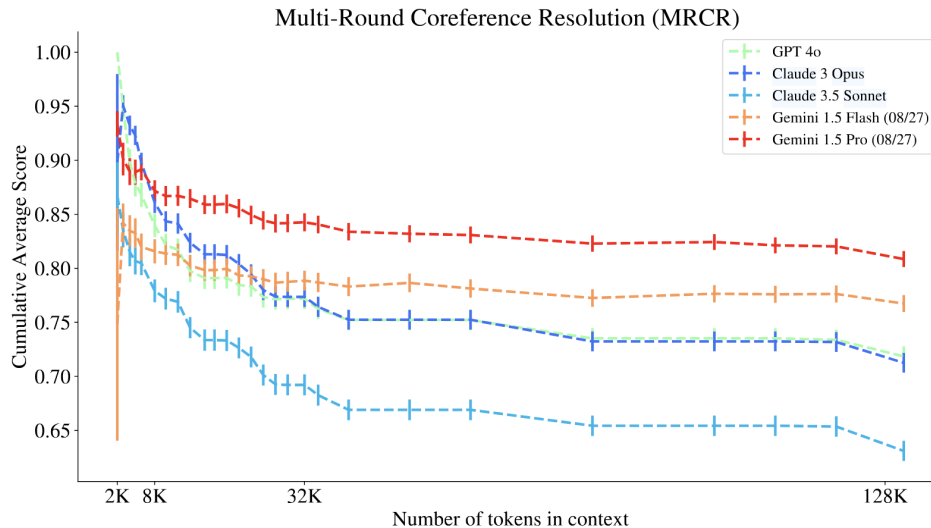


Figura 1.24: In majoritatea modelelor se observa o scadere a performantelor dupa 32K tokeni pe sarcini de gandire complexa[14]

Din punct de vedere al eficientei computationale, Li si colaboratorii sai au propus Lychee-Cluster [28], o metoda de gestionare a cache-ului KV care pastreaza coerenta semantica locala prin fragmentare bazata pe delimitatori si construiesc un index ierarhic recursiv. Aceasta abordare transforma cautarea in cache dintr-o scanare liniara intr-un proces de eliminare in timp logarithmic, obtinand o accelerare de **pana la 3.6 ori** a inferentei cu o degradare neglijabila a performantelor modelului. Pentru aplicatiile agentice care implica conversatii lungi cu apeluri frecvente la unelte, astfel de optimizari sunt esentiale pentru a mentine latentia scazuta si costurile sub control.

Industria dezvoltarii software are probabil una din cele mai profitabile intrebuintari ale acestor arhitecturi agentice, datorita volumului masiv de munca existent in domeniu, asa ca nu a durat mult pana ce marii jucatori pe piata modelelor de IA au inceput sa ofere produse care sa asiste programatorii in sarcinile lor. La momentul actual exista mai multe solutii competitive: Codex de la OpenAI, Gemini Command Line Interface (CLI) de la Google sau Claude Code de la Anthropic. Toate acestea sunt solutii proprietare care sunt facute sa ruleze cu modelele proprii si pentru care utilizatorul trebuie sa plateasca. Exista si solutii open-source precum OpenCode, Kiro sau OpenClaw cu care detinatorii de modele se lupta pentru a domina piata, desi lupta nu este stransa din cauza monopolului solutiilor proprietare.

Companie	Model	Pret [\$/MTok]	Capabilitati
Anthropic (link)	Claude Opus 4.8	\$5/input MTok, \$25/output MTok	Cel mai capabil model disponibil, imbunatatire semnificativa pentru programare agentica fata de 4.6 si 4.7, gandire adaptiva, 1M fereastră de context, 128k tokens max. la iesire
	Claude Opus 4.6	\$5/input MTok, \$25/output MTok	Lent, inteligent, gandire adaptiva, 1M fereastră de context, 128k tokens max. la iesire
	Claude Sonnet 4.6	\$3/input MTok, \$15/output MTok	Mai rapid, compromite niste inteligenta pentru performante, gandire adaptiva, 1M fereastră de context, 64k tokens max. la iesire
	Claude Haiku	\$1/input MTok, \$5/output MTok	Cel mai rapid, fara gandire adaptiva, 200k fereastră de context, 64k tokens max. la iesire
OpenAI (link)	gpt-5.5	\$5/input MTok, \$30/output MTok	Noua clasa de inteligenta pentru programare si munca profesionala, gandire adaptiva cu niveluri multiple de efort (low/medium/high/xhigh), suport multimodal, 1M fereastră de context si 128k tokens max. la iesire
	gpt-5.4	\$2.5/input MTok, \$15/output MTok	Cel mai inteligent pentru sarcini agentice, de programare si profesionale. 1M fereastră de context si 128k tokens max. la iesire
	gpt-5.4-mini	\$0.75/input MTok, \$4.5/output MTok	Model mai mic, rapid dar puternic pentru programare si subagenti. 400k fereastră de context si 128k tokens max. la iesire
	gpt-5.4-nano	\$0.20/input MTok, \$1.25/output MTok	Model mic si foarte rapid pentru sarcini usoare in volum mare. 200k fereastră de context si 128k tokens max. la iesire
Google (link)	Gemini 3.1 Pro	\$2/input MTok, \$12/output MTok < 200k tokens, \$4/input MTok, \$118/output MTok >= 200k tokens	Model pentru sarcini complexe ce necesita cunostiinte dezvoltate. Fereastră de context 1M in / 64k out
	Gemini 3.1 Flash	\$0.5/input MTok, \$3/output MTok	Inteligenta comparabila cu modelul Pro la o viteza mai mare. Fereastră de context 1M in / 64k out
	Gemini 3.1 Flash-Lite	\$0.25/input MTok, \$1.5/output MTok	Model construit pentru costuri reduse si volum mare de sarcini. Fereastră de context 1M in / 64k out
	Gemma 4	Open Source	Familie de modele open-source pentru rulare locala, cost redus si personalizare
DeepSeek (link)	DeepSeek V4	\$0.30/input MTok, \$0.50/output MTok	Model general de ultima generatie, suporta mod cu si fara gandire adanca, 1M fereastră de context, cache hit: \$0.03/input MTok
	DeepSeek R1	\$0.55/input MTok, \$2.19/output MTok	Model specializat in rationament si gandire adanca pentru probleme complexe, 128k fereastră de context, cache hit: \$0.14/input MTok
Meta (link)	Llama 4	Open Source	Performante similare cu GPT-4o sau Gemini 2.0 Flash, dar open-source. Fereastră de context 10M/1M in functie de model.
Qwen/Alibaba (link)	Qwen 3	Open Source	Familie de modele open-source (Apache 2.0) cu arhitectura hibrida Mixture of Experts (MoE), gandire adaptiva activabila, performante frontier la programare si rationament. Disponibil in variante de 0.6B pana la 235B parametri
	Qwen 3.6 Plus	\$0.325/input MTok, \$1.95/output MTok	Varianta Application Programming Interface (API) proprietara (Alibaba Cloud), arhitectura hibrida MoE cu atentie liniara, gandire adaptiva, 1M fereastră de context, 64k tokens max. la iesire
Mistral AI (link)	Mistral Large 3	\$0.50/input MTok, \$1.50/output MTok	Cel mai capabil model Mistral, open-weight, multimodal, performante de nivel frontier. 262k fereastră de context
	Mixtral 8x22B	Open Source	Model Mixture-of-Experts open-source (Apache 2.0), 8 experti de 22B parametri fiecare, disponibil pentru rulare locala sau prin API. 66k fereastră de context

Tabela 1.1: Comparatia catorva din modelele existente si a capabilitatile acestora in comparatie cu pretul de utilizare

Nota: Toate modelele au si versiuni pentru diferite nivele de efort pentru gandire (reasoning effort) sau pentru date multimodale ce pot influenta si costul dar si rezultatele

Din tabelul 1.1 se poate deduce ca utilizarea frecventa a unui model de ultima generatie poate ajunge cu usurinta sa coste o mica avere. Cu o mica aproximare, 200.000 de tokens sunt echivalentul unei carti de 400 de pagini. Dar contorizarea nu este facuta doar pentru instructiunile originale. Intr-o conversatie cu un model de limbaj fiecare pereche raspund-intrebare din istoricul conversatiei respective este introdus ca date de intrare pentru urmatorul prompt pe care utilizatorul il introduce. Astfel, in loc de cativa tokeni de intrare, intr-o conversatie lunga cu un istoric dezvoltat, costul de intretinere al conversatiei creste exponential.

De asemenea, fiecare apel catre unelte externe precum cautarile web, raman in istoricul conversatiei pentru ca modelul sa aiba context in query-urile ulterioare. Fiecare dintre aceste apeluri la unelte consuma o buna parte din fereastra de context a modelului, crescand costul.

Din punct de vedere al capabilitatii de rezolvare a sarcinilor reale din industria dezvoltarii software, clasamentul celor mai bune modele se schimba frecvent. Utilizand SWE-Bench Pro [15], putem masura puterea de abordare a acestor modele de top contra unor sarcini complexe (Figura 1.25), iar Figura 1.26 ofera o perspectiva complementara, in functie de cost.

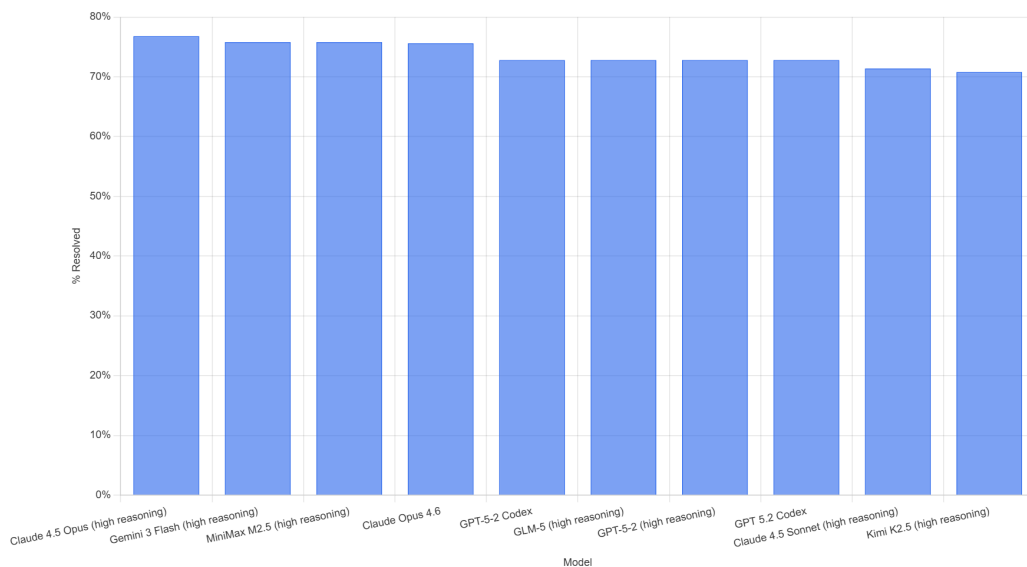


Figura 1.25: Clasamentul modelelor in SWE-Bench Pro [15] la momentul actual in functie de performanta rezolvarii sarcinilor complexe in limba engleza

Pe langa puterea de rezolvare, un alt fenomen a aparut care schimba rata de adoptie a unui model: costul. Desi performantele modelelor cresc si acestea sunt din ce in ce mai integrate in vietile noastre de zi cu zi, asemenea telefoanelor mobile, costul de rulare al acestor modele creste si el. Cele mai puternice modele au nevoie de foarte multe resurse pentru antrenare dar si pentru inferenta. Acest lucru se reflecta asupra utilizatorului final care trebuie sa plateasca per-token sau un abonament lunar care include un numar limitat de tokeni. Modelele cu capacitati de gandire adanca, pe langa faptul ca au un cost mai mare per-token, consuma si mai multi tokeni din cauza gandirii dezvoltate sau a ferestrei de context mari, ridicand si mai mult costurile. Pentru asta, fiecare detinator de modele foundationale, are mai multe versiuni ale aceluiasi model, pentru intrebuintari de diverse complexitati, pentru a ajuta cu costurile dar si cu balansarea resurselor. Spre exemplu Google detine modelele din gama Gemini, cu versiunile "Flash", "Thinking", "Pro".

Avand in vedere problema costului dar si a balansarii traficului, aplicatiile de asistenti de programare de pe piata, au implementat o modalitate de a clasifica prompt-urile utilizatorului in functie de complexitate si rutarea acestui prompt catre un model potrivit sa rezolve aceasta problema si optim din punct de vedere al costului.

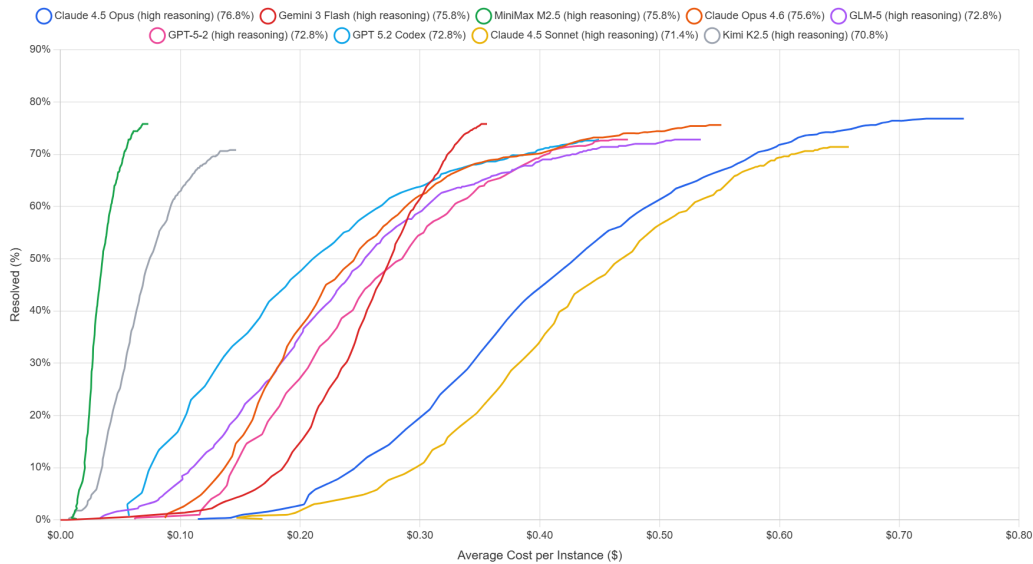


Figura 1.26: Clasamentul modelelor in SWE-Bench Pro [15] la momentul actual in functie de cost

1.4 Cerinte pentru o solutie agentica in dezvoltarea software

Pentru a realiza un asistent inteligent in domeniul dezvoltarii software este nevoie sa atingem cateva obiective principale, printre care: o interfata accesibila din diferite medii de dezvoltare pentru a satisface cat mai multe nevoie, unelte ce pot rula in terminalul oricarui sistem de operare, posibilitatea de a alege dintr-o sumedenie de modele, comenzi pentru a controla parametrii modelelor si un prompt de sistem si skill-uri specializate pentru operatii din domeniu.

Agentii functioneaza, pe langa procedeele expuse anterior, pe una dintre aceste filozofii de programare: Spec-Driven Development (SDD) sau Test-Driven Development (TDD). Primul se refera la programarea pe baza unor documente de specificatie ce expun in detaliu toate necesitatile implementarii, iar cel din urma se refera la programarea dupa cerintele sumare si executarea testelor la final pentru a confirma comportamentul dorit.

Pe langa aceste obiective de baza, trebuie sa ne asiguram ca modelul raspunde doar prompt-urilor cu scop legitim, si sa ne asiguram ca modelul nu poate rula comenzi care pun in pericol integritatea sistemului pe care ruleaza sau compromite date sensibile, lucruri pe care le vom discuta in capitolele ce urmeaza.

1.5 Limitari ale arhitecturilor agentice

Desi entuziasmul pentru bula IA atinge cote maxime, inca este mult loc de imbunatatire, iar aceste platforme de asistenti de programare au inca multe defecte pe care trebuie sa le depaseasca.

In primul rand, **bariera principala in avansarea acestor asistenti este fereastra de context**. Aceasta fereastră reprezinta cata informatie poate sa tina minte agentul. Pentru sisteme dezvoltate cu zeci de mii de linii de cod, sau solutii impartite pe micro servicii in mai multe proiecte, acest context limiteaza masiv capabilitatile agentilor. Exista multe tehnici de optimizare a acestui context de la compresie, realizarea unor fisiere ce rezuma cunostiintele actuale si inceperea unei conversatii noi care pleaca de la acel rezumat pana la rularea de cod pentru actiuni repetitive sau alegerea cu grija a prompt-ului. Dar din pacate toate aceste tehnici nu sunt de ajuns. Multe functionalitati care fac un agent performant precum tool-urile reusesc sa consume rapid aceasta fereastră de context. Ca exemplu, 200.000 de tokeni pot

parea un numar mare, dar chiar si acest numar, similar cu memoria hardware, este defapt, mai mic. Din aceasta fereastră doar 160.000 de tokens sunt disponibili cu adevarat. O parte din ei sunt rezervati pentru promptul de sistem, auto-compactare sau pentru iesire. Depasirea acestei ferestre de context scade astfel performantele agentului si se recomanda inceperea unei conversatii noi.

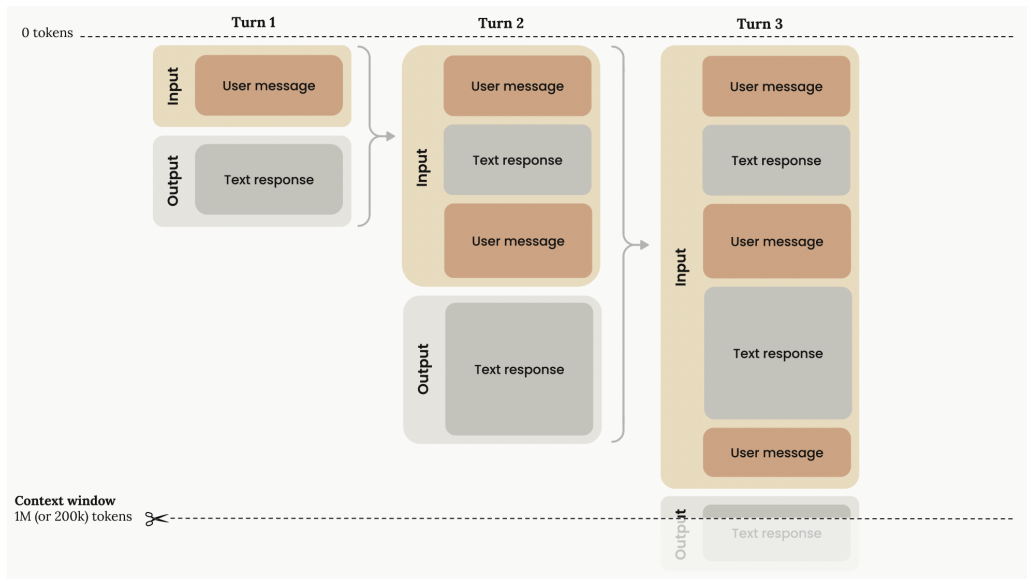


Figura 1.27: Reprezentarea ferestrei de context intr-o conversatie uzuala (sursa)

In al doilea rand, modelele curente par sa genereze date de iesire mai detaliate decat este necesar, preferand cantitatea peste calitate. De multe ori in sarcini de tipul Pull Request (PR) Review, agentii se grabesc sa relateze multe probleme simple sau de formatare si rateaza probleme importante de logica.

In al treilea rand, similar cu punctul anterior, agentii nu au o buna intelegere legata de cand ar trebui sa opreasca investigatiile sau sa incerce alte solutii si raman blocati in bucle infinite de a incerca aceleasi lucruri si a esua.

Pentru toate aceste probleme se lucreaza in mod curent la rezolvare de care vom vorbi ulterior.

1.6 Justificarea alegerilor de implementare

Solutia finala include mai multe categorii care trebuiesc comparate independent.

Pentru inceput, modelele disponibile initial vor fi din gama Gemini, Claude si OpenAI, cu posibilitatea de extindere la modele open-source rulate local sau in cloud.

Pentru partea de framework agentic, ofertele sunt numeroase si variate. Fiecare dintre optiunile de mai jos are avantaje si dezavantaje care trebuiesc cantarite in contextul aplicatiei noastre.

In materie de limbaj de programare, **Python** a fost alegerea evidenta deoarece este aproape folosit in totalitate in astfel de aplicatii, devenind un standard in industrie. Ecosistemul este de asemenea foarte matur si exista o sumedenie de biblioteci pentru a ne ajuta in implementare. De asemenea inca un atu este portabilitatea cu care vine, Python putand fi instalat cu usurinta pe orice sistem de operare cu o simpla comanda.

Am pornit de la ideea ca trebuie sa avem nevoie de un framework gandit pentru utilizat de unelte cu suport pentru mai multe modele, care sa suporte mai multe ramuri de executie, sa incorporeze managementul starii si al memoriei si sa poata construi un workflow robust si

consistent. Pentru asta, am adunat o colectie de framework-uri folosite in industrie pentru a le compara si a decide care este cel potrivit.

Tabela 1.2: Comparație framework-uri pentru workflow-uri agentice

Criteriu	LangChain	LangGraph	Google Gen AI / Vertex	LiteLLM	OpenAI SDK
Rol Principal	Orchestrator general de LLM-uri și integrari.	Framework pentru agenți stateful și fluxuri ciclice.	Platformă enterprise integrată în Google Cloud.	Proxy sau router universal pentru 100+ LLM-uri.	Client oficial pentru modelele OpenAI.
Nivel Abstr.	Înalt (multe componente gata de utilizat).	Mediu/Înalt (control fin asupra grafurilor).	Înalt (Managed Services în Vertex AI).	Jos (strategie de traducere API).	Jos/Mediu (interacțiune directă).
Capabilități Agentice	Limitate (predominant fluxuri liniare).	Excelent (bucle, feedback și multi-agent).	Bun (suport nativ pentru tool-uri).	N/A (doar rutare de mesaje).	Depinde de utilizator (manual sau Assistants API).
Suport Multi-Model	Da (agnostic).	Da (prin ecosistemul LangChain).	Optimizat pentru Gemini.	Absolut (scopul principal).	Nu (doar OpenAI).
Management Stare	Încorporat (Memory buffers).	Superior (persistență și „time travel”).	Gestionat de platformă.	N/A (stateless).	Threads (doar în Assistants API).
Învățare	Medie (ecosistem foarte mare).	Abruptă (necesită logica grafurilor).	Ușoară (Graphical User Interface (GUI) intuitiv în Cloud).	Ușoară (identic cu OpenAI Software Development Kit (SDK)).	Foarte Ușoară, documentat

La nivel conceptual, LangGraph modeleaza un workflow agentic ca un *graf orientat* in care nodurile reprezinta unitati de calcul (apeluri catre LLM, executii de tool-uri sau functii Python arbitrare), iar muchiile descriu tranzitiile dintre acestea. Spre deosebire de abordările liniare de tip “chain”, muchiile pot fi *conditionale*, ceea ce permite ramificari dinamice in functie de starea curenta, precum si *cicluri*, esentiale pentru bucle de tip “reason-act-observe”. Intregul graf opereaza asupra unui obiect de *stare* partajata, tipat explicit (de regula un `TypedDict`), pe care fiecare nod il poate citi si actualiza prin *reducers* declarativi, eliminand astfel ambiguitatile legate de propagarea contextului intre pasi.

Un alt element central este mecanismul de *checkpointing*: starea grafului este serializata dupa fiecare pas si poate fi reluata, inspectata sau derulata inapoi (*time travel*). Acest lucru ofera doua proprietati importante pentru un asistent de dezvoltare software: persistenta conversatiei intre sesiuni si posibilitatea de a introduce puncte de *human-in-the-loop* in care executia este suspendata pana cand utilizatorul confirma o actiune (de exemplu, aplicarea unei modificari de cod sau rularea unei comenzi).

In final am ales sa folosesc **LangGraph** datorita controlului mare pe care il ofera asupra

desfasurarii workflow-ului agentic si a integrarii cu Langfuse ¹⁴ dar si pentru compatibilitatea cu logica desfasurarii procesului de asistenta in dezvoltarea software, respectiv buclele de scriere de cod, feedback, investigare si imbunatatire, in ciuda complexitatii mai mari pe care o constituie acest framework.

Pentru interactiunea cu utilizatorul, ideea de pornire a fost ca trebuie sa avem o interfata care poate fi rulata in orice context. Gandul m-a purtat automat catre un TUI deoarece orice sistem de operare detine un terminal care poate fi pornit in orice mediu de programare, oferind o gama larga de utilizatori potentiali. Mai departe am constituit o lista de abordari si librarii pe care le putem folosi pentru a construi aceasta interfata, fiecare cu avantaje si dezavantaje care trebuiesc comparate.

Tabela 1.3: Comparație biblioteci pentru interfața cu utilizatorul

Bibliotecă	Arhitectură	Avantaje	Dezavantaje
Textual	Motor TUI pe ecran complet folosind buffer-ul alternativ al terminalului. Arbore de widget-uri bazat pe Document Object Model (DOM) (similar cu Hyper-Text Markup Language (HTML))	• Sistem de layout similar cu Cascading Style Sheets (CSS) (grid, dock) • Formatare de text încorporată nativ • Complet asincron și bazat pe evenimente	• Folosește un scrollbar fals (nu funcționează scroll-ul nativ) • Supraîncărcarea DOM-ului cauzează lag dacă mesajele vechi nu sunt șterse • Dificultate la copierea/lipirea nativă a textului
Prompt Toolkit și Rich	Experiență interactivă folosind buffer-ul principal al terminalului. Afișare prin stdout standard.	• Derulare perfectă prin scrollbar-ul nativ al terminalului • Zero lag sau probleme de memorie pe măsură ce istoricul crește • Selectarea și căutarea nativă a textului funcționează impecabil	• Nu este un motor real de layout GUI (doar flux liniar) • Dificil de implementat panouri laterale sau complexe • Limitat strict la input ancorat în partea de jos
Urwid	TUI clasic pe ecran complet folosind buffer-ul alternativ al terminalului. Ierarhie strictă de widget-uri.	• Extrem de matur, stabil și testat în producție • Randare foarte performantă pentru liste mari • Gestionare excelentă a input-ului standard din terminal	• API învechit și curbă de învățare mai abruptă • Lipsește stilizarea modernă încorporată • Sincron implicit (necesită integrare suplimentară)

¹⁴Unealta de observabilitate open-source pentru agenti, disponibila la <https://langfuse.com>

Initial, abordarea cu Textual parea corecta pentru ca ofera un sistem de layout foarte puternic care poate fi folosit pentru a construi o interfata complexa cu panouri laterale si meniuri. Insa, in timpul implementarii a trebuit sa abandonez aceasta idee deoarece am intampinat probleme cu comportamentul interfetei in cadrul unei conversatii mai lungi cu elemente animate, pe care o sa le dezvoltam in capitolul de implementare.

Astfel, am finalizat lista alegerilor arhitecturale principale, vom motiva si restul alegerilor punctual, pe parcursul implementarii.

Capitolul 2

PRACTICI CURENTE IN INDUSTRIE

In acest capitol vom aborda problematica plasarii proiectului in industrie si alegerile pe care le-am facut pentru a propune un produs competitiv.

2.1 Software open-source

Momentan, cea mai mare cota de piata o au furnizorii de modele mari de limbaj precum Anthropic, Google sau OpenAI care lanseaza aplicatii care au ca obiectiv blocarea utilizatorilor in ecosistemul lor personal, prin unelte ce urmaresc domenii diverse.

In timp ce exista si alternative open source, acestea se lovesc de problemele tipice pentru astfel de programe. Pentru inceput, companiile ce furnizeaza accesul la aceste modele au tarife diferentiale pentru utilizarea cheilor API in astfel de programe si orice incercare de a insela sistemul poate rezulta in restrictionarea accesului la platformele furnizorilor.

De asemenea, o alta problema intampinata de platformele open source sunt lipsa fondurilor sau numarul mic de contribuitori, suportul pentru suita intreaga de functionalitati a programelor detinute de marile firme sau publicitate.

In ciuda acestor dificultati, avand in vedere cresterea vertiginioasa a costurilor, faptul ca accesul la program este gratuit este un mare plus pentru utilizatori. Un alt avantaj este suportul pentru modele de limbaj open source. Acestea sunt foarte usor de folosit chiar pe calculatorul utilizatorului daca detine componente hardware suficiente pentru modelul ales, reducand costurile si mai mult. Desi si pentru solutiile proprietare se pot face ajustari care sa permita folosirea modelelor open source, este preferat un program a carui scop este o platforma agnostica de furnizor.

2.2 Flux agentic ce ruleaza local

Un alt plus este **confidentialitatea datelor si controlul asupra aplicatiei**. Toate datele raman pe dispozitivul sau serverul utilizatorului (exceptand cazurile in care acesta alege explicit sa foloseasca modele de la furnizori si nu modele open source). Acesta ofera un mare avantaj pentru domenii in care se lucreaza cu date importante sau care necesita o confidentialitate ridicata.

Toate fisierele de configuratie sunt disponibile local si au o sumedenie de setari pe care utilizatorul le poate modifica dupa bunul plac.

Ruland aplicatia local, agentul are acces direct la mediul de lucru si astfel are context despre sarcinile pe care trebuie sa le execute. Mai mult, este eliminata necesitatea interventiei umane (pe langa actiunile periculoase ce necesita permisiuni speciale). Acesta poate executa actiuni si prelua controlul asupra dispozitivului, permitand astfel automatizari complexe, recurente sau care ruleaza o singura data, usurand sarcini repetitive salvand astfel foarte mult timp utilizatorului.

Actiunile pe care le poate rula un agent local sunt numeroase datorita integrarii cu terminalul sistemului de operare prin care poate controla o multime de aspecte.

2.3 Observabilitate

Majoritatea panourilor cu informatii de utilizare de la furnizorii de modele de limbaj sunt sumare sau insuficiente. Aplicatiile in terminal au uzual mai multe date de utilizare spre deosebire de website-urile in care se desfasoare experienta conversationala, dar nu ofera prea multe date despre ce unelte in mod cert consuma cel mai mult context.

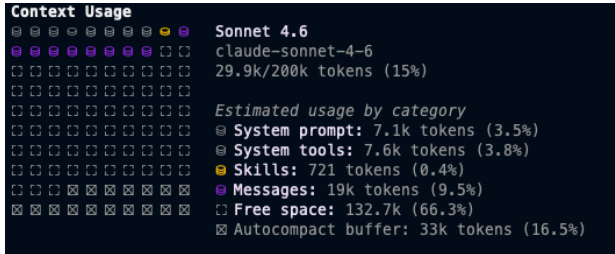


Figura 2.1: Reprezentarea ferestrei de context intr-o conversatie in Claude Code

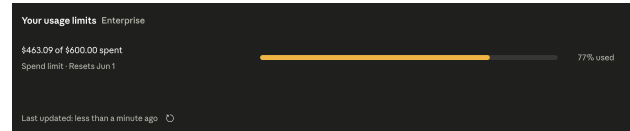


Figura 2.2: Reprezentarea utilizarii in Claude (varianta web) (sursa)

Astfel, in implementarea acestui asistent de programare, observabilitatea va juca un rol important pentru transparenta cu utilizatorul. Acesta poate interoga orice metrica disponibila in legatura cu consumul de tokeni si functionarea sistemului in timp real folosind Langfuse cat si panoul web inclus cu aplicatia.

2.4 Securitate si confidentialitate

Din punctul de vedere al datelor utilizatorului si confidentialitate utilizarea aplicatiei ofera confidentialitate totala. Datele acestuia nu pleaca dispozitivul sau decat daca acesta opteaza pentru modele ale furnizorilor sau daca isi configureaza Langfuse pentru observabilitate.

In ceea ce priveste securitatea si integritatea sistemului, programul va avea mai multe masuri, optionale sau obligatorii care se asigura ca agentul nu poate realiza actiuni cu totul distructive. Cateva din aceste sisteme sunt: rularea comenzilor intr-un container pe principiul de sandboxing, gardarea logica a apelarii uneltelor (protejarea apelarii secventiale a uneltelor, de exemplu trebuie sa citeasca un fisier inainte sa scrie in el), si gardarea intrarilor si iesirilor contra comportamentelor malitioase.

2.5 Cost

Costul este minim pentru asistentul de programare pe care urmeaza sa il prezint in urmatoarele capitole. Costul de utilizare este gratuit. De asemenea, daca utilizatorul alege sa opteze pentru modele open source, singurele costuri pe care va trebui sa le suporte pe langa achizitionarea unui dispozitiv capabil sa ruleze aceste modele sunt curentul electric si internetul.

Mai mult, aplicatia este optimizata pentru a reduce consumul de tokeni, deci si in cazul folosirii furnizorilor de modele, costul va fi unul pe cat se poate de redus, lasand la o parte taxele puse de acestia pe utilizarea modelelor in aplicatii terte.

2.6 Aplicatii OSS concurente

Pe langa aplicatiile furnizorilor exista si cateva aplicatii open source care s-au bucurat de o oarecare adoptie.

Tabela 2.2: Top 6 asistenti de programare open source

Aplicatie	Audienta si scop	Puncte forte	Puncte slabe	Functionalitati cheie
OpenClaw (GitHub) Massachusetts Institute of Technology	Utilizatori care doresc un asistent IA personal, always-on, accesibil pe orice canal de comunicare	• Cel mai popular proiect open source din categorii • Suport multi-canal masiv (WhatsApp, Telegram, Slack, Discord, iMessage, Signal etc.) • Sandboxing cu Docker, Secure Shell si OpenShell • Aplicatii native macOS, iOS, Android • Agnostic fata de furnizor, gateway local-first	• Orientat spre asistent personal, nu exclusiv programare • Configurare complexa pentru multi-canal • Necesita Node.js 22+ si infrastructura gateway • Nu are integrare directa in IDE	Gateway local, multi-canal, sandboxing, voice wake, live canvas, cron, skills, multi-agent, apps native
OpenHands (GitHub) Massachusetts Institute of Technology	Dezvoltatori si echipe enterprise care necesita un agent autonom complet cu izolare securizata	• Sandboxing in containere Docker • GUI web si CLI disponibile • SDK Python pentru constructia de agenti personalizati • Suport Kubernetes si enterprise • Agnostic fata de furnizor (Claude, GPT, Ollama etc.)	• Necesita Docker • Configurare initiala complexa • Nu are integrare directa in IDE • Curba de invatare mai abrupta	Sandboxing, SDK, GUI web, CLI, Kubernetes, integrari Slack/Jira/Linear
Cline (GitHub) Apache 2.0	Dezvoltatori care prefera lucrul direct din IDE cu control granular si automatizari avansate	• Extensie VS Code si plugin JetBrains • SDK Node.js si CLI headless pentru CI/CD • Echipe multi-agent si agenti programati • Suport MCP si plugin-uri extensibile • Agnostic fata de furnizor (200+ modele via OpenRouter)	• Consum ridicat de tokeni datorita contextului mare • Fara sandboxing nativ incorporat • Complexitate ridicata, multe functionalitati • Dependent de ecosistemul VS Code/JetBrains	Extensie IDE, SDK, CLI, multi-agent, MCP, agenti programati, integrari Slack/Discord/Telegram
Goose (GitHub) Apache 2.0	Utilizatori generali si dezvoltatori care doresc un agent IA nativ, performant si extensibil	• Aplicatie nativa desktop (macOS, Linux, Windows) • Construit in Rust deci are o performanta ridicata • Suport MCP (70+ extensii) • Agent general, nu doar pentru cod • Agnostic fata de furnizor (15+ furnizori, suport Agent Communication Protocol)	• Mai putin specializat pe programare • Ecosistem de extensii mai tanar • Nu are integrare directa in IDE • Documentatie in curs de maturizare	Aplicatie nativa, CLI, API, MCP, Agent Communication Protocol, Rust, multi-platform, Linux Foundation
Aider (GitHub) Apache 2.0	Dezvoltatori individuali care prefera un workflow minimalist din terminal cu integrare Git	• Mapare automata a codului sursa (repo map) • Integrare Git nativa cu commit-uri automate • Suport 100+ limbaje de programare • Linting si testare automata • Agnostic fata de furnizor (Claude, GPT, DeepSeek, Ollama)	• Fara sandboxing • Exclusiv terminal, fara GUI • Capabilitati agentice limitate fata de concurenta • Nu are extensie nativa de IDE	Terminal, Git auto-commit, repo map, voice-to-code, linting, modele locale

Tabela 2.1: Produse IA ale principalilor furnizori de modele de limbaj

Companie	Produs	Descriere	Audienta	Pret
Anthropic	Claude (web/app)	Asistent IA conversational pentru analiza, scriere, programare si rationament, cu suport multimodal si artefacte interactive	Utilizatori generali, profesioniști	Freemium (Pro: \$20/luna, Max: \$100–200/luna)
	Claude Code	Agent de programare agentic care ruleaza in terminal, Integrated Development Environment (IDE) sau Slack; poate naviga coduri mari, crea PR-uri si rula teste automat	Dezvoltatori software	Inclus in Pro/Max (API: cost per token)
	Claude Cowork	Agent agentic de productivitate care lucreaza cu fisiere locale si aplicatii cloud (Slack, Google Drive) pentru a automatiza sarcini recurente	Profesionisti, echipe	Inclus in Pro/Max
	Claude Design	Instrument de prototipare vizuala integrat in Claude, care genereaza interfete GUI interactive si mockup-uri pe baza descrierilor in limbaj natural	Designeri, dezvoltatori	Inclus in Pro/Max
Google	Gemini (web/app)	Asistent IA conversational multimodal cu acces la cautare Google, extensii si generare de imagini; disponibil pe web, Android si iOS	Utilizatori generali	Freemium (Advanced: \$19.99/luna in Google One AI Premium)
	Gemini CLI	Agent de programare open-source (Apache 2.0) care ruleaza in terminal, cu suport Model Context Protocol (MCP), cautare Google si context de 1M tokeni	Dezvoltatori software	Gratuit (60 req/min, 1000 req/zi cu cont Google)
	NotebookLM	Asistent de cercetare alimentat de Gemini care analizeaza documente incarcate (Portable Document Format (PDF), site-uri, video-uri) si genereaza rezumate, podcast-uri audio si conexiuni intre surse	Cercetatori, studenti	Gratuit (Plus: \$7.99/luna, Business: contact)
	Vertex AI	Platforma enterprise pentru constructia, antrenarea si implementarea de agenti IA si modele Machine Learning (ML), cu acces la 200+ modele si instrumente Machine Learning Operations (MLOps)	Echipe enterprise, ingineri ML	Platit (pay-as-you-go, de la \$0.0001 per operatie)
	Firebase Studio	Mediu de dezvoltare cloud (IDE) cu agenti IA integrati pentru constructia de aplicatii full-stack (backend, frontend, mobil) direct din browser	Dezvoltatori software	Gratuit in previzualizare (3 workspace-uri; se include in 2027)
OpenAI	ChatGPT (web/app)	Asistent IA conversational cu suport multimodal, cautare web, generare de imagini, analiza de date si plugin-uri; disponibil pe web, iOS, Android si desktop	Utilizatori generali, profesioniști	Freemium (Plus: \$20/luna, Pro: \$200/luna)
	Codex	Agent de inginerie software cloud care executa sarcini de programare in paralel, in sandbox-uri izolate preincarcate cu codul din repository-ul GitHub	Dezvoltatori software, echipe	Inclus in ChatGPT Pro/Business/Enterprise (API: \$1.50/\$6 per MTok)
Microsoft/ GitHub	GitHub Copilot	Agent de programare integrat in GitHub si disponibil ca aplicatie CLI, extensie IDE (VS Code, JetBrains) si in github.com; ofera completare de cod, chat, revizuire PR-uri si rezolvare automata de issue-uri	Dezvoltatori software, echipe	Freemium (gratuit pentru repository publice; Pro: \$10/luna, Business: \$19/luna, Enterprise: \$39/luna)

Capitolul 3

IMPLEMENTARE

3.1 Mediul de dezvoltare

Expunerea elementelor constitutive disponibile care alcatuiesc mediul hardware cat si software este imperativa pentru relatarea procesului de dezvoltare si reproducerea pasilor prin care s-a ajuns la produsul final.

3.1.1 Cerinte hardware

Cerintele hardware pentru dezvoltarea si rularea acestei aplicatii sunt minime. In cazul in care folosim modelele din cloud este necesar doar un laptop sau computer ce poate rula Python 3 cu conexiune activa la internet. Deoarece inferenta nu este realizata local, nu este necesar un GPU performant sau o cantitate mare de Random Access Memory (RAM). Daca se doreste rularea unui model local, cerintele hardware depind de modelul ales, dar cresc exponential necesitand acces la o placa video moderna si o cantitate semnificativa de RAM.

Tabela 3.1: Hardware recomandat pentru modele open-source

Model	Parametri	GPU Recomandat	Video Random Access Memory (VRAM)	RAM
LLaMA 3	70B	2x NVIDIA A100 80GB sau 2x RTX 4090	40–160 GB (cuantizat vs FP16)	64–128 GB
Mistral	7B	NVIDIA RTX 3060 / RTX 4090	4–16 GB	8–32 GB
Qwen 2.5 (14B)	14B	NVIDIA RTX 3090 / RTX 4090	16–24 GB	16–64 GB
Gemma 4 (E4B)	4.5B efectivi (8B total)	NVIDIA RTX 3060 / RTX 4060	4–16 GB (cuantizat vs BF16)	8–32 GB

3.1.2 Cerinte software

Cerintele software pentru rularea acestei aplicatii sunt minime de asemenea. Este necesar doar un sistem de operare cu terminal precum Windows, Linux sau macOS, cu Python 3 instalat si acces la internet.

In ceea ce priveste mediul de dezvoltare orice mediu de dezvoltare cu suport pentru Python este suficient, iar singurul program necesar este `uv`¹ pentru a instala dependintele necesare.

3.2 Structura

Structura aplicatiei reflecta separarea responsabilitatilor in mai multe directoare specializate. Directorul `src` contine logica agentului: graful LangGraph, uneltele, sistemul de permisiuni, clientul MCP, pipeline-ul de compactare, sub-agentii, sandbox-ul Docker si tracing-ul. Directorul `ui` contine interfata cu utilizatorul: bucla principala, comenzile slash, sistemul de skill-uri, gestionarea sesiunilor si preferintelor. Directorul `web` contine un dashboard analitic Flask care vizualizeaza sesiunile, consumul de tokeni si activitatea uneltelor. Directorul `eval` contine un runner de evaluari cu metrice precum Bilingual Evaluation Understudy (BLEU), Recall-Oriented Understudy for Gisting Evaluation (ROUGE) si potrivire semantica. Documentatia detaliata per-subsistem se gaseste in `docs`, iar configurarea implicita in `config/config.yml`.

<code>vibe-cli/</code>	<code>src/</code>	<code>config/</code>
- <code>config/</code>	- <code>main.py</code>	- <code>config.yml</code>
- <code>docs/</code>	- <code>tools.py</code>	\ - <code>eval.yml</code>
- <code>eval/</code>	- <code>tool_guards.py</code>	
- <code>examples/</code>	- <code>providers.py</code>	<code>ui/</code>
- <code>scripts/</code>	- <code>permissions.py</code>	- <code>app.py</code>
- <code>src/</code>	- <code>sandbox.py</code>	- <code>ui.py</code>
- <code>tests/</code>	- <code>mcp_client.py</code>	- <code>commands.py</code>
- <code>ui/</code>	- <code>artifacts.py</code>	- <code>skills.py</code>
- <code>web/</code>	- <code>attachments.py</code>	- <code>sessions.py</code>
- <code>ARCHITECTURE.md</code>	- <code>capabilities.py</code>	- <code>preferences.py</code>
- <code>FIRST_START.md</code>	- <code>clipboard.py</code>	- <code>provider_handler.py</code>
- <code>HEADLESS_USAGE.md</code>	- <code>log_context.py</code>	- <code>pastes.py</code>
- <code>Makefile</code>	- <code>logging_setup.py</code>	- <code>paths.py</code>
- <code>pyproject.toml</code>	- <code>multimodal.py</code>	- <code>profile.py</code>
- <code>README.md</code>	- <code>tracing.py</code>	- <code>config_init.py</code>
- <code>UPDATING.md</code>	- <code>compaction/</code>	- <code>vibe_md.py</code>
\ - <code>uv.lock</code>	\ - <code>subagents/</code>	\ - <code>updater.py</code>

Figura 3.1: Structura directoarelor vitale ale proiectului

3.3 Configurare

Configurarea aplicatiei se face prin intermediul fisierului `config.yml` care contine catalogul de furnizori (Ollama, OpenAI, Claude, Amazon Bedrock, Gemini), modelele disponibile pentru fiecare, pipeline-ul de compactare, setarile de tracing, configuratia sandbox-ului sau a subagentilor si a uneltelor. Cheile API sunt declarate ca variabile de mediu (de exemplu `${OPENAI_API_KEY}`) si sunt rezolvate la pornire. Daca nici o cheie nu este setata si Ollama nu este accesibil, aplicatia solicita interactiv introducerea unei chei la pornire prin comanda `/model` sau `/key`.

De asemenea pentru procesul de evaluare al capabilitatilor folosit pentru teste de sanatate si regresie, se pot modifica setarile din fisierul `/.vibe-cli/eval/eval.yml`

¹`uv` este un manager de pachete pentru Python, similar cu `pip`, dar mult mai eficient

Preferintele utilizatorului (provider-ul implicit, modelul selectat per provider, nivelul de efort, serverele MCP, listele de permisiuni) sunt persistate in `~/.vibe-cli/preferences.json` si supravietuiesc intre sesiuni. Sesiunile de conversatie sunt salvate ca log-uri de evenimente in `~/.vibe-cli/sessions/`, sortate per proiect (directorul de lucru).

3.4 Interfata cu utilizatorul

Interfata serveste ca un mediu de legatura intre utilizator si agent, dar si ca un spatiu de configurare al aplicatiei. Aceasta apeleaza functii ale agentului in acelasi proces Python, comunicarea fiind prin apeluri de functii, nu prin retea.

In dezvoltarea interfetei, am pornit folosind solutia oferita de biblioteca Textual. La finalul primului prototip interfata arata modern si functiona corect. Insa, pe masura ce am populat conversatia cu asistentul, am observat ca functionalitatea de scroll a ferestrei nu functiona corect, sarind catre partea de sus a ecranului. Dupa ce m-am interesat mai atent la solutiile implementate de alte programe asemanatoare precum Claude Code, am observat ca acestea nu folosesc un TUI propriu zis, ci doar scriu in terminal folosind stdout, lasand astfel scroll-ul nativ al terminalului sa se ocupe de istoricul conversatiei. Am decis astfel sa abandonez solutia cu Textual si sa implementez o interfata bazata pe **prompt_toolkit** si **Rich** care sa scrie in terminal, oferind astfel o experienta mult mai fluida si fara probleme de scroll sau lag.

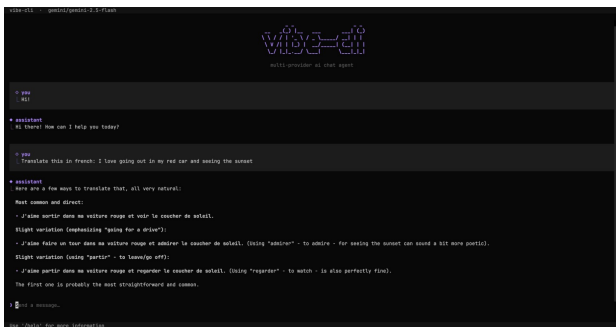


Figura 3.2: Prima versiune a interfetei, realizata cu ajutorul libreriei Textual

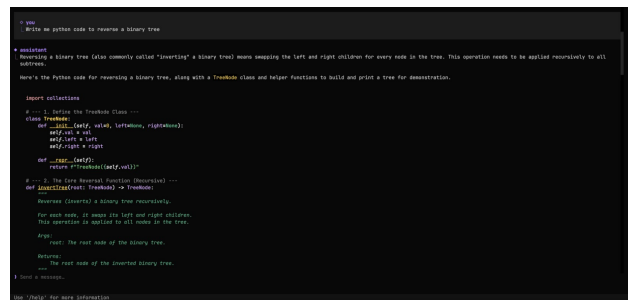


Figura 3.3: Continuarea conversatiei cu prima versiune a interfetei, realizata cu ajutorul libreriei Textual

In final, a fost o decizie buna sa abandonez implementarea cu Textual, deoarece am reusit sa produc rezultate mai bune folosind terminalul clasic. Solutia finala este si mai estetica, cat si simpla, functionala si performanta.

Schimbarea nu s-a produs intr-un stadiu avansat al dezvoltarii, ci chiar de la primele prototipuri de interfata, lucru care nu a ingreunat cu mult procesul de dezvoltare.

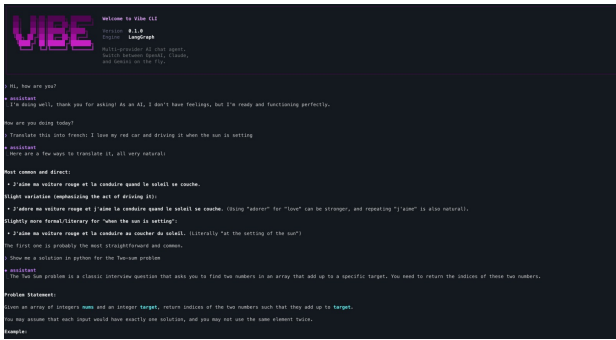


Figura 3.4: A doua versiune a interfetei, realizata fara biblioteca Textual

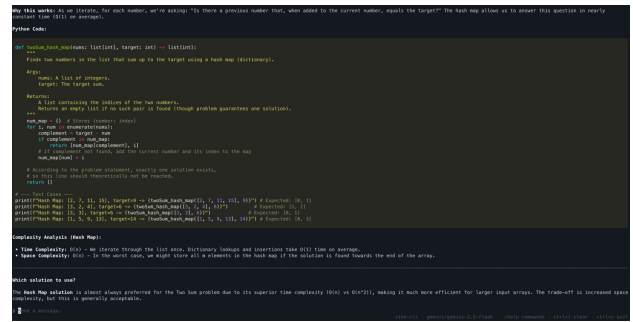


Figura 3.5: Continuarea conversatiei cu a doua versiune a interfetei, realizata fara biblioteca Textual

Dupa inlocuirea implementarii, am continuat sa adaug functionalitati pentru a imbogati experienta utilizatorului. Interfata ofera mai multe elemente vizuale menite sa informeze utilizatorul in timp real despre activitatea agentului:

- **Spinner animat cu eticheta de faza si cronometru:** textul spinner-ului reflecta faza curenta si masoara timpul scurs, de exemplu `thinking...` 1.2s in timpul inferentei LLM, sau `Read(README.md)`... 0.4s in timpul executiei unei unelte
- **Log persistent de unelte:** fiecare apel de unealta lasa o linie $\leftrightarrow$ `Tool(arg)` in istoricul vizibil, astfel incat si uneltele care se executa rapid sunt vizibile
- **Panouri de diff pentru write_file:** apelurile reusite de scriere sunt redactate ca un diff unificat intr-un panou Rich cu numarul de linii adaugate si sterse
- **Paste folding:** textele lipite mai lungi de 200 de caractere sunt inlocuite in prompt cu un placeholder scurt `[Paste #N <len> chars]` pentru lizibilitate, continutul original fiind restaurat inainte de trimitere
- **Footer de timp la sfarsitul fiecărei ture:** o linie care indica durata turei, cu o fraza aleasa aleator

Pe langa elementele vizuale, functionalitatile principale includ:

- Unelte inteligente de interactiune cu mediul virtual (create/editare/citire fisiere, cautare web, executare comenzi in terminal) care instruiesc modelul de limbaj sa le foloseasca intr-un mod inteligent din punct de vedere al consumului ferestrei de context.
- Gardarea apelului logic al uneltelor. Uneltele au anumite restrictii pentru a asigura folosirea corecta. De exemplu, modelul nu poate apela unealta `edit_file` fara ca mai intai sa apeleze unealta `read_file`. De asemenea sunt gardari de citire a fisierelor. Modelul nu poate citi un fisier intreg pentru a garanta ca nu isi va umple toata fereastra de context cu output-ul acelei unelte.
- Alegerea modelului din 5 provideri (Ollama, OpenAI, Claude, Bedrock, Gemini) si a efortului de gandire. Furnizorii de modele de limbaj nu sunt limitati si lista poate creste prin extinderea clasei de baza `BaseProvider`.
- Suport pentru intrari multimodale (imagini, fisiere PDF, fisiere text si documente), atasate in prompt prin cale locala sau Uniform Resource Locator (URL), atunci cand modelul selectat ofera capabilitati multimodale (vezi tabelul 4.5); pentru modelele exclusiv textuale, atasamentele sunt respinse cu un mesaj explicativ.

- Gestionarea sesiunilor cu posibilitatea de a salva, sterge, lista sau continua conversatii anterioare. Fiecare proiect include istoricul propriu al conversatiilor.
- Sistemul de permisiuni per-unealta cu liste de always-allow si always-deny persistente
- Conectarea cu servere MCP prin transport Standard Input-Output (STDIO), Hyper-Text Transfer Protocol (HTTP) sau Server-Side Events (SSE)
- Skill-uri bazate pe fisiere Markdown cu incarcare progresiva (doar numele si descrierea in prompt-ul de sistem, corpul incarcat la cerere)
- Sub-agenti specializati extensibili (cativa agenti de baza precum: general, code search, web research, dar pot fi adaugati mai multi sub-agenti prin extinderea clasei `Subagent`) care ruleaza in paralel cu modele configurabile per-agent.
- Pipeline de compactare a contextului cu mai multe stadii (trunchiere, sumarizare, deduplicare) configurabil.
- Sandbox Docker optional pentru executia izolata a comenzilor
- Tracing prin Langfuse Cloud pentru observabilitate
- Dashboard web analitic (Flask) cu vizualizare de metrice, cost si activitate
- Auto-completare pentru comenzile slash si actualizare automata a versiunii

3.5 Instalare si utilizare

3.5.1 Cerinte preliminare

Aplicatia necesita Python 3.10 sau o versiune ulterioara si este disponibila pe sistemele de operare Linux si macOS; instalarea pe Windows nu este suportata. Pentru accesul la modelele de limbaj este necesar fie un server Ollama local pornit prin `ollama serve`, fie cel putin o cheie API valida pentru unul dintre furnizorii cloud (`OPENAI_API_KEY`, `ANTHROPIC_API_KEY` sau `GOOGLE_API_KEY`). In lipsa oricarei configuratii prealabile, aplicatia ofera la pornire un flux interactiv pentru selectia furnizorului si introducerea cheii.

3.5.2 Instalare

Metoda recomandata este executarea script-ului oficial de instalare, care detecteaza si instaleaza la nevoie gestionarul de pachete `uv`, dupa care obtine aplicatia direct din depozitul Git:

```
curl -fsSL https://raw.githubusercontent.com/Moshulika/vibe-cli/main/scripts/install.sh
| bash
```

Alternativ, instalarea se poate efectua manual prin `uv tool install` sau `pipx install`, indicand depozitul Git ca sursa. Dezinstalarea se face prin `uv tool uninstall vibe-cli`. Pentru dezvoltare, dupa clonarea depozitului, comanda `make dev-setup` instaleaza dependentele si hook-urile `pre-commit`, iar `make ui` lanseaza interfata de chat. Sunt expuse de asemenea tinte `make` pentru rularea suitei de teste (`make test`), formatarea codului (`make format`) si pornirea dashboard-ului analitic web (`make web`).

3.5.3 Configurare la prima rulare

Dupa instalare, comanda **vibe** este disponibila global din orice director. La prima pornire, aplicatia creeaza arborele de configuratie sub `~/.vibe-cli/`, ce contine fisierul de preferinte `preferences.json` (furnizorul implicit, modelul selectat per furnizor, nivelul de efort, cheile API si permisiunile per unealta), un identificator anonim `profile.json`, directorul cu skill-uri exemplificative si log-urile structurate. Furnizorii configurati anterior sunt validati automat printr-un apel usor catre endpoint-ul fiecaruia, fara a consuma tokeni; cheile invalide sunt purificate si semnalate utilizatorului. In absenta unei configuratii valide, un flux interactiv solicita selectia furnizorului si introducerea cheii API (cu mascarea intrarii), validand-o inainte de salvare.

3.5.4 Utilizare

Dupa configurare, utilizatorul interactioneaza cu modelul printr-o sesiune persistenta de chat, salvata automat in evenimente granulare sub directorul corespunzator proiectului curent. Comenzile pot fi rulate in orice moment al conversatiei si au formatul `/comanda`, cu auto-completare la apasarea tastei Tab. Sunt disponibile si scurtaturi de tastatura precum **CTRL+L** pentru a goli conversatia si **CTRL+C** pentru inchiderea aplicatiei cu salvarea automata a sesiunii. Pe langa intrarea textuala, utilizatorul poate atasa imagini, fisiere PDF, fisiere text sau documente, transmise nativ modelului atunci cand acesta dispune de capabilitati multimodale. Dashboard-ul web analitic poate fi pornit separat prin **vibe web**, expus implicit pe portul 5050, si ofera vizualizari pentru consumul de tokeni, costuri, latente, evenimente de compactare si activitatea sub-agentilor.

Comanda	Descriere
<code>/help</code>	Listarea tuturor comenzilor si a scurtaturilor de tastatura
<code>/model</code>	Selectarea provider-ului si modelului, sau setarea cheii API
<code>/key</code>	Setarea sau actualizarea cheii API per provider
<code>/effort</code>	Selectarea nivelului de efort al modelului (low/medium/high)
<code>/mcp</code>	Configurarea serverelor MCP (adaugare/stergere/activare)
<code>/skills</code>	Listarea, activarea sau reincarcarea skill-urilor
<code>/subagents</code>	Listarea tipurilor de sub-agenti si configurarea modelelor per tip
<code>/plan</code>	Activarea/dezactivarea modului de planificare
<code>/agent</code>	Activarea/dezactivarea modului agentic
<code>/permissions</code>	Vizualizarea si editarea listelor de permisiuni per unealta
<code>/clear</code>	Stergerea conversatiei curente si rotirea la o sesiune noua
<code>/sessions</code>	Listarea sesiunilor salvate pentru proiectul curent
<code>/resume</code>	Continuarea ultimei sesiuni sau a uneia specifice prin ID
<code>/compaction</code>	Inspectarea pipeline-ului de compactare si selectarea modelului
<code>/compact</code>	Fortarea unei compactari imediate a contextului
<code>/tracing</code>	Configurarea si activarea tracing-ului Langfuse
<code>/score</code>	Evaluarea ultimului tur in Langfuse (good/bad)
<code>/attach</code>	Atasarea datelor multimodale de tip imagini, PDF sau fisiere text ca si cale de sistem
<code>/paste</code>	Atasarea datelor multimodale de tip imagini, PDF sau fisiere text din clipboard
<code>/dataset</code>	Adaugarea ultimului tur la un dataset Langfuse
<code>/update</code>	Actualizarea la ultima versiune din git
<code>/exit</code>	Salvarea conversatiei si inchiderea aplicatiei

Tabela 3.2: Comenzi disponibile in interfata

3.5.4.1 Modul *headless*

Pe langa sesiunea interactiva, aplicatia expune un mod *headless*, neinteractiv, activat prin parametrul `-p/-prompt` (cu suport pentru citirea promptului din `stdin` prin `vibe -p -`). Acesta executa o singura tura (un apel catre model urmat de bucla de procesare a uneltelor) si tipareste rezultatul final, in format text sau `json` (`-o text|json`), dupa care iese cu un cod conform conventiilor Portable Operating System Interface (POSIX): 0 la succes, 1 la eroare de executie, 2 la argumente invalide si 130 la intreruperea prin `SIGINT`. Modul este *hermetic* in mod implicit: dezactiveaza MCP, salvarea sesiunii, tracing-ul si interactiunile cu utilizatorul, comportament reactivabil selectiv prin optiuni precum `-save-session`. Autentificarea se realizeaza automat prin detectarea variabilelor de mediu corespunzatoare furnizorilor suportati (`OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, `GOOGLE_API_KEY`, `BEDROCK_API_KEY`, `OLLAMA_URL`), scurtcircuitata de flag-ul `-provider`.

Sunt expuse de asemenea optiuni pentru controlul fin al executiei: selectia modelului (`-model`), reglarea esantionarii prin `-effort` (low/medium/high, mapate la temperaturile 0.3/0.5/0.8) sau direct prin `-temperature`, modificarea promptului de sistem (`-system` pentru inlocuire, exclusiv cu `-append-system-prompt` pentru extindere), restrictionarea uneltelor disponibile (`-allowedTools`) si limitarea bugetului de iteratii (`-max-turns`). Iesirea `json` include furnizorul, modelul, raspunsul final, lista apelurilor de unelte, skill-urile utilizate, statistici de consum, numarul de iteratii si durata executiei, facilitand integrarea in pipeline-uri automatizate si in scripturi de evaluare.

Acest mod este exceptional pentru integrari cu alte programe (wrapper²) sau folosirea in medii de tip CI/CD.

3.6 Gestionarea contextului si a memoriei

Aplicatiile ce utilizeaza principii agentice au nevoie de o logica complexa pentru management-ul contextului si al memoriei, din doua motive diferite: personalizare si consistenta conversatiei.

3.6.1 Memoria si istoricul conversatiei

In cazul memoriei, aceasta este folosita pentru a extrage informatii si preferinte pe termen lung. Cateva intrebuintari sunt extragerea de informatii cu legatura la mediul de lucru si preferinte specifice acestuia. Majoritatea aplicatiilor existente pe piata folosesc o tactica asemanatoare pentru stocarea acestor metode:

1. Un folder ascuns (`.claude` in cazul Claude Code, `.copilot` in cazul GitHub Copilot, `.codex` in cazul Codex) care contine preferinte in legatura cu modul de lucru in cadrul proiectului, precum si rutine sau capabilitati specifice lui in format text, pe care agentul le citeste si le utilizeaza dupa bunul plac.
2. Pentru preferintele globale ce au legatura cu personalitatea utilizatorului si modul de lucru preferat de acesta, sau indicatii si decizii pe care ar trebui sa le ia in anumite situatii, exista uzual un folder similar cu cel local pentru proiecte, doar ca intr-o locatie globala, ce contine de asemenea un mic rezumat text pe care agentul il extrage automat in timpul utilizarii si il foloseste in viitoarele actiuni.

²Un wrapper este o aplicatie care inglobeaza o alta aplicatie si foloseste output-ul acesteia pentru a-l prelucra in continuare

3. Istoricul conversatiilor este stocat de obicei local, pe dispozitivul utilizatorului. Fiecare conversatie este atribuita unui proiect (unic in functie de calea pe care se afla acesta in sistemul utilizatorului), careia ii este atribuit suplimentar un identificator unic. Acest istoric contine fiecare prompt pe care utilizatorul il trimite catre modelul de inteligenta artificiala, raspunsul pe care agentul il da si toate apelurile catre unelte si alte operatiuni de sistem. Acest istoric este reintrodus integral in contextul agentului daca se reia conversatia. Un dezavantaj in acest caz, este ca uzual memoria este salvata in doua locuri diferite:
 - Memoria integrala este stocata uzual intr-o baza de date pe serverele furnizorului de servicii si contine tot istoricul conversatiei pe care utilizatorul il poate consulta (contine text, raspunsuri si artefacte incarcate).
 - Memoria contextuala reprezinta o limitare a modelelor actuale. Aceasta este memoria care este inserata in model cu fiecare prompt. Modelele functioneaza cu o fereasta maxima de context. In cazul conversatiilor lungi cu multe unelte utilizate de catre agent sau cu multiple fisiere (sau doar de o marime foarte mare) video/audio sau alte tipuri de documente, aceasta fereasta poate fi epuizata rapid, necesitand un procedeu numit compactarea contextului. Prin aceasta compactare, se sumarieaza sau se sterg efectiv parti neimportante din conversatie pentru a pastra aceasta fereasta de context in limite normale. Depasirea ferestrei rezulta in nefunctionarea modelului deoarece acesta nu va accepta cererea noastra.

In implementare, s-au folosit multe din standardele industriei:

- Utilizeaza doua locuri de stocare a memoriei: stocare intr-un folder ascuns a conversatiilor integrale si in memorie pentru istoricul conversatiei compactate (daca este cazul)
- Preferinte locale si globale. Pe masura ce utilizatorul foloseste agentul, acesta isi noteaza preferintele acestuia si le ia in considerare in cererile viitoare.
- Toate acestea sunt stocate local pentru pastrarea intimitatii utilizatorului

3.6.2 Contextul

Contextul este unul dintre punctele de interes pentru dezvoltarea sistemelor agentice datorita implicatiilor pe care le are in executarea sarcinilor complexe si de lunga durata sau cu cantitati mari de date.

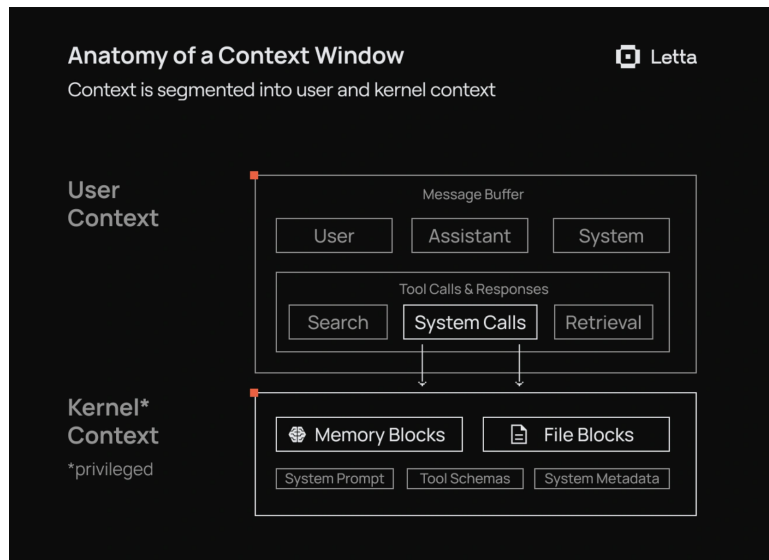


Figura 3.6: Impartirea ferestrei de context (sursa)

Mai multe structuri se lupta pentru o parte din fereastra de context:

- Promptul de sistem - instructiuni pentru comportamentul agentului. Contine printre altele rolul pe care acesta il joaca, restrictii comportamentale, exemple de raspunsuri si capabilitati accesibile.
- Mesajele utilizatorului - prompt-ul pe care utilizatorul il trimite catre agent.
- Raspunsurile asistentului - raspunsul oferit catre utilizator la cererea acestuia. Intotdeauna contextul declarat (spre ex. 200k tokeni) va avea marimea adevarata $C = C_{\max} - O_{\max}$, unde C este contextul adevarat, $C_{\max}$ contextul declarat, si $O_{\max}$ numarul maxim de tokeni de iesire suportati de model. Din aceasta valoare se scad si alte valori rezervate de sistem, precum spatiul tampon pentru compactare.
- Atasamente - documente, imagini, audio si videoclipuri
- Skill-uri, sau alte instructiuni - descrierea si titlul tuturor skill-urilor, sau a rutinelor salvate care pot fi folosite de catre agent sunt injectate in context pentru ca agentul sa poata alege in cunostiinta de cauza un skill sau rutina din cele disponibile.
- Unelte - similar skill-urilor, semnatura uneltelor, descrierea acestora, datele de iesire si instructiunile lor sunt disponibile modelului prin injectarea in context pentru a putea fi alese in cunostiinta de cauza. Se aplica atat uneltelor locale sau celor expuse prin MCP.
- Output-ul uneltelor - datele de iesire ale uneltelor, spre exemplu continutul unui fisier rezultat din utilizarea unei unelte pentru citirea fisierelor poate fi injectat direct in context. In cazul fisierelor mari, acest lucru poate insemna o problema pentru fereastra de context.
- Cautari pe internet - cautarile pe internet returneaza raspunsuri Java Script Object Notation (JSON) sau HTML care uzual nu contin strict date punctuale, ci si intregul continut al paginii. Acest lucru ajuta pentru positionarea elementelor in pagina dar adauga multe caractere ce umplu fereastra de context.

3.6.2.1 Retrieval Augmented Generation (RAG)

Pentru conservarea contextului in lucrul cu documente de marime mare, sau cu colectii de documente, a fost adoptata tehnica RAG [29]. Acesta se bazeaza pe un proces independent care ingesteaza documentele de interes prin partitionare semantica prin care agentul poate cauta cu usurinta, ajungand astfel sa extraga dintr-un document de dimensiuni mari doar bucati de interes.

Dezavantajul acestei tehnici este complexitatea infrastructurii necesare. In primul rand este nevoie de un serviciu separat care sa proceseze si sa stocheze aceste date, iar in final modelul trebuie sa poata accesa aceste date intr-un mod eficient.

Odata cu progresele aduse marimii acestei ferestre de context, modelele de varf ajung sa aibe pana la unul sau doua milioane de tokens ca fereastră de context. Astfel, nevoia pentru RAG scade. Pentru majoritatea intrebuintarilor, aceasta marime a ferestrei este mai mult decat suficienta, chiar si pentru conversatii mai complexe, in ciuda limitarilor cunoscute precum scaderea performantelor pe date ce se afla in mijlocul ferestrei de context [13].

In implementarea actuala *nu au fost folosite tehnici RAG* din cauza complexitatii de implementare, a infrastructurii necesare si a beneficiilor estompate de performantele curente ale modelelor de limbaj.

3.6.2.2 Gestionarea datelor multimodale

Data fiind alegerea de abandonare a tehnicii RAG, in implementare avem nevoie de o modalitate inteligenta de a prelucra datele multimodale pentru prezevarea contextului.

Prima problema care apare in lucrul cu date multimodale apare din cauza necesitatii pentru suportul unei plaje foarte largi de modele de limbaj. Datorita arhitecturii agnostice de furnizor care permite folosirea oricarui model, avem nevoie sa detectam capabilitatile de prelucrarea a tipurilor de fisiere pentru fiecare model in parte. Din pacate, furnizorii de modele sau creatorii care le includ in Ollama, nu specifica nicaieri consistent in metadatele modelului daca acesta capabilitati pentru lucrul cu anumite tipuri de fisiere.

Din acest motiv, am avut de ales intre doua solutii de implementare:

- Punerea utilizatorului in situatia in care trebuie sa se documenteze si sa declare in configuratie capabilitatile modelului pe care doreste sa il foloseasca
- Incercarea automata de trimitere de date multimodale catre model, iar in cazul in care acesta refuza datele, sa bifam in fisierul de preferinte faptul ca acest model nu suporta respectivul tip de date.

Din motive legate de experienta utilizatorului am ales sa continuam cu a doua solutie.

La finalul deciziei legate de detectarea capabilitatilor, a trebuit sa lucram la gestionarea contextului pentru aceste date. Am ales astfel o abordare hibrida, compusa din doua ramuri pe care agentul poate alege sa le apeleze. In prima faza pentru fisierele de tip text sau PDF, agentul poate citi fisierele in format pur textual folosindu-se de uneltele uzuale de citit fisiere, beneficiind de optimizarile acestora, precum folosirea punctuala in zone de interes sau truncarea datelor de iesire. Daca modelul considera ca are nevoie de o vedere completa, pentru fisiere PDF, daca acesta suporta formatul ca tip de date pentru intrare, se poate uita direct pe intregul fisier prin mai multe citiri succesive de maxim 20 de pagini (configurabil). Pozele sunt introduse direct in context daca modelul suporta acest tip de date si sunt limitate la o marime de 10MB.

Compactarea este realizata prin metode ce ruleaza imediat dupa folosirea uneltelor, cum ar fi truncare, referentiere sau sumarizare, fisierele fiind stocate automat ca artefacte.

3.6.2.3 Compactarea

Compactarea conversatiei este un cumul de tehnici ce asigura compatibilitatea conversatiei curente cu restrictiile de intrare ale modelului. Aceasta este folosita predominant in sistemele agentice, deoarece aici se pot genera cantitati mari de date ce epuizeaza fereastra de context rapid.

Compactarea are loc uzual la produsele existente in piata in mod automat cand conversatia atinge in jur de 80% din marimea maxima a ferestrei de context pentru modelul respectiv. Acest lucru ofera o margine de eroare pentru a incerca sa nu porneasca procesul de compactare pana nu se termina tura curenta ³

Aceasta este realizata prin mai multe tehnici printre care cele mai utilizate sunt:

- Formatarea rezultatului returnat de unelte - reprezinta actiuni de formatare deterministice. Un exemplu bun ar fi formatarea output-ului JSON pentru a elimina caracterele nefolositoare precum spatiile.
- Truncarea rezultatului returnat de unelte - reprezinta truncarea efectiva a datelor de iesire ale uneltelor. Mai exact se pastreaza un numar de caractere de la inceputul datelor si un numar de caractere de la sfarsit. Restul este sters.
- Sumarizarea rezultatului returnat de unelte - aceasta metoda implica apelul catre un model de limbaj care sumarizeaza datele de iesire si inlocuieste datele reale cu acest rezumat.
- Substitutia rezultatului returnat de unelte cu referinte - tehnica care salveaza rezultatul returnat de unelte in fisiere externe si il inlocuieste cu un tag pe care modelul il foloseste pentru a accesa fisierul si datele.
- Deduplicarea mesajelor - obiectele care au un scor de similaritate prea ridicat sunt sterse
- Stergerea mesajelor de importanta scazuta - reprezinta stergerea mesajelor de un anumit tip pre-determinat, de exemplu `tool_call` sau `assistant_thinking`
- Sumarizarea intregii conversatii - fiecare mesaj este sumarizat de catre un model de limbaj
- Truncarea intregii conversatii - stergerea celor mai vechi mesaje din conversatie

Consultand metodele enumerate mai sus, putem observa ca exista o sumedenie de metode de optimizare a istoricului conversatiei, dar in principal acestea abordeaza problema datelor de iesire ale uneltelor. Uzual, acestea se executa imediat dupa terminarea turei curente. Doar metodele de compactare pentru intreaga conversatie se realizeaza la atingerea pragului de compactare, si se executa pana cand marimea contextului este sub un anumit prag.

In implementarea curenta, adoptam toate metodele de compactare enumerate mai sus. In produsele existente aceste setari nu pot fi modificate, dar in implementarea curenta am ales sa facem **aceste setari configurabile** pentru a studia comportamentul agentului si al istoricului cu diferite strategii de compactare si efectul pe care acesta il are asupra coerentei acestuia. Configurarea mecanismului de compactare poate fi consultata in figura 3.7.

³O tura reprezinta intrebarea utilizatorului si toate procesele pe care le executa agentul pana la producerea raspunsului final

```

1 compaction:
2   tool_persistence: false
3   enforce_session_model: true
4   trigger:
5     threshold: 0.80
6     min_turns_kept: 4
7   pipeline:
8     - tool_truncate:
9       enabled: false
10      max_tool_tokens: 2000
11      head_tail: [800, 400]
12     - tool_summarize:
13       enabled: false
14       max_tool_tokens: 2000
15       head_tail: [800, 400]
16       timeout_s: 30
17       max_summary_tokens: 500
18     model:
19   - format_compress:
20     enabled: false
21     per_tool: {}
22   - ref_substitute:
23     enabled: false
24     min_ref_tokens: 1500
25     excerpt_tokens: 200
26   - dedupe:
27     enabled: false
28     similarity: 0.92
29     backend: string
30   - importance_prune:
31     enabled: false
32     keep_kinds: [human,
33      system_prompt, assistant_text,
34      tool_result_last]
35     drop_kinds: [
36      assistant_thinking]
37   # Strategies:
38   # - trim - Drop the oldest whole
39   #           turns until size <= target.
40   # Cheapest, no LLM,
41   # deterministic. Loses information
42   # - summarize - Drop the oldest
43   #               whole turns AND LLM-summarize
44   #               their
45   # content into one SystemMessage
46   # so the model retains
47   # high-level continuity.
48   # - hybrid - Keep the most
49   #            recent 'compaction.trigger.
50   #            min_turns_kept'
51   # turns verbatim; summarize
52   # everything older into one
53   # SystemMessage. Recommended
54   # default.
55   - bulk:
56     enabled: false
57     strategy: hybrid
58     summarize:
59       model: ${COMPACTION_MODEL}
60       max_summary_tokens: 1500
61 storage:
62   backup: true
63   keep_backups: 3
64   artifacts: true

```

Figura 3.7: Configurarea pipeline-ului de compactare a contextului

Nota: Unele strategii de compactare sunt de tipul *eager*, adica ruleaza imediat cum tura se termina, iar altele ruleaza doar la atingerea limitei de context. De asemenea unele strategii nu sunt compatibile una cu cealalta.

Compactarea conversatiei functioneaza pe principiul unui sir de executie configurabil. Acest sir de executie este pornit cand nivelul de ocupare al contextului atinge valoarea `threshold`. Apoi, in functie de ce pasi sunt setati ca fiind porniti, fiecare executa un numar de transformari localizate sau globale care pot fi configurate in sectiunile corespunzatoare. Pentru pasii care necesita conexiunea cu un model de limbaj, acesta poate fi cel principal, sau poate fi ales un model mai mic disponibil. Un mic indrumar al setarilor poate fi consultat aici:

- `tool_truncate` poate fi configurat pentru a porni la un rezultat al uneltelor mai mare de `max_tool_tokens` si se poate alege cate token-uri de la inceputul si finalul raspunsului sa fie pastrate
- `tool_summarize` similar cu unaalta anterioara, doar ca in loc sa taie cu totul mijlocul raspunsului de la unelte, acesta il sumarizeaza cu ajutorul unui model de limbaj configurabil.
- `format_compress` poate specifica caror unelte le este aplicata aceasta transformare si care este pragul.
- `ref_substitute` specifica care este numarul minim de token-uri pentru a fi extrage raspunsul uneltei intr-un artefact extern si a injecta in locul sau un tag care artefactul de locul in conversatie.
- `dedupe` contine pragul de similaritate de la care un mesaj este considerat duplicat si metrica dupa care se calculeaza similaritatea
- `importance_prune` contine listele cu tipurile de mesaje care trebuiesc pastrate si cele care trebuiesc sterse in cazul compactarii
- `bulk` contine rutina de sumarizare a intregii conversatii cu mai multe strategii

3.6.2.4 Tehnici pentru eficientizarea ferestrei de context

Exista de asemenea numeroase tehnici care ajuta semnificativ la pastrarea in limite normale a ferestrei de context. Printre acestea se numara utilizarea eficienta a uneltelor, realizata prin utilizarea inteligenta a comenzilor. Spre exemplu, rularea comenzilor in terminal cu argumente ce limiteaza marimea output-ului, precum `tail` sau `head`, sau prin comenzi ce gasesc anumite fisiere si fraze punctual precum `grep` sau `find`.

O alta practica pentru conservarea contextului este pornirea de subagenti care realizeaza cercetarea sau executa diverse procedee si intorc doar un rezumat al actiunilor sau informatiilor pe care le-au aflat catre agentul principal. Astfel, contextul agentului orchestrator ramane curat in timp ce agentii muncitori realizeaza actiunile necesare completarii sarcinii. Aceasta arhitectura multi-agent permite agentului orchestrator sa se concentreze pe logica problemei complexe si sa delege sub-sarcini altor agenti, acesta doar interpretand rezultatele acestora.

Ambele tehnici sunt utilizate in implementarea actuala si sunt detaliate in urmatoarea sectiune.

3.7 Componente ale arhitecturilor agentice moderne

Pentru a avea un flux de lucru agentic, modelul de limbaj trebuie sa fie adaptat pentru a urma instructiuni ce il ghideaza in a folosi diverse unelte, capabilitati predefinite sau pentru interactiunea cu o echipa de agenti.

3.7.1 Unelte

Inima arhitecturii agentice este reprezentata de uneltele pe care modelele de limbaj invata sa le utilizeze. Acestea nu reprezinta nimic mai mult decat metode uzuale ale limbajului de programare cu precizarea ca modelul de limbaj are acces la semnatura acestora (gruparea de nume al functiei, parametrii de intrare, date returnate) impreuna cu o descriere detaliata de utilizare.

```

1 @tool
2 def list_directory(path: str = ".", show_hidden: bool = False) -> str:
3     """List files and directories at the given path.
4     Args:
5         path: Directory path to list (default: current directory).
6         show_hidden: Whether to include hidden files (default: false).
7     """
8     try:
9         p = Path(path).expanduser().resolve()
10        if not p.exists():
11            return f"Directory not found: {path}"
12        if not p.is_dir():
13            return f"Not a directory: {path}"
14        entries = sorted(p.iterdir(), key=lambda e: (not e.is_dir(), e.name.lower()))
15        lines = [f"Contents of {p}:", ""]
16        for entry in entries:
17            if not show_hidden and entry.name.startswith("."):
18                continue
19            prefix = "[Folder] " if entry.is_dir() else "[File] "
20            size = ""
21            if entry.is_file():
22                s = entry.stat().st_size
23                if s < 1024:
24                    size = f" ({s}B)"
25                elif s < 1_048_576:
26                    size = f" ({s / 1024:.1f}KB)"
27                else:
28                    size = f" ({s / 1_048_576:.1f}MB)"
29            lines.append(f" {prefix}{entry.name}{size}")
30        if len(lines) == 2:
31            lines.append(" (empty)")
32        return "\n".join(lines)
33    except PermissionError:
34        return f"Permission denied: {path}"
35    except Exception as e:
36        return f"Error listing directory: {e}"

```

Figura 3.8: Implementarea unei functii pe care agentul o foloseste pentru a lista directoriile si fisierele la o anumita cale pe sistem

Aceste unelte sunt alese cu grija pentru a mentine un echilibru intre numarul total de unelte si suita de capabilitati necesare pe care agentul le are in dotare.

Numarul de unelte afecteaza direct modul de gandire al modelului si performanta in a executa sarcini complexe. Cu prea putine, acesta poate intampina probleme in executia problemelor cu adevarat sofisticate deoarece ii poate lipsi o capabilitate necesara. Pe de alta parte, daca are prea multe acesta nu se va putea decide asupra unei singure unelte, iar in lipsa unei descrieri detaliate creste riscul de folosire gresita sau ineficienta a uneltelor.

De asemenea, pentru o eficienta maxima, scopul acestora nu trebuie sa se suprapuna. Uneltele trebuie sa fie unice, sa aiba un scop clar si sa poti da un exemplu clar pentru momente in care agentul sa utilizeze respectiva unealta, dar si pentru cand nu ar trebui sa o foloseasca, sau sa prefere o alta unealta in detrimentul acesteia. De exemplu, un set de unelte folosit universal in mai multe solutii agentice sunt:

- Listarea directoarelor si fisierele
- Citirea fisierele
- Crearea fisierele cu sau fara continut
- Editarea fisierele
- Cautari pe internet
- Executia comenzilor in terminal
- Utilizare skill
- Pornire subagent

Utilizarea acestor unelte trebuie ingradita totusi de notiuni logice. Executia acestora este determinata de siruri logice de evenimente care trebuie indeplinite anterior executiei. Spre

exemplu, un agent nu ar trebui sa scrie intr-un fisier daca nu l-a citit inainte, deoarece aceste nu are contextul aceluui fisier si poate scrie lucruri irelevante sau halucinate. Acest lucru este implementat cu ajutorul framework-ului LangGraph printr-un hook ⁴ ce este apelat inainte sau la finalul executiei unei unelte. Rezumandu-ne la exemplul cu uneltele de citire scriere din fisiere ilustrat si in fisierul de configurare din care am extras figura 3.9, aceste hook-uri salveaza un Secure Hash Algorithms (SHA) ⁵ al fisierului citit in memorie, pe care mai apoi unealta de scriere a fisierului il citeste si il compara cu SHA-ul calculat individual. Daca aceste doua coduri coincid, inseamna ca fisierul citit este exact cel care urmeaza sa fie scris, astfel este garantata integritatea contextului inainte scrierii. Deoarece uneltele sunt un bloc logic deterministic, acest lucru garanteaza verificarea integritatii contextului inainte de fiecare apel potential distructiv. Agentul are acces doar la umplerea parametrilor functiei cu valori arbitrare, dar nu are control asupra logicii efective a uneltei.

```

1 tool_guards:
2   enabled: true # master switch - false
      disables ALL guards
3   read_before_write:
4     enabled: true
5     # When true (default), a freshness check
      requires the file's current SHA
6     # to match what was read. When false,
      only "ever read in this session"
7     # is required - looser, but skips the
      disk read on every write/edit.
8     check_freshness: true

```

Optiunea de a configura aceste gardari revine utilizatorului ce poate porni sau opri aceasta functionalitate. Fiind o aplicatie OSS, daca utilizatorul este si dezvoltator software, acesta poate modifica codul sursa si extinde logica de gardare cu mai multe reguli ce pot fi oprite sau pornite din aceasta sectiune.

Figura 3.9: Configurarea gardarii executiei uneltelor

Agentul trebuie sa utilizeze in mod inteligent aceste unelte. Pe langa gardarea logica a secventei de unelte, acesta trebuie sa stie cum poate folosi uneltele la potentialul maxim. Un bun exemplu pentru acest argument este executia de cod scris de agenti in terminal. Pentru optiuni repetitive care ar consuma o buna parte din context, modelele sunt antrenate sa scrie portiuni mici de cod, uzual in limbajul Python datorita simplitatii, si a disponibilitatii crescute. Astfel, agentul va scrie aceste bucati de cod si le va executa in terminal pentru a realiza actiuni complexe in doar cateva linii de cod, sporind eficienta cercetarii si a testarii. Aceasta utilizare este fructificata de promptul de sistem care instruieste agentul, dar si din procesul de antrenare.

Antrenarea pentru scrierea, folosirea si executia de cod a devenit o prioritate pentru agenti, modelele de limbaj fiind antrenate explicit folosind RL sa foloseasca secvente de cod pentru facilitarea rezolvarii problemelor care ar beneficia dupa urma unei astfel de abordari. Aceasta logica este utilizata si pentru testarea de baza a ideilor si explorarea acestora inainte de implementarea propriu zisa, agentul ruland un mic script demonstrativ in terminal pentru a dovedi ca principiul pe care se bazeaza idea ce urmeaza a fi implementata este valid.

3.7.2 Modul de planificare

Planificarea sarcinilor complexe foloseste principiul Chain of Thought [9]. Astfel, executia unei sarcini complexe este impartita in doua parti: planificare si executie.

⁴Un hook este un punct de interceptare a executiei unei functii care permite inserarea altor blocuri logice ce sunt rulate in momentul apelarii acestui hook

⁵Codarea SHA este cea mai comuna metoda de verificare a integritatii unui fisier. Este folosita un algoritm care calculeaza un cod unic in functie de toate datele continute de un fisier, astfel orice mica modificare a fisierului schimba complet codul, fara a fi necesara citirea integrala a acestuia de fiecare data

În prima fază, agentul realizează cercetarea necesară pentru implementare și documentează totul într-un fișier. Mai apoi, se va folosi de contextul din acel fișier pentru a ține minte toate punctele pe care trebuie să le bifeze. Multe astfel de moduri de planificare împart planul în câteva secțiuni bine delimitate printre care se regăsesc: ideea și descrierea planului, fișiere afectate, modificări punctuale și cel mai important niste cazuri pe care acesta trebuie să le bifeze, împartându-și o execuție lungă în mai multe sub-sarcini mai mici, individuale și simple.

Acest mod de planificare nu este nimic mai mult decât o schimbare de prompt de sistem într-o combinație cu o antrenare de tip RL care recompensează crearea de planuri detaliate și consistente în situații care necesită așa ceva.

Modul de planificare reprezintă materializarea SDD în contextul programării de către sisteme agentice. O altă abordare folosită în aceste sisteme este aceea de TDD, care permite mai întâi experimentarea și scrierea de cod fără o documentație bine pusă la punct, dar testarea amanunțită la final.

3.7.3 Skills

Skill-urile sau capacitățile nu sunt decât un set predefinit de instrucțiuni pentru realizarea punctuală a unei sarcini repetitive și sunt compuse din două componente care se află într-un fișier de tip Markdown pentru formatare augmentată.

Prima componentă este în format YAML Ain't Markup Language (YAML), și reprezintă date referitoare la skill, precum numele, descrierea, o recomandare pentru cazuri în care utilizarea acestuia este dorită, și un steag ce setează activarea acestei capacități. Aceste date sunt disponibile agentului (dacă skill-ul este activat) și sunt inserate în contextul tuturor conversațiilor pentru ca modelul să aibă acces la lista de skill-uri din care poate alege.

A doua componentă este reprezentată de capacitatea propriu-zisă. Aceasta este formată din instrucțiuni scrise în format textual, în limbaj natural. De obicei aceste instrucțiuni sunt structurate în pași bine definiți, cu reguli și exemple.

```

1 ---
2 name: pr-review
3 description: Structured pull-request review - risk surface, testing, naming, error handling.
4 when_to_use: When the user asks for a code review, PR review, or feedback on a diff.
5 enabled: false
6 ---
7 You are now performing a pull-request review. Walk through the diff and respond with these
  sections, in this order. Skip a section only if there's truly nothing to say there.
8
9 1. Summary - one short paragraph: what does this change do, in plain language.
10 2. Risk surface -- what could break in production. Concurrency, data migrations, error
  paths, rollback story, blast radius.
11 3. Correctness -- bugs, off-by-ones, race conditions, mishandled None/empty cases, wrong
  types, security issues (injection, auth, secrets).
12 4. Tests -- is the change covered. Are the tests testing the right thing. Any missing edge
  cases.
13 5. Style & naming -- readability nits only if they actually hurt comprehension. Don't
  bikeshed.
14 6. Suggestions -- concrete proposed changes, ideally with a snippet.
15
16 Rules of engagement:
17 - Be specific: cite file paths and line numbers.
18 - Distinguish blockers from nits - say so explicitly.
19 - If the diff is too large to review thoroughly, say so and pick the riskiest files first.
20 - No fluff, no praise sandwich. The reader is technical and time-constrained.

```

Figura 3.10: Implementarea unei capacități de PR review

Utilizarea cea mai frecventă a acestor capacități este pentru a asigura execuția consistentă și reproductibilă (deoarece agentul primește același prompt de fiecare dată) a unui flux de

executie dorit in rezolvarea unei sarcini.

3.7.4 Subagenti

Odata cu cresterea capabilitatilor de gandire ale agentilor, a aparut conceptul de echipa de agenti. In aceasta echipa, unul este agentul principal numit si orchestrator, care comanda executia subagentilor si centralizeaza informatiile returnate de acestia. Acesta are contextul integral al problemei si gandeste la nivel inalt. Subagentii nu au context decat pentru partea lor de problema, iar la finalul executiei sau cercetarii, comunica orchestratorului un rezumat al descoperirilor sau al actiunilor adoptate.

Un subagent este intr-o forma simplificata, un alt apel la un model de inteligenta artificiala cu o alta personalitate si care trebuie sa rezolve o problema specifica. Intr-un mod simplist, acesta are acces la un numar limitat sau specializat de unelte pe care numai el le poate folosi si are un prompt de sistem diferit care ii da instructiuni cu privire la task-ul specific pe care acesta trebuie sa il duca la bun sfarsit. De asemenea ii este comunicat formatul in care trebuie sa returneze raspunsul orchestratorului si faptul ca acesta nu trebuie sa iasa din cuvantul agentului principal.

Subagentii au aceleasi limitari precum agentul principal, de exemplu contextul, iar datorita faptului ca uzual subagentii folosesc modele mai putin performante decat modelul principal si in acest caz este necesara compactarea contextului. Agentul orchestrator porneste agentii secundari uzual printr-o unealta speciala suportata de framework-ul in care este dezvoltat, in cazul acesta, LangGraph, precum in figura 3.11. In final, si acestia trebuia monitorizati si instrumentati pentru inregistrarea corecta a metricilor.

```

1 @tool
2 async def spawn_subagent(
3     subagent_type: str,
4     prompt: str,
5     timeout_s: int | None = None,
6     tool_call_id: Annotated[str, InjectedToolCallId] = "",
7 ) -> str:
8     """Spawn a specialised subagent to handle a focused sub-task in isolation.
9
10    The subagent runs its own tool-calling loop with a filtered toolset and a
11    dedicated system prompt, then returns a single final report. You only see
12    that report -- the subagent's intermediate steps are not surfaced. Use
13    this to parallelise independent work or to delegate a deep dive without
14    polluting your own context.
15
16    Pick a 'subagent_type' that matches the work:
17    - 'general' -- open-ended research / multi-step tasks (full toolset)
18    - 'code_search' -- read-only code lookups (paths, symbols, snippets)
19    - 'web_research' -- web search + synthesis (no file/shell access)
20
21    Call this tool multiple times in one assistant turn to spawn subagents
22    in parallel -- they run independently.
23
24    Args:
25        subagent_type: One of the registered types above.
26        prompt: The task description for the subagent. Be specific: it has
27            none of your conversation context.
28        timeout_s: Optional hard timeout in seconds. Defaults to the type's
29            built-in default; capped by config.
30    """
31
32    from src.subagents import run_subagent
33
34    parent_ctx = _ctx()
35    result = await run_subagent(
36        subagent_type=(subagent_type or "").strip(),
37        prompt=(prompt or "").strip(),
38        parent_ctx=parent_ctx,
39        timeout_s=timeout_s,
40        parent_tool_call_id=tool_call_id or None,
41    )
42    # Surface result + usage to the parent via the ctx so the tools node
43    # can log a SUBAGENT_RESULT session event with proper token attribution.
44    if tool_call_id:
45        parent_ctx.setdefault("pending_subagent_results", {})[tool_call_id] = {
46            "subagent_type": (subagent_type or "").strip(),
47            "prompt": (prompt or "").strip(),
48            "text": result.text,
49            "iterations": result.iterations,
50            "tool_call_count": result.tool_call_count,
51            "timed_out": result.timed_out,
52            "error": result.error,
53            "duration_s": result.duration_s,
54            "usage": result.usage,
55        }
56    return result.text

```

Figura 3.11: Implementarea unei functii pe care agentul o foloseste pentru a porni un subagent

La momentul actual *nici un produs de pe piata nu permite configurarea in detaliu a subagentilor*. Implementarea noastra permite **configurarea completa a fiecarui subagent**, precum modelul utilizat pentru fiecare subagent in parte, promptul de sistem, posibilitatea unui subagent de a porni subagenti proprii si alte setari precum numarul maxim de iteratii per subagent, adancimea maxima de subagenti, si timpul maxim pe care orchestratorul il asteapta pentru a primi un raspuns. Configurarea de baza poate fi consultata in figura 3.12

```

1 subagents:
2   enabled: true
3   default_timeout_s: 120
4   max_timeout_s: 600
5   max_depth: 2 # max recursive subagent spawn
6   types:
7     general:
8       model_override: opus4-7 # model
9       override
10      max_iterations: 25
11      default_timeout_s: 180
12      code_search:
13        model_override: sonnet4-6
14        max_iterations: 15
15        default_timeout_s: 90
16      web_research:
17        model_override: gemma4
18        max_iterations: 18
19        default_timeout_s: 120
20      max_parallel: 3
21      child_context:
22        max_tool_tokens: 2000 # tool_result
23        body_cap (tokens, ~4 chars each)
24        head_tail: [800, 400] # head/tail
25        excerpt sizes when truncating

```

Figura 3.12: Configurarea subagentilor

3.8 Extensibilitate

Deoarece aplicatia este gandita pentru a fi open-source, arhitectura aplicatiei trebuie sa fie usor extensibila de utilizatori ce doresc sa aduca modificari in modul de functionare, in special in punctele de extindere. Cateva din acesta puncte de extindere gandite de la bun inceput sunt: logica de compactare, setul de unelte, setul de subagenti, setul de furnizori si setul de skill-uri.

3.8.1 MCP si adaugarea de unelte

Aplicatia contine un client MCP pentru adaugarea uneltelor externe si integrarea cu servicii tert. Acesta accepta metodele de comunicare comune in industrie: STDIO, HTTP si SSE. Pentru a configura un server MCP si a accesa uneltele pe care le pune la dispozitie in aplicatie utilizatorul poate utiliza comanda `/mcp` in interiorul interfetei, iar serverul va fi incarcat automat sau poate adauga personal serverul in fisierul `/.vibe-cli/preferences.json` si sa reporneasca aplicatia.

Serverele MCP pot expune atat unelte cat si skill-uri sau prompt-uri. Si pot fi reincarcate sau gestionate prin comanda `/mcp`

Adaugarea de unelte poate fi facuta cu usurinta, fiind o aplicatie open-source. Daca utilizatorul este un dezvoltator si a clonat proiectul local, acesta poate adauga o unealta dupa modelul din figura 3.8 si sa isi implementeze propria logica. Uneltele sunt definite la calea `src/tools.py`. Dupa adaugarea uneltelor, aplicatia trebuie reinstalata/repornita.

3.8.2 Adaugarea de subagenti

Adaugarea de subagenti este la fel de simpla precum adaugarea de unelte. Utilizatorul trebuie doar sa isi adauge un fisier pentru agentul sau in `src/subagents/types` precum modelul furnizat in figura 3.13. Crearea unui nou subagent implica pe scurt extinderea clasei `Subagent` si completarea campurilor cu date despre subagent precum: nume, descriere, rol.

Optiunea de utilizare apartine utilizatorului, impreuna cu configurarea acestora. Aplicatia furnizeaza 3 subagenti de inceput: cel general, cel pentru cercetarea codului si cel pentru cercetari online. Fiecare vine cu setul sau de instructiuni si unelte. Utilizatorul poate alege ce model foloseste pentru fiecare, cat de lungi pot fi cautarile sale si timpul de asteptare dupa raspuns al subagentilor. De asemenea o alta setare de interes este posibilitatea subagentilor de a porni multipli subagenti proprii. Agentii sunt executati uzual in paralel si pot exista mai multi agenti de acelasi tip ce ruleaza in paralel cu instructiuni diferite. Acest lucru poate fi limitat de asemenea din configuratie. In final, contextul subagentilor are si el o setare pentru gestionarea raspunsului de la unelte.

```

1 @register
2 class CodeSearchSubagent(Subagent):
3     name = "code_search"
4     description = (
5         "Fast read-only code search. Finds files/functions/symbols and "
6         "reports paths + line ranges + relevant snippets. Cannot write or "
7         "execute side-effectful commands."
8     )
9     default_timeout_s = 90
10    max_iterations = 15
11    # Read-only toolset. 'run_command' is included for grep/rg/find use.
12    allowed_tools = ["read_file", "list_directory", "run_command", "fetch_artifact"]
13
14    def role_prompt(self, prompt: str, parent_ctx: dict) -> str:
15        return (
16            "ROLE: read-only code-search subagent.\n\n"
17            "Your job is to LOCATE code that matches the orchestrator's "
18            "request and report findings concisely. You are an advisor only "
19            "- even though 'run_command' is available, it is for read-only "
20            "searches (rg / grep / find / ls / cat / wc). NEVER invoke "
21            "installers, builds, test runs, package managers, or any "
22            "command that mutates state.\n\n"
23            "Stop searching once you have enough to answer; do not enumerate "
24            "exhaustively.\n\n"
25            "Final report format:\n"
26            " - For each match: 'path:line' + a one-line description.\n"
27            " - When useful, include a short snippet (<=6 lines).\n"
28            " - End with a one-sentence takeaway for the orchestrator."
29        )

```

Figura 3.13: Inregistrarea unui nou subagent

În final, agentul adăugat și pornit din fișierul de configuratie `config.yml` conform figurii 3.12. Pentru a se aplica schimbările aplicația trebuie reînaltată/repornită.

Definiția unui subagent depășește însă simpla pereche nume/descriere: fiecare tip fiind o subclasă Python a clasei `Subagent` înregistrată prin decoratorul `@register`, ceea ce oferă dezvoltatorului un set bogat de puncte de extensie:

- `role_prompt(prompt, parent_ctx)` – fragment specific de rol, atașat contractului comun `SUBAGENT_CONTRACT`, cu acces la prompt și la contextul părintelui pentru adaptare dinamică;
- `allowed_tools` / `excluded_tools` – listă de comenzi permise și blocate intersectate cu setul activ de unelte al părintelui;
- `tool_names(parent_ctx)` și `system_prompt(prompt, parent_ctx)` – hook-uri prin care setul de unelte, respectiv prompt-ul de sistem, pot fi recalculat la fiecare invocare;
- `max_iterations` și `default_timeout_s` – limite pentru bucla de tool-calling, suprascriibile din `subagents.types.<tip>` în `config.yml`;
- model dedicat per tip, configurabil prin `/subagents <tip> <provider> <model>`, cu fallback pe modelul principal de chat.

La execuție, fiecare subagent rulează o buclă completă de tool-calling `LangChain` cu același flux de permisiuni și aceleași `tool_guards` deterministe ca părintele (de exemplu `read-before-write`, prin partajarea `ctx["file_reads"]`), beneficiază de compactare locală a mesajelor de tip `ToolMessage` supradimensionate, de telemetrie `Langfuse` cuibărită sub span-ul părintelui, de agregarea consumului de token-uri (`input/output/cache_read/cache_write`) și de respectarea limitelor `subagents.max_depth` și `subagents.max_parallel`. În cazul în care modelul iese din buclă fără raport final, este reapelat o singură dată pentru a emite raportul.

Modelul actual nu suporta definitii declarative bazate pe fisiere Markdown (analog skill-urilor), scoping de servere MCP per subagent, streaming intermediar al iesirii subagentului catre orchestrator, si nici politici de *retry/replan* dincolo de `max_iterations`.

3.8.3 Creare de skill-uri

Crearea de skill-uri este reprezentata de scrierea unui fisier de tip Markdown si de plasarea acestuia fie in containerul global de skill-uri aflat la `/.vibe-cli/skills`, fie in folderul `.vibe` din cadrul proiectului in care se lucreaza curent pentru skill-uri locale. Pentru aplicarea schimbarilor sau gestionarea skill-urilor, utilizatorul poate folosi comanda `/skills`

3.8.4 Suportul pentru furnizori de modele de limbaj

LangGraph furnizeaza un strat grozav de abstractizare a furnizorilor de modele de limbaj. Totusi, chiar si cu acest ajutor, adaugarea unui nou furnizor implica schimbari in mai multe module ale aplicatiei, dar si constrangerea suportului pentru furnizori suportati de LangGraph. Pentru a reduce aceste schimbari la minimum, intreaga suprafata a furnizorilor este concentrata in modulul `src/providers.py`, unde fiecare furnizor este o subclasa a clasei `BaseProvider`. O instanta a noii clase este inregistrata in dictionarul `PROVIDER_REGISTRY`, iar restul aplicatiei (factory-ul `get_model` din `src/main.py`, selectia modelului per subagent din `src/subagents/base.py`, comenzile `/model` si `/key` din interfata, precum si validarea si persistarea cheilor API din `ui/provider_handler.py`) deriva automat comportamentul din acest registru.

O subclasa `BaseProvider` declara cateva elemente esentiale:

- `name` - cheia utilizata in `config.yml` si in `preferences.json`;
- `env_var` - numele variabilei de mediu (sau lista de nume, rezolvate in ordine) pentru cheia API, sau `None` pentru furnizorii care nu necesita autentificare, precum Ollama;
- `validate(cfg)` - o verificare ieftina, fara apel LLM, de tipul `models.list()`;
- `create_model(cfg)` - metoda care returneaza instanta `ChatModel` LangChain. Optional, sunt disponibile carlige precum `base_url(cfg)` pentru furnizori locali si `unreachable_help(cfg)` pentru mesaje de ajutor afisate cand validarea esueaza.

Concret, adaugarea unui nou furnizor presupune patru modificari mici:

- o noua subclasa in `src/providers.py` adaugata in `PROVIDER_REGISTRY`;
- un bloc `providers.<nume>` in `config/config.yml` cu `api_key: ${ENV_VAR}`, lista de modele si temperatura, plus eventuale campuri specifice (de exemplu `region` pentru Bedrock sau `base_url` pentru Ollama);
- tarifele per model in `web/data/pricing.py`, sau adaugarea numelui furnizorului in `_FREE_PROVIDERS` daca este local sau gratuit, astfel incat estimarea costurilor din panoul web sa functioneze corect.

In final, LangGraph fiind un proiect open-source, dependenta de suportul pentru furnizorii de modele de limbaj nu este o problema, comunitatea putand oricand sa adauge suport.

3.8.5 Adaugarea tehnicilor de compactare

Pipeline-ul de compactare este construit ca un sistem de extensii inmagazinate intr-un registru, astfel incat adaugarea de strategii noi nu necesita modificari exhaustive ale codului. Fiecare strategie este o subclasa a uneia dintre clasele de baza `Stage` (executie *eager*, per eveniment) sau `ThresholdStage` (executie la atingerea pragului de ocupare, cu vizibilitate asupra intregului istoric), declarata in `src/compaction/stages/` si inregistrata explicit in `src/compaction/__init__.py`. Adaugarea unei strategii noi presupune patru modificari:

- implementarea metodei `apply_eager(event, *, store)` sau `apply_threshold(tctx)`, care schimba `event["projected"]`, respectiv declara identificatorii evenimentelor eliminate;
- inregistrarea clasei in `EAGER_REGISTRY` sau `THRESHOLD_REGISTRY`;
- optional, adaugarea ei intr-un grup de exclusiune mutuala `EAGER_EXCLUSION_GROUPS` daca scrie in acelasi camp al evenimentului ca o strategie existenta (validarea esueaza la pornire cu `CompactionConfigError` daca doua strategii incompatibile sunt active simultan);
- configurarea ei in `config/config.yml` sub `compaction.pipeline`.

Strategiile asincrone care depind de un model de limbaj urmeaza un protocol in doua etape: o prima rulare sincrona produce un rezultat sigur (de exemplu, trunchiere cap-coada) marcat cu starea `truncated_pending`, iar o sarcina de fundal pornita prin `ectx.spawn(...)` inlocuieste ulterior proiectia cu varianta sumarizata si o rescrie in sesiune prin `on_event_updated`. In caz de esec sau indisponibilitate a modelului, starea trece in `truncated_fallback`, garantand astfel ca bugetul de tokeni este respectat indiferent de rezultatul apelului LLM. Campul `event["content"]` ramane in toate cazurile imutabil, iar orchestratorul realizeaza inainte de fiecare trecere o copie de siguranta a fisierului de sesiune, astfel incat orice transformare este reversibila.

Pe baza acestui mecanism, urmatoarele strategii sunt candidate naturale pentru extinderea setului de strategii de compactare:

- **memoizarea apelurilor de unealta:** indexarea rezultatelor dupa perechea (nume, argumente normalizate) si reutilizarea lor pentru apeluri identice ulterioare, plasata in pre-dispatch-ul nodului de permisiuni, in afara pipeline-ului propriu-zis;
- **proiectia diferentiala a citirilor repetate:** pentru reapelarea `read_file` pe aceeasi cale, inlocuirea rezultatului cu un `diff` unificat fata de citirea anterioara, sau cu marcajul `[unchanged]` cand suma SHA coincide, valorificand mecanismul `ctx["file_reads"]` deja existent;
- **fereastră glisanta de fapte ancorate:** un `SystemMessage` persistent, regenerat la fiecare trecere de prag, care pastreaza obiectivele utilizatorului, conventiile proiectului si fisierele aflate in focus, transportat printr-un tip de eveniment rezervat `KIND_ANCHOR` ce nu este niciodata eliminat;
- **proiectori structurati per unealta:** extractoare declarative care pastreaza doar campurile efectiv consumate de agent (de exemplu `{title, url, snippet}` pentru cautarea web, `nume + tip` pentru listarea de directoare, codul de iesire si ultimele linii pentru comenzile de terminal), oferind un compromis intre costul si agresivitatea celor existente;

Strategiile bazate pe scorare de importanta cu LLM ca arbitru au fost evaluate si respinse pentru aceasta etapa, deoarece sunt acoperite deterministic de `importance_prune` si `bulk.hybrid`, iar caracterul lor nereproductibil si costul suplimentar nu justifica adoptarea.

3.9 Securitatea in mediul agentic

Atunci cand incredintam controlul deplin, sau cel putin accesul la comenzi cu potential periculos, trebuie sa avem niste masuri de siguranta pentru a ne asigura ca agentul nu incearca voit sau accidental sa intreprinda o actiune nedorita.

Exista multe motive pentru care un agent ar putea incerca sa ruleze comenzi periculoase, iar taxonomia recenta a amenintarilor specifice sistemelor agentice [30] le grupeaza in trei mari clase.

In primul rand, *prompt injection* indirect: continutul ingerat de agent dintr-un fisier, o pagina web sau iesirea unei unelte poate contine instructiuni adversariale care detorneaza obiectivul initial, fenomen demonstrat empiric impotriva unor solutii de productie precum Azure Prompt Shield si Meta Prompt Guard, unde tehnici simple de perturbare la nivel de caracter sau bazate pe Adversarial Machine Learning (AML) ating rate de evaziune apropiate de 100% [31].

In al doilea rand, *utilizarea gresita a uneltelor*, fie prin halucinarea argumentelor, fie prin compunerea unor secvente de apeluri inofensive individual, dar distructive in ansamblu (de exemplu `rm -rf` aplicat unei cai derivate eronate).

In al treilea rand, *data poisoning* la nivelul memoriei sau al contextului persistent, prin care un atacator planteaza fapte false ce vor fi recuperate si tratate drept context autoritar la invocari ulterioare.

In ultimul rand, un alt motiv poate fi frustrarea. In unele cazuri, agentul determina ca cel mai usor mod de a rezolva o eroare este de a sterge cu totul fisierul sau functionalitatea care produce aceasta eroare.

Pentru ca aliniamentul modelului [32] si filtrele de tip guardrail ⁶ [33], [34] nu ofera garantii suficiente in fata acestor vectori, aplicatia adopta o aparare in adancime, structurata pe trei niveluri descrise in continuare: izolarea executiei prin sandboxing, restrictionarea actiunilor prin permisiuni explicite si delegare catre subagenti cu privilegii reduse, si interceptarea cererilor periculoase sau in afara scopului prin guardrails dedicate.

3.9.1 Sandboxing

Tehnica de sandboxing reprezinta rularea comenzilor intr-un mediu izolat, bazat pe principiul *least privilege*. In aplicatie, mecanismul izoleaza comenzile de shell ale agentului intr-un container Docker dedicat sesiunii curente, in care directorul de lucru al utilizatorului este montat prin `bind-mount` la `/workspace`, astfel incat modificarile asupra fisierelor se propaga imediat catre gazda, fara a fi necesara o etapa explicita de sincronizare. Functionalitatea este *opt-in* si degradeaza elegant la executia pe gazda atunci cand procesul Docker nu este disponibil, pastrand astfel portabilitatea aplicatiei.

Configurarea este expusa printr-o sectiune dedicata in `config.yml` conform figurii 3.14. La pornirea sesiunii, aplicatia citește aceasta configuratie, sondeaza disponibilitatea daemonului cu un *timeout* scurt si aplica o politica de validare a directorului de lucru organizata pe

⁶Guardrail-ul reprezinta o combinatie intre instructiuni de prompt care reglementeaza comportamentul modelului pentru a nu executa sau intoarce catre utilizatoru secvente nedorite, cat si filtre deterministe ce blocheaza executia sau intoarcerea catre utilizator a acestor fraze nedorite

trei niveluri: refuzul ferm al cailor sensibile (`/`, `/tmp`, `$HOME` si subdirectoarele sale directe), acceptul automat al directoarelor care contin marcatori de proiect (`.git`, `pyproject.toml` etc.) si validarea pe baza de dimensiune si numar de fisiere pentru cazurile ambigue. Politica poate fi suprascrisa explicit de utilizator prin variabila de mediu `VIBE_SANDBOX_FORCE`.

```

1 sandboxing:
2   enabled: false
3   image: python:3.13-slim
4   network: bridge # bridge / none
5   memory: 2g
6   cpus: 2
7   timeout_default: 30 # default 'run_command'
8     timeout in seconds
9   blocked_paths:
10    - "~/"
11   ambiguous_max_mb: 500
12   ambiguous_max_files: 10000

```

Configuratia permite controlul parametrilor de izolare: imaginea de baza, modul de retea, limitele de memorie si Central Processing Unit (CPU), timpul implicit de executie, lista de cai interzise si pragurile (dimensiune maxima, numar maxim de fisiere) ce delimiteaza directoarele considerate ambigue.

Figura 3.14: Configurarea sandbox-ului Docker pentru executia izolata a comenzilor

Containerul este pornit cu `docker run -d -rm`, avand toate capabilitatile retrase (`-cap-drop ALL`), executand sub identitatea numerica a utilizatorului gazda (`-user <uid>:<gid>`), cu limitele de memorie, Central Processing Unit (CPU) si retea impuse de configuratie, si traieste pe toata durata sesiunii. Toate comenzile sunt rulate ulterior prin `docker exec`. In caz de esec la pornire, modulul de sandboxing intoarce o pereche (`None`, motiv) fara a propaga exceptia, iar orchestratorul continua executia pe gazda. Inchiderea containerului este garantata la terminarea sesiunii printr-o rutina dedicata.

Rutarea apelurilor este realizata la nivelul uneltei `run_command`, care consulta la fiecare invocare un `ContextVar` ce contine eventualul `handle` al sandbox-ului si delega executia fie containerului, fie gazdei. Uneltele de fisiere (`read_file`, `write_file`, `list_directory`) raman pe gazda, deoarece `bind-mount`-ul face cele doua perspective echivalente, iar evitarea apelului prin Docker reduce semnificativ latentia. Atunci cand sandbox-ul este activ, setul de unelte este extins cu `run_command_host`, o *escape hatch* explicit conditionata de permisiuni, care permite executia controlata pe gazda atunci cand este strict necesar. Subagentii mostenesc `handle`-ul prin copierea superficiala a contextului parintelui, astfel incat toate apelurile imbricate de unealta impartasesc acelasi container si beneficiaza in mod uniform de izolare.

Garzile transversale (de exemplu `read_before_write`) si verificarile de permisiuni se aplica identic pentru apelurile rutate catre sandbox si pentru cele executate pe gazda, fiind plasate inaintea executiei propriu-zise. In acest fel, izolarea oferita de container reprezinta o linie de aparare suplimentara, complementara mecanismului de permisiuni descris in continuare, si nu il inlocuieste.

3.9.2 Permisii

O masura de aparare necesara este aprobarea executiei de unelte de catre utilizator. Unele unelte, precum cele de citire din fisiere sunt aprobate din oficiu pentru a scadea necesitatea de a cere permisiuni omului. In schimb, omul este cel care are ultimul cuvint in executia comenzilor. Acesta poate opta sa aprobe fiecare comanda in parte, toate comenzile de un tip in timpul sesiunii curente sau permanent, precum in figura 3.15. Acestuia i se prezinta ce unealta se doreste a fi folosita si cu ce argumente.

Toate aceste restrictii privind permisiunile se aplica atat agentului orchestrator cat si subagentilor.



Figura 3.15: Cererea de aprobare a permisiunii de a scrie un fisier

3.9.3 Guardrails, intentii periculoase si cereri in afara scopului (OOS)

O aplicatie in productie trebuie sa raspunda doar la cereri care corespund cerintelor pentru care a fost proiectat. Cu un sistem nedeterminist precum cele care folosesc modele de limbaj, acest lucru este exceptional de dificil. In ciuda diverselor metode de combatere a acestor incercari de a face aplicatia sa faca ceva ce nu ar trebui sa faca, uzual inca exista o sumedenie de metode prin care acestea pot fi pacalite.

In general se folosesc o combinatie de instructiuni de prompt, antrenament, selectii de cuvinte interzise, sau un apel catre un model extern care ar decide daca cererea este valida sau nu.

De asemenea, furnizorii de modele de limbaj au propriile metode de aparare prin care incearca sa combata acest lucru.

Aplicatia nu prezinta momentan astfel de filtre, bazandu-se pe cele impuse de furnizorii de modele.

3.10 Observabilitate

Observabilitatea aplicatiei este oferita pe doua niveluri complementare: tracing extern, prin integrarea cu Langfuse Cloud, si introspectie locala, prin panoul web analitic livrat impreuna cu aplicatia. Ambele componente sunt optionale, dezactivate implicit, si nu transmit date catre alte servicii in afara celor configurate explicit de utilizator.

3.10.1 Langfuse

Pentru tracing extern, aplicatia integreaza Langfuse Cloud prin intermediul unei interceptari la graful LangGraph al agentului. Activarea functionalitatii se face fie prin editarea fisierului `config.yml`, fie, prin comanda interactiva `/tracing setup`, care persista cheile in `preferences.json`. Rezolvarea credentialelor urmeaza o ordine deterministica de precedenta: preferintele locale au prioritate, urmate de fisierul de configuratie si, in final, de variabilele de mediu `LANGFUSE_*`.

La fiecare invocare a grafului, evenimentele sunt interceptate si publicate catre Langfuse, asincron. Datele incarcate in cloud sunt sub forba unui trace⁷ principal care contine intregii ture de conversatie, alaturi de span-uri⁸ ierarhice pentru fiecare apel catre modelul de limbaj, fiecare

⁷Un trace este un mesaj de tip JSON care contine date. Este folosit in monitorizarea distribuita si reprezinta 'urma' sau 'dovada' unor actiuni si contine detalii despre acestea. Un trace poate contine mai multe obiecte.

⁸Un span reprezinta o unitate de observare a unei singure actiuni

invocare de unealta si fiecare traversare de nod din graf. Pentru fiecare span sunt inregistrate printre altele: numele modelului, intrarile, iesirile, consumul de tokeni si latentă. Trace-urile sunt etichetate cu identificatorul sesiunii curente (Universally Unique Identifier (UUID)v7 generat local), un identificator anonim al utilizatorului si tag-uri descriptive (de ex. modul activ, furnizorul si modelul selectat), permitand astfel agregarea per sesiune si filtrarea ulterioara in interfata Langfuse.

Pe baza identificatorului ultimului trace, doua comenzi suplimentare ofera bucla de feedback necesara evaluarii continue: `/score` adauga scoruri numerice sau calitative ultimei interactiuni, iar `/dataset` adauga turul curent intr-un set de date persistent, util pentru benchmarking si teste de regresie comportamentala. In combinatie cu mecanismul *LLM-as-judge* oferit de Langfuse, aceste comenzi formeaza fundamentul unei suite de regresie pentru calitatea raspunsurilor agentului.

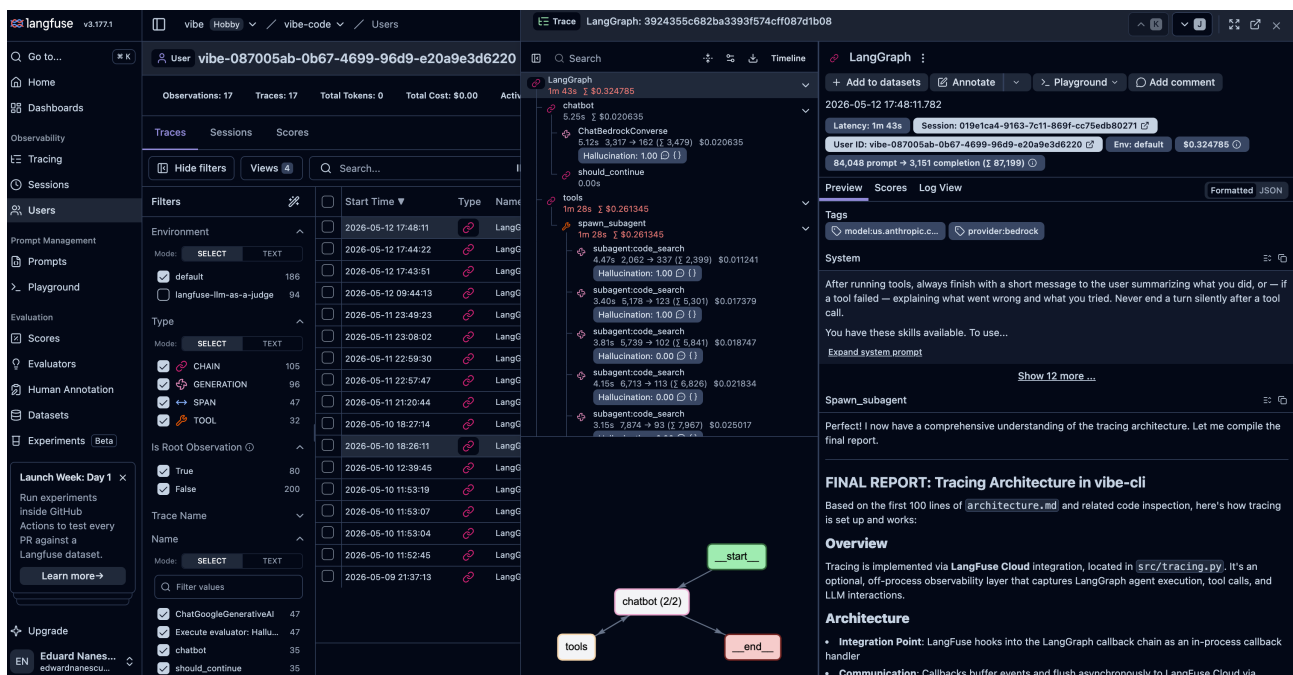


Figura 3.16: Interfata Langfuse

In figura 3.16 putem observa interfata Langfuse si capabilitatile pe care ni le ofera.

In partea stanga avem navigarea care ne permite sa accesam diferitele functionalitati, printre care si sortarea trace-urilor dupa sesiuni si utilizatori, rubrica de management al prompt-ului in care putem stoca si versiona diferite prompt-uri pentru testare si a determina cauzalitatea intre prompt-uri si degradari de performanta. In final avem si zona de calitate unde putem agrega recenziile utilizatorilor (spre ex. din butoanele de like/dislike ale conversatiei cu o interfata web a unui asistent de tipul ChatGPT), evaluatori automati ce ne ofera scoruri pentru diferite categorii si chiar si verificare umana.

In partea centrala observam ca ne aflam in meniul *Users*, unde putem vedea toate sesiunile (si trace-urile daca deschidem o sesiune) facute de un utilizator, cu filtre si alte mecanisme de cautare.

In partea dreapta am deschis trace-ul asociat unei sesiuni si putem observa locul in care Langfuse stralucește. Putem investiga executia conversatiei pas cu pas, investigand graficul de executie, fiecare apel de unealta si raspunsul acestora, gandirea efectiva a modelului de limbaj, intregul istoric al conversatiei in format augmentat cat si metrice cu privire la tokens. De asemenea tot aici putem vedea si evaluarile automate pe care le-am configurat, care ne spun daca raspunsurile agentului sunt corect factual sau sunt halucinate.

3.10.2 Panou web

Panoul web este o aplicatie Flask locala, read-only, ce vizualizeaza in mod direct fisierele de sesiune din `~/vibe-cli/sessions/`, fara intermedierea unei baze de date. Aplicatia este lansata prin comanda `vibe web` si este expusa implicit pe `http://127.0.0.1:5050`, cu posibilitatea de a configura adresa si portul prin parametri de linie de comanda.

Interfata, organizata pe o singura pagina, ofera selectoare pentru proiectul si sesiunea analizata si afiseaza, pentru sesiunea selectata, un sumar al consumului de tokeni pe categorii de evenimente, evolutia dimensiunii contextului raportata la limita modelului, frecventa si latentia apelurilor catre unelte (cu percentile p_{50} si p_{95}), debitul de generare exprimat in tokeni pe secunda, costul estimat per apel pe baza unui tabel de preturi configurabil, precum si istoricul evenimentelor de compactare cu diferentele corespunzatoare. Componente dedicate permit inspectarea granulara a fiecarei unelte invocate, cu argumentele si rezultatul aferent, precum si reconstructia turului conversational eveniment cu eveniment.

Sincronizarea cu starea actuala a sesiunilor si a log-urilor este realizata prin SSE, o singura conexiune persistenta fiind multiplexata pentru ambele fluxuri. Detectia modificarilor se face prin compararea metadatelor sistemului de fisiere (cale, timp de modificare, dimensiune) la intervale de 1.5 secunde, iar reincarcarile sunt debounce-uite la 250 de milisecunde pentru a evita actualizari redundante. Un panou inferior afiseaza ultimele 1000 de linii din log-ul textual al aplicatiei, cu auto-derulare inteligenta care suspenda fixarea la baza atunci cand utilizatorul consulta inregistrari anterioare. Atunci cand tracing-ul Langfuse este activ, un buton dedicat din bara superioara deschide proiectul corespunzator in interfata externa, fara a expune cheile API catre browser.

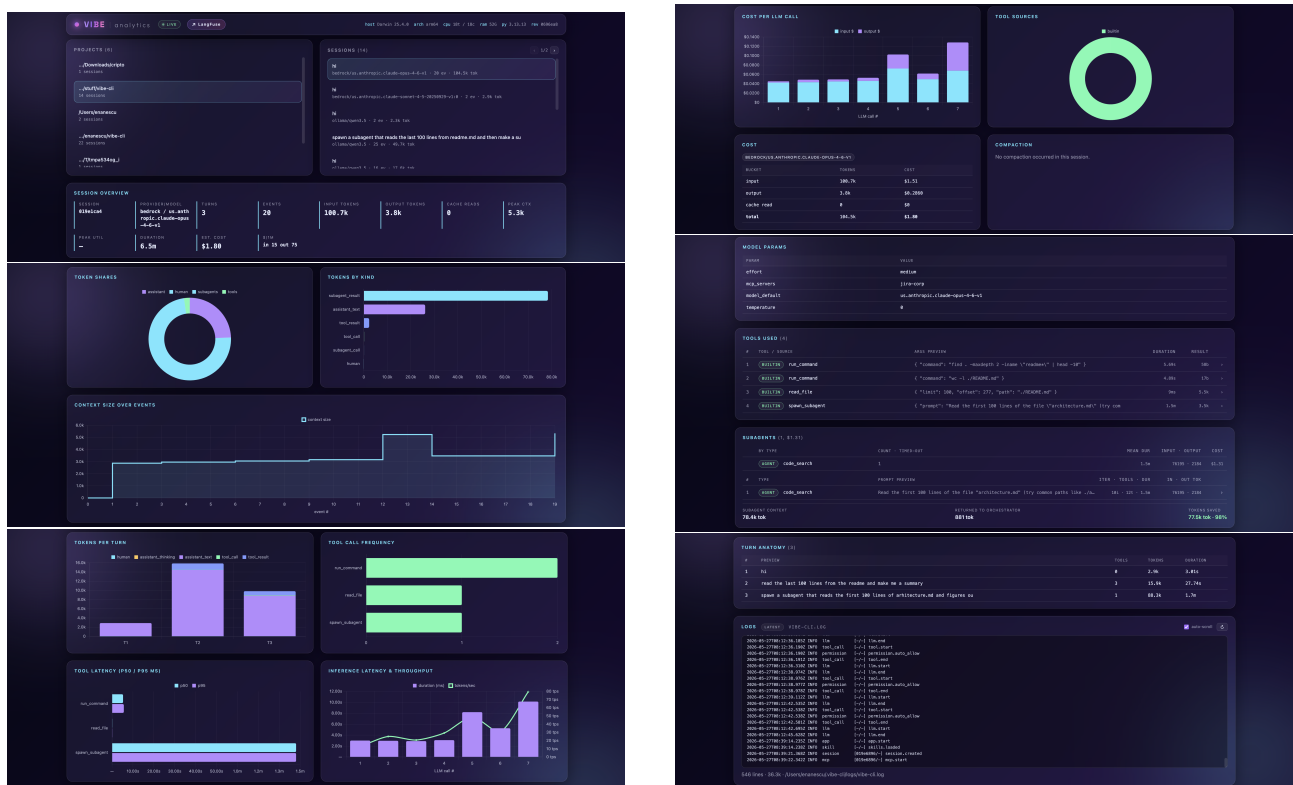


Figura 3.17: Imagini cu interfata web a aplicatiei

Scopul panoului web este acela de a aduce transparenta pentru un produs care are acces la sistemul utilizatorului. Astfel, toate actiunile sunt contorizate, si toate costurile sunt explicate, spre deosebire de restul aplicatiilor disponibile pe piata care ascund aceste numere.

Capitolul 4

REZULTATE EXPERIMENTALE

Avand in vedere finalizarea implementarii, am condus un set de teste in timpul utilizarii asistentului de programare cu diferite variatii ale configuratiei pentru extragerea unui mod optim de utilizare si strangerea de date legate de mai multe aspecte ale folosirii care influenteaza performantele aplicatiei.

Dat fiind natura nedeterminista a raspunsurilor modelelor de limbaj, schimbarile de configuratie precum modelul utilizat sau promptul de sistem, cat si uneltele puse la dispozitie si alte aspecte pot avea un impact consistent asupra datelor de iesire si actiunile intreprinse de agent.

4.1 Pozitionarea fata de concurenta

La finalul implementarii, rezultatul este comparabil din punct de vedere al capabilitatilor dar si al performantei cu aplicatiile existente pe piata in momentul de fata.

Calitatile ce diferentiaza abordarea noastra de restul competitiei sunt: integrarea usoara cu modele OSS ce pot rula local pe masina utilizatorului, scapand de dependenta de un furnizor de modele de limbaj scapand astfel de costurile asociate cu folosirea acestor unelte, transparenta interactiunii cu modelele de limbaj prin panoul web si integrarea cu Langfuse dar si prin optiunile exhaustive de configuratie prin care utilizatorul poate controla majoritatea functionalitatilor vitale.

Pentru a sustine cantitativ aceasta diferentiere, in tabelul 4.1 am rezumat principalele caracteristici ale celor mai populare unelte agentice de programare pentru terminal (*Claude Code* dezvoltata de Anthropic, *OpenAI Codex CLI* dezvoltata de OpenAI si *GitHub Copilot CLI* dezvoltata de GitHub) in raport cu solutia propusa.

Tabela 4.1: Comparatie intre **vibe-cli** si principalele unelte agentice de programare pentru terminal disponibile pe piata

Aplicatie	Furnizori si model de cost	Puncte forte	Puncte slabe (raportat la solutia propusa)
Claude Code (Anthropic) Sursa inchisa, proprietara	Doar modele Anthropic Claude (Amazon Web Services Bedrock si Vertex AI doar in editia enterprise). Abonament Pro/Max sau cheie API cu plata per-token.	• Maturitate si integrare cu modelele Claude • Extensibilitate prin MCP, subagenti, skills, hooks si extensii • Permisuni granulare si sandboxing pe macOS/Linux/Windows • Suprafete multiple (terminal, VS Code, JetBrains, web)	• Cod sursa inchis, dependenta totala de un singur furnizor • Fara suport documentat pentru modele OSS locale • Cost obligatoriu (abonament sau per-token), fara regim gratuit • Fara panou web propriu pentru transparenta; doar consola Anthropic de facturare si comanda pentru context in aplicatie. • Tracing nativ minim (suport partial OpenTelemetry)
OpenAI Codex CLI (GitHub) Licenta Apache-2.0	Doar modele OpenAI. Inlus in plan ChatGPT Plus/Pro/Business sau cheie API cu plata per-token.	• Singura solutie comerciala cu cod sursa complet deschis (Apache-2.0) • Implementare in Rust, performanta ridicata si executabil unic • Suport pentru MCP, subagenti, skills, reguli, interceptari si sandboxing • Autentificare directa cu un cont ChatGPT existent	• <i>Furnizor unic</i>: doar modele OpenAI, fara rutare alternativa documentata • Fara suport pentru modele OSS locale (Ollama, llama.cpp) • Cost obligatoriu (abonament ChatGPT sau cheie API) • Documentatie minima privind transparenta utilizarii (tokeni, cost, istoric) • Fara integrare cu Langfuse sau alt backend de tracing
GitHub Copilot CLI (GitHub Docs) Sursa inchisa, proprietara	Modele gazduite de GitHub (Claude, GPT); rutare optionala catre endpoint-uri OpenAI-compatibile (Ollama, vLLM, Azure OpenAI, Anthropic) prin variabile de mediu. Abonament Copilot Pro/Business/Enterprise + credite consumate per interactiune.	• Singura solutie comerciala cu rutare explicita catre Ollama si endpoint-uri locale • Configurabilitate avansata: agenti per utilizator/repo/organizatie, MCP, skills, hooks, memorie persistenta • Integrare nativa cu workflow-urile GitHub (PR-uri, issue-uri, context de repo) • Comenzi de inspectie a sesiunii (/usage, /context, /compact)	• Cod sursa inchis, abonament Copilot obligatoriu • Pret opac (credite consumate per interactiune, fara mapare publica la cost) • Status de previzualizare publica • Fara panou web sau interfata grafica; analiza disponibila numai prin comenzi in terminal • Fara integrare cu Langfuse sau OpenTelemetry
vibe-cli (GitHub) Licenta Massachusetts Institute of Technology	OpenAI, Anthropic, Google Gemini si Ollama , comutabile pe loc prin /model. Aplicatie gratuita; cost zero pentru rulare locala cu Ollama, sau exclusiv tariful furnizorului ales in regim cloud.	• Suport nativ multi-provider, inclusiv modele OSS locale prin Ollama, fara cheie API • Panou web Flask local, read-only, cu metrici per sesiune (tokeni pe categorii, evolutia contextului, cost estimat, latentia uneltelor cu p_{50}/p_{95}, debit, compactari, reconstructia turului) si Integrare nativa Langfuse prin callback-uri LangChain, cu <i>tagging</i> per subagent si evaluari automate • Suport multimodal (imagini, PDF, fisiere text si documente), conditionat de capabilitatile modelului selectat • Subagenti tipizati paraleli (general, code_search, web_research) cu model selectabil per tip • Configuratie YAML exhaustiva: provideri, MCP, subagenti, skills, <i>tool guards</i> deterministe, permisiuni <i>allow/deny</i>, sandboxing Docker, compactare si tracing	• Maturitate redusa fata de produsele furnizorilor mari, ecosistem in dezvoltare • Fara extensie nativa de IDE (utilizare exclusiv din terminal) • Suport oficial limitat la macOS si Linux (fara instalare nativa pe Windows) • Curba de invatare mai abrupta pentru configurarea avansata din config.yml

In tabelul 4.1, se observa mai multe diferente, in special independenta fata de un furnizor unic de modele, integrarea cu modele locale OSS, panoul web propriu si tracing-ul nativ cu Langfuse, pozitioneaza solutia propusa intr-un punct distinct fata de concurenta, oferind in acelasi

timp un set comparabil de capabilitati agentice (subagenti, MCP, sandboxing, configurabilitate avansata).

In continuare, in tabelul 4.2 prezentam o comparatie cu cei mai relevanti asistenti de programare OSS din ecosistem (*OpenHands*, *Cline* si *Aider*), care impart cu solutia propusa avantajul codului sursa deschis si al independentei fata de un furnizor unic de modele. Mentionam ca *OpenClaw*, desi este cel mai popular proiect OSS din tabelul 2.2, nu a fost inclus in aceasta comparatie deoarece este pozitionat ca asistent personal multi-canal (mesagerie, voce, automatizari), nu ca asistent dedicat dezvoltarii software.

Tabela 4.2: Comparatie intre *vibe-cli* si principalii asistenti de programare OSS din ecosistem

Aplicatie	Furnizori si model de cost	Puncte forte	Puncte slabe (raportat la solutia propusa)
OpenHands (GitHub) Massachusetts Institute of Technology	Agnostic fata de furnizor (Claude, GPT, Ollama si altele). Aplicatie gratuita; cost limitat la tariful furnizorului ales.	• Sandboxing robust in containere Docker • Interfata grafica web si cli disponibile in paralel • SDK Python pentru construirea de agenti personalizati • Suport Kubernetes si scenarii enterprise • Integrari Slack, Jira, Linear	• Cerinte de resurse ridicate, Docker obligatoriu pentru fluxul principal • Configurare initiala complexa, curba de invatare abrupta • Fara integrare nativa Langfuse pentru tracing si evaluari automate • Fara panou de transparenta cost si tokeni la nivel de sesiune • Fara suport explicit pentru subagenti tipizati cu model dedicat per tip
Cline (GitHub) Apache 2.0	Agnostic, cu rutare prin OpenRouter (200+ modele: Anthropic, OpenAI, Ollama). Aplicatie gratuita; cost limitat la tariful modelului ales.	• Extensie nativa pentru VS Code si plugin JetBrains • SDK Node.js si cli headless pentru CI/CD • Echipe multi-agent si agenti programati • Suport MCP si plugin-uri extensibile • Acces la peste 200 de modele prin OpenRouter	• Consum ridicat de tokeni datorita contextului mare incarcat permanent • Fara sandboxing nativ incorporat • Dependent de ecosistemul VS Code/JetBrains, fara aplicatie standalone • Fara panou web propriu de transparenta tokeni si cost • Fara integrare Langfuse sau alt backend de tracing
Aider (GitHub) Apache 2.0	Agnostic (Claude, GPT, DeepSeek, Ollama). Aplicatie gratuita; cost limitat la tariful modelului ales.	• Mapare automata a codului sursa (<i>repo map</i>) pentru context relevant • Integrare Git nativa cu commit-uri automate per modificare • Suport pentru peste 100 de limbaje de programare • Linting si testare automata cu reincercare la esec • Suport explicit pentru modele locale	• Fara sandboxing • Capabilitati agentice limitate, fara subagenti tipizati paraleli • Fara suport MCP pentru extinderea uneltelor • Fara panou web propriu de transparenta • Fara integrare Langfuse pentru tracing si evaluari automate

Pentru a sintetiza pozitionarea *vibe-cli* fata de intregul ansamblu al concurentei (atat unelte comerciale cat si OSS), in continuare listam succint functionalitatile pe care solutia propusa le preia de la celelalte aplicatii (demonstrand caracterul de inlocuitor complet) si, simetric, functionalitatile prezente la concurenta dar neacoperite direct in implementarea curenta, clasificate in trei categorii: functionalitati in afara scopului proiectului, functionalitati care exista indirect prin mecanisme deja prezente si functionalitati genuin absente.

4.1.0.1 Functionalitati prezente in concordanta cu aplicatiile concurente propriie-tare si OSS:

- | | |
|---|--|
| 1. Suport multi-provider | 10. Sesiuni persistente cu <i>save/resume</i> |
| 2. Modele OSS locale prin Ollama | 11. Compactare automata a istoricului conversatiei |
| 3. Suport MCP | 12. Cautare web integrata |
| 4. Subagenti specializati paraleli | 13. Citire, scriere si editare de fisiere |
| 5. Skills | 14. Executie de comenzi shell |
| 6. Interceptari per tura si <i>tool guards</i> deterministe | 15. Persistarea preferintelor utilizatorului |
| 7. Permisuni <i>allow/deny</i> per unealta | 16. Intrari multimodale (imagini, PDF, fisiere text si documente) cand modelul selectat le suporta |
| 8. Sandboxing optional in container Docker | |
| 9. Comenzi cu autocompletare | |

4.1.0.2 Functionalitati absente fata de aplicatiile concurente:

1. In afara scopului proiectului:

- | | |
|--|---|
| (a) Aplicatie nativa desktop sau mobila sau extensii IDE | (e) Sandbox in cloud si executie la distanta |
| (b) Control prin vorbire | (f) Suport Kubernetes si orchestrare enterprise |
| (c) Suport multi-canal (Slack, Discord, WhatsApp) | (g) Magazin propriu de extensii |
| (d) Artefacte interactive | |

2. Existente indirect, prin mecanisme deja prezente:

- (a) Extensia pentru mediile de dezvoltare este acoperita prin terminalul integrat al oricarui IDE sau sistem de operare
- (b) Integrarea cu GitHub pentru PR-uri sau GitHub Issues este realizata prin servere MCP sau invocarea utilitarului `gh`
- (c) Rutare prin agregatoare externe de tip OpenRouter este usor extensibila prin editarea `PROVIDER_REGISTRY` din codul sursa.
- (d) Nu exista un SDK extern pentru terti, dar modulele Python pot fi importate direct.
- (e) Suport pentru operatiuni de commit Git per modificare nu este suportat nativ dar este realizabil prin skills si prin executie de comenzi shell
- (f) Maparea codului sursa de tip *repo map* e acoperita partial de subagentul `code_search`

3. Absente:

- | | |
|---------------------------------|---|
| (a) Instalare nativa pe Windows | (b) Executie de actiuni la intervale predefinite sau la momente declarate de timp |
|---------------------------------|---|

Asadar, prin acoperirea functionalitatilor centrale ale concurentei la care se adauga panoul web local, integrarea Langfuse, subagentii tipizati si *tool guards*-urile deterministe, *vibe-cli* se pozitioneaza ca un inlocuitor functional complet pentru sarcinile uzuale de programare asistata din terminal, in timp ce diferentele in raport cu concurenta tin in principal de suprafetele de utilizare (extensii IDE, aplicatii desktop, canale de mesagerie) si de maturitatea ecosistemului din jurul aplicatiei.

4.2 Influenta lungimii si a exactitatii prompt-ului asupra performantei. De ce un prompt prea exact nu garanteaza rezultate mai bune

Unul dintre experimentele rulate a fost evaluarea performantelor agentului in sarcini de PR review pe 6 proiecte open-source cu doua tipuri de prompt de sistem:

1. Un prompt sumar, de tipul "Review these PRs, output your findings in this format". Promptul complet este disponibil **aici**
2. Un prompt detaliat, de aproximativ 300 de linii care incearca sa maximizeze gasirea tuturor problemelor si indeamna agentul sa isi dea silinta sa gaseasca doar probleme reale si sa evite problemele triviale. Promptul complet este disponibil **aici**

Experimentul s-a desfasurat in urmatoarele conditii:

- Baza de date pe care o folosim contine 50 de review-uri umane considerate ideale. Dorim sa testam daca agentul nostru reuseste sa gaseasca toate problemele pe care le-au gasit inginerii umani in aceste proiecte.
- Baza de date contine o lista cu fiecare review uman, linia la care acesta a adaugat comentariul dar si intregul proiect de la momentul producerii review-ului pentru context.
- A fost folosita o arhitectura ce porneste mai multe procese paralele ce ruleaza agentul cu acces complet la unelte si subagenti, pentru a procesa mai repede cele 50 de review-uri.
- La finalul procesului de review, un alt model de limbaj mai mic se uita la review-urile umane si la cele generate de agent si verifica daca acestea relateaza aceleasi probleme.
- Derivam scorurile de precizie, recunoastere si F1 din aceste metrice.
- Modelele folosite au fost Claude Opus 4.7 pentru review-ul efectiv si Claude Haiku 4.5 pentru agentul judecator si deduplicare.

Pentru a cuantifica performanta agentului, fiecare comentariu produs in raport cu setul de referinta uman este clasificat in una dintre urmatoarele categorii:

- **True Positive (TP)**: o problema semnalata de agent care corespunde unei probleme identificate si in review-ul uman de referinta.
- **False Positive (FP)**: o problema semnalata de agent careia nu i se asociaza nicio observatie din review-ul uman (zgomot sau alerta gresita).
- **False Negative (FN)**: o problema prezenta in review-ul uman pe care agentul nu a reusit sa o identifice (omisiune).

- **True Negative (TN)**: o linie de cod pentru care nici agentul, nici recenzentul uman nu au formulat o observatie; in contextul PR review-ului acest numar este dominant si dificil de definit riguros, motiv pentru care nu intervine in metricile raportate.

Pornind de la aceste categorii, definim *precizia* ca fractiunea de alerte corecte din totalul alertelor emise de agent, *recunoasterea* (engl. *recall*) ca fractiunea de probleme reale efectiv detectate, iar scorul F_1 ca media armonica a celor doua, oferind o singura valoare scalara ce penalizeaza dezechilibrul intre precizie si recunoastere:

$$\begin{aligned} \text{Precizie} &= \frac{TP}{TP + FP}, \\ \text{Recunoastere} &= \frac{TP}{TP + FN}, \\ F_1 &= 2 \cdot \frac{\text{Precizie} \cdot \text{Recunoastere}}{\text{Precizie} + \text{Recunoastere}} = \frac{2 TP}{2 TP + FP + FN}, \end{aligned} \quad (4.1)$$

unde Precizie, Recunoastere, $F_1 \in [0, 1]$, iar valori apropiate de 1 corespund unei performante superioare. Un agent care raporteaza foarte putine probleme poate atinge o precizie ridicata cu o recunoastere scazuta, in timp ce un agent care raporteaza agresiv obtine recunoastere mare in detrimentul preciziei; scorul F_1 surprinde acest compromis intr-o singura metrica.

Problemele noi sunt probleme pe care modelul judecator le-a considerat ca fiind valide in ciuda faptului ca nu se regaseau prin review-urile umane.

Rezultatele experimentului pot fi regasite in tabelele 4.3 si 4.4.

Prompt	Proiect	Precizie	Recunoastere	F1	Probleme noi	PRs
Detaliat	cal.com	64.3%	80.6%	71.5%	11	10
Detaliat	discourse-graphite	71.1%	75.0%	73.0%	11	10
Detaliat	grafana	72.7%	68.2%	70.4%	9	10
Detaliat	keycloak	73.3%	63.6%	68.1%	8	9
Detaliat	keycloak-greptile	100.0%	100.0%	100.0%	0	1
Detaliat	sentry	81.2%	52.6%	63.9%	3	6
Detaliat	sentry-greptile	37.0%	75.0%	49.6%	1	4
Detaliat	TOTAL	66.5%	70.6%	68.5%	43	50

(a) Metrici per proiect

Severitate	Precizie	Recunoastere	F1	TP	FP	FN	Probleme noi
critical	58.8%	88.9%	70.8%	8	7	1	2
high	75.7%	80.5%	78.0%	33	18	8	23
medium	59.5%	72.3%	65.3%	34	34	13	16
low	67.6%	53.8%	60.0%	21	11	18	2

(b) Metrici pe nivele de severitate

Tabela 4.3: Rezultatele pentru prompt-ul detaliat: metrici per proiect si pe nivele de severitate

Prompt	Proiect	Precizie	Recunoastere	F1	Probleme noi	PRs
Simplu	cal.com	18.2%	83.9%	30.0%	1	10
Simplu	discourse-graphite	24.4%	100.0%	39.2%	2	10
Simplu	grafana	16.2%	90.0%	27.5%	0	9
Simplu	keycloak	25.8%	95.5%	40.7%	2	9
Simplu	keycloak-greptile	33.3%	100.0%	50.0%	0	1
Simplu	sentry	27.8%	73.7%	40.3%	1	6
Simplu	sentry-greptile	15.1%	83.3%	25.5%	1	4
Simplu	TOTAL	20.9%	88.8%	33.8%	7	49

(a) Metrici per proiect

Severitate	Precizie	Recunoastere	F1	TP	FP	FN	Probleme noi
critical	17.3%	100.0%	29.5%	9	43	0	0
high	25.0%	95.0%	39.6%	38	120	2	2
medium	22.6%	89.4%	36.1%	42	161	5	5
low	16.3%	78.9%	27.0%	30	154	8	0

(b) Metrici pe nivele de severitate

Tabela 4.4: Rezultatele pentru prompt-ul simplu: metrici per proiect si pe nivele de severitate

Analizand datele din tabelele 4.3 si 4.4, putem observa un rezultat neasteptat. Avand in vedere tehnica "Chain of Thought" [9], ne-am fi asteptat ca un agent caruia i se spune pas cu pas cum trebuie sa opereze sa aiba performante mai bune decat unul care este lasat liber.

Totusi, uitandu-ne mai atent la datele de tip "False Positive", putem intui care este cauza acestor rezultate. Din cauza restrictiilor impuse de prompt, modelul este reticent in a spune sau cerceta probleme din afara ariei pe care o acopera prompt-ul pe sistem. Acesta incearca sa maximizeze precizia si scorul F1, astfel face review-uri despre care este sigur ca sunt corecte, avand astfel un scor F1 mare si o precizie buna, dar o recunoastere mai mica decat prompt-ul simplu. Numeric, acest tip de prompt a produs in total 70 de rezultate fals pozitive.

Pe de alta parte, folosind prompt-ul simplu, agentul a fost lasat sa scrie orice problema, verificata sau nu. Acesta a scris orice pseudo-probleme (reala sau nu) care ar fi putut fi posibila. Astfel, semnaland mai multe probleme (fictive sau nu) a reusit sa acopere o suprafata mai mare de probleme, iar probabilitatea ca o problema reala sa se afle prin cele sugerate creste. Acest lucru este clar uitandu-ne la sectiunea de "False Positives" unde, in loc de 70 de review-uri ca si in cazul prompt-ului detaliat, aici avem in total 478 de comentarii marcate ca fals pozitive.

Acest studiu reflecta o problema dominanta in lumea IA: alegerea intre precizie si recunoastere. Cum realizam un agent care are o recunoastere perfecta si identifica corect toate instantele ce trebuiesc recunoscute fara a returna rezultate ireale. In domenii sensibile precum domeniul medical sau militar, acest subiect este foarte dezbatut, deoarece in general daca incercam sa marim recunoasterea, precizia va scadea. O alternativa este folosirea scorului F1, dar nici aceasta metoda nu este perfecta.

In experimentul de fata, se ridica aceeasi problema: preferam un agent care gaseste toate problemele critice dintr-un sistem, dar acest detaliu este ascuns printr-o sumedenie de raportari de severitate mica sau false, ajungand sa fie ignorat de ingineri? Sau ne bazam ca inginerul este judecatorul final si ca agentul este doar o unealta, in final tot inginerul ajungand sa faca un review complet, bazandu-se cat mai putin pe IA din cauza ca poate rata probleme majore?

Raspunsul este cel mai probabil, undeva la mijloc. Proiectantul sistemului agentic trebuie sa gaseasca un echilibru bun intre precizie si recunoastere astfel incat produsul sa aibe sens.

4.3 Contextul in mediul agentic si compactare. Consecintele metodelor de compactare

Compactarea este un procedeu necesar, dupa cum am mentionat si in capitolele anterioare. Dar, in practica, acesta nu este un procedeu perfect si are un numar de consecinte, in functie de metoda, sau setul de metode de compactare alese. Din utilizarea solutiei agentice dezvoltate cu mai multe combinatii de metode de compactare, am extras un set de repercusiuni ale folosirii acestor metode, pe categorii, dar si un set de metode recomandat pentru performante optime in raport cu integritatea informatiilor din fereastra de context.

Cea mai utila clasificare in functie de momentul in care ruleaza compactarea (imediat dupa apelul uneltelor sau cand depaseste o limita), ci pe natura pierderii pe care o introduc in context. Stadiile care taie literal blocuri din rezultatele uneltelor (`tool_truncate` si `ref_substitute`) pierd date intregi, dar lasa o urma vizibila in conversatie: un marker de trunchiere sau o referinta `@artifact:<sha>`. Stadiile care parafrazeaza prin LLM (`tool_summarize` pe rezultatele individuale ale uneltelor, `bulk` cu strategie `summarize` sau `hybrid` pe intregul istoric) pastreaza intinderea conversatiei, dar pierd nuante fara nicio urma vizibila pentru agent. Toate trei stadiile care rescriu rezultatele uneltelor (`tool_truncate`, `tool_summarize`, `ref_substitute`) opereaza asupra aceluasi slot proiectat si sunt declarate mutual exclusive in pipeline-ul `vibe-cli`; oricare configuratie valida contine cel mult unul dintre ele.

4.3.1 Stadii care taie date din context

Spre exemplu, `tool_truncate` pastreaza un cap si o coada peste `max_tool_tokens` (implicit $800 + 400$ la prag de 2000) si externalizeaza restul in artefacte externe. `ref_substitute` merge mai departe: lasa doar un extras de `excerpt_tokens` si o referinta. In ambele cazuri, in sesiunile testate, agentul ajunge in scurt timp sa **reapeleze unealta cu argumente mai inguste** (`grep` dupa un identificator, `read_file` pe un interval mai mic) pentru a recupera fragmentul disparut, in loc sa cheme `fetch_artifact`. Costul real al acestor stadii nu este, deci, pierderea de informatie care este recuperabila, ci tura suplimentara pe care agentul o consuma cautandu-o, vizibila in cresterea numarului de apeluri de unealta per sarcina.

4.3.2 Stadii care parafrazeaza date din context

In contrast, `tool_summarize` este, in cazul de succes, strict superior trunchierii pure: ruleaza in doua faze (o trunchiere sincrona, apoi o sumarizare LLM asincrona care inlocuieste forma trunchiata cand termina), iar in caz de esec ramane la forma trunchiata. `bulk` face acelasi tip de transformare la nivelul intregului istoric. Diferenta fata de stadiile care taie este ca **pierderea nu lasa urma**: rezultatul are aceeasi forma textuala, dar afirmatii cantitative (numere de linii, nume de fisiere, identificatori) pot disparea silentios. Efectul practic observat este pierderea de *recall*: agentul nu mai cere date pe care nu le mai are, pentru ca rezumatul ii spune ca le-a vazut deja. Cand modelul de compactare este unul mai mic decat cel principal (configurabil via `preferences.compaction_model`), acelasi mecanism produce ocazional afirmatii incorecte in rezumat, care apoi se propaga ca premisa in turele urmatoare.

4.3.3 Stadii auxiliare

Restul stadiilor opereaza cu alte principii si pot insoti orice combinatie de mai sus. `format_compress` reformateaza JSON-ul fara apel LLM si fara pierdere semantica: in practica nu am observat efecte negative si poate fi tinut activat neconditionat. `dedupe` elimina apeluri identice si rezultate similare peste un prag (implicit 0,92); pragurile sub 0,90 au sters in sesiunile

noastre rezultate de teste cu output similar dar semnificatie diferita, ceea ce a mascat regresii reale. `importance_prune` este robust cu setarile implicite: pierderea evenimentelor `assistant_thinking` reduce vizibil contextul fara consecinte observabile, iar pastrarea doar a ultimului `tool_result` per tura functioneaza pentru ca rezultatele intermediare sunt subsumate de cel final.

4.3.4 Secventa de stadii de compactare recomandata

Configuratia care a oferit cel mai bun compromis intre reamintire si ocupare a ferestrei de context, in experimentele rulate, are urmatoarea forma:

1. `format_compress` activat aduce castig de spatiu garantat, pierdere zero;
2. `tool_summarize` ca singur reprezentant al grupei mutual exclusive de rescriere a rezultatelor uneltelor (`tool_truncate` ramane fallback-ul sau sincron, deci este implicit prezent);
3. `dedupe` cu pragul implicit 0,92, positionat dupa `format_compress` pentru ca normalizarea formatarei creste rata de detectie;
4. `importance_prune` cu configurarea implicita;
5. `bulk` cu strategia `hybrid` si `min_turns_kept = 4`, suficient pentru a acoperi o iteratie completa cerere-investigare-raspuns.

Pentru sarcini scurte de tip intrebare si raspuns un pipeline complet dezactivat este perfect functional. Pentru sarcini cu citiri exhaustive de cod, inlocuirea `tool_summarize` cu `ref_substitute` reduce semnificativ ocuparea, cu conditia ca promptul de sistem sa instruiasca explicit agentul cand sa apeleze `fetch_artifact`. In modul `tool_persistence: true`, in care intregul istoric al uneltelor traieste intre turne, prezenta a cel putin unui stadiu ce ruleaza imediat devine obligatorie: in lipsa lui, fereastra se poate umple intr-un numar relativ mic de ture cu volum normal de unelte, inainte ca pragul `threshold` sa apuce sa declanseze `bulk`.

4.4 Interactiunea dintre modele, context si unelte. Modele optime pentru sarcini de programare

Nu toate modelele au capabilitati similare cand vine vorba despre sarcini de dezvoltare software. Pe langa necesitatea capabilitatilor de gandire rationala adanca, acestea trebuie sa fie antrenate pentru a apela functii in mod eficient si pentru a respecta dogme de programare precum SDD sau TDD.

Pentru a cuantiza exact cat de importanta este alegerea unui tip de model pentru sarcini de dezvoltare software, am incercat rularea programului cu mai multe tipuri de modele, cu capabilitati diferite si marime (ca numar de parametrii) diferita.

Modelele folosite, impreuna cu caracteristicile lor relevante, sunt prezentate in tabelul 4.5. Selectia acopera atat modele accesate prin API (familiile Claude si Gemini), cat si modele open-weight rulate local prin `ollama`, intr-o plaja larga de dimensiuni (de la 1.5 miliarde la 32 miliarde de parametrii) si specializari (uz general, rationament, generare de cod).

Model	Tip	Parametrii	Marime pe disc	Multimodal
Claude Opus 4.7	LLM rationament	nepublicat	nepublicat	da
Claude Sonnet 4.6	LLM uz general	nepublicat	nepublicat	da
Claude Haiku 4.5	LLM compact, uz general	nepublicat	nepublicat	da
Gemini 2.5 Flash	LLM uz general	nepublicat	nepublicat	da
deepseek-r1:32b	LLM, rationament	32 B	19 GB	nu
gemma4:26b	LLM, uz general	26 B	17 GB	da
phi4:14b	LLM, uz general	14 B	9.1 GB	nu
llama3.2:3b	Small Language Model, uz general	3 B	2.0 GB	nu
devstral-small-2	LLM, generare cod	24 B	15 GB	nu
qwen3-coder:30b	LLM, generare cod	30 B	18 GB	nu
qwen2.5-coder:7b	LLM, generare cod	7 B	4.7 GB	nu
qwen2.5-coder:1.5b	Small Language Model, generare cod	1.5 B	986 MB	nu

Tabela 4.5: Modelele testate: tip, numar de parametrii, marime pe disc si suport multimodal.

Pentru rezultate comparabile, toate modelele au avut de realizat aceleasi operatii. Testul a constat din 6 sarcini de intensitate diferita, executate pe un proiect real. Fiecare etapa a avut propriul mod de recompensare: de la metrici de similaritate, la LLM-as-judge, la teste ascunse. Toate acestea testeaza fie exactitatea cu care agentul a relatat faptele, uneltele sau skill-urile pe care le-a apelat si daca in final operatiunea trece testul. Aceste metrici sunt fie procentaje, binare (admis sau respins) sau un scor in functie de cat de mult a fost efectuata rezolvarea si logica cu care modelul a gandit.

Configuratia testului impreuna cu sarcinile pe care agentii le-au avut de rezolvat pot fi consultate aici

4.4.1 Rezultate

Rezultatele agregate ale rularilor sunt prezentate in tabelul 4.6, sortate dupa scorul mediu obtinut pe ansamblul cazurilor de test.

#	Furnizor	Model	Param. (Mld.)	Scor	Durata	Cost (USD)	Tokeni totali
1	anthropic	claude-opus-4-7	—	0.9062	7 min 27 s	13.40	803 585
2	anthropic	claude-sonnet-4-6	—	0.8743	10 min 4 s	0.06	17 606
3	anthropic	claude-haiku-4-5	—	0.8081	7 min 0 s	0.02	17 361
4	gemini	gemini-2.5-flash	—	0.7871	3 min 6 s	0.16	400 841
5	ollama	devstral-small-2	24	0.7051	14 min 22 s	0.00	14 620
6	ollama	gemma4	26	0.6803	9 min 49 s	0.00	6 241
7	gemini	gemini-2.5-pro	—	0.6431	4 min 14 s	0.67	435 404
8	ollama	qwen3-coder	30	0.6256	4 min 33 s	0.00	13 926
9	ollama	gemma4	8	0.3803	3 min 26 s	0.00	6 852
10	ollama	qwen3.5	9.7	0.2963	7 min 28 s	0.00	15 031
11	ollama	deepseek-r1	32	0.0747	4 min 15 s	0.00	1 484
12	ollama	llama3.2:	3	0.0590	35 s	0.00	5 380
13	ollama	qwen2.5-coder	7	0.0442	32 s	0.00	834
14	ollama	qwen2.5-coder	1.5	0.0298	21 s	0.00	828
15	ollama	phi4	14	0.0000	Eroare	0.00	—
16	gemini	gemini-2.0-flash	—	0.0000	Eroare	0.00	—

Tabela 4.6: Clasamentul modelelor dupa scor mediu, durata totala de rulare si cost. Modelele rulate local prin ollama au cost zero. Modelele proprietare nu au datele referitoare la marimea modelului publice

4.4.1.1 Analiza rezultatelor

Interpretand rezultatele obtinute putem face mai multe observatii:

1. Numarul de parametrii nu este intotdeauna direct proportional cu capabilitatile. O buna parte din capacitatea de rezolvare a problemelor in dezvoltarea software este extrasa din specializarea modelelor pe sarcini de acest tip. De aceea modelele care au fost pregatite pentru asa ceva au performante mai bune in ciuda numarului de parametrii. Un viitor experiment ar fi incercarea modelelor de la Anthropic care aici au dominat tabela pe alte domenii de activitate.
2. Putem vedea influenta capabilitatilor de gandire adanca asupra numarului de tokeni folositi (si automat a costului). Prin tehnica Chain of Thought [9] aceste modele reusesc sa isi formeze o logica superioara care ajuta la rezolvarea problemelor complexe. Dar, acest procedeu vine cu un cost. Iar in cazul Opus 4.7, acest cost este mare. Pentru 6 sarcini de programare acesta a reusit sa produca prin gandire si cercetarea fisierelor 800.000 de tokeni, in valoare de aproximativ 13 dolari. Un cost enorm comparat cu celelalte modele, chiar si de la Anthropic.
3. Modelele prea mici fie nu suporta apelarea de unelte din cauza depasirii ferestrei de context, fie nu sunt antrenate pentru asa ceva si pur si simplu intorc o eroare. Gemini 2.0 Flash si Phi4 pur si simplu intorc erori care denota incompatibilitatea cu sarcini agentice. Restul modelelor cu scoruri sub 10%, pur si simplu au halucinat rezolvarea problemei si nu au apelat nici o unealta cu adevarat. Unele dintre ele, chiar daca ar fi reusit sa apeleze uneltele, nu ar fi putut sa duca o conversatie prea complexa din cauza ferestrei mici de context.
4. Modelele locale necesita hardware performant pentru a rezolva probleme reale. Desi rezultatele sunt bune si comparabile cu cele ale modelelor de top, la o fractiune din numarul de parametrii, timpul de executie inca creaza o experienta a utilizatorului neoptima pe sisteme neperformante.
5. Variante ale modelelor antrenate special pentru sarcini de dezvoltare software arata o diferenta semnificativa in performante, precum Qwen3.5 si Qwen3-Coder.

Anomalii in rezultate:

1. Modele care detin in teorie capabilitati de gandire (Sonnet, Deepseek R1) nu au exercitat in timpul testului aceste capabilitati la potentialul maxim, ceea ce sugereaza setari implicite de efort ale modelului reduce.
2. Modelul Haiku a avut o performanta surprinzatoare avand in vedere ca acesta este considerat cel mai slab din flota Anthropic. Acest lucru se datoreaza in mare discrepantei de concurenta din cadrul testului. Nu am avut acces la modele de aceeasi generatie OpenAI sau Gemini cu care sa putem face o comparatie corecta, iar experimentul urmarea in principal relatia dintre cost si performanta.
3. Modele cu mai putini parametrii au avut un scor mult mai mic decat modele cu numar mai mic de parametrii. Spre exemplu Deepseek R1 nu a fost antrenat pentru apelarea uneltelor si desi acesta stia teoretic cum ar rezolva problema, nu a apelat nici o unealta. Pe scurt, pur si simplu a fabricat o naratiune, o poveste despre cum ar rezolva el problema, iar la final a declarat-o rezolvata.

Recomandari extrase din acest experiment:

1. Pentru rularea modelelor locale, este necesar un hardware bun. Timpul de inferenta poate fi mai crescut decat la modelele furnizorilor, sau mai mic pentru unele modele.
2. Costul modelelor creste odata cu capabilitatilor de gandire si exponential cu marimea contextului.
3. In general, modelele prea mici (mai mici de 20 de miliarde de parametrii) nu reusesc sa se descurce in sarcini de programare. Pot raspunde la intrebari punctuale, dar nu pot lucra intr-un mediu agentic. Risc mare de halucinatii.
4. Modelele neantrenate pe sarcini agentice (apel de unelte, gandire) nu au performante in mediul agentic pentru dezvoltarea software. Risc mare de halucinatii.
5. Modelele mai mici ale furnizorilor de modele de limbaj sunt mai avantajoase din punct de vedere al raportului calitate-pret.
6. Gandirea creste si ea timpul de inferenta. O comparatie buna ar fi intre Devstral Small 2 si Gemma4:26b. Desi Devstral este un model mai mic, din numarul de tokeni utilizati putem deduce ca a avut o gandire mai adanca ce a dus la timpul ridicat. In acelasi timp, Gemma a avut rezultate similare cu un timp mai scazut de inferenta (desi Gemma are si o arhitectura diferita, mai performanta)
7. Modele open-source recomandate: Gemma4 pentru echilibrul dintre scor si performanta, Devstral pentru scor, cu dezavantajul timpului mare de inferenta si Qwen3-Coder pentru timpul foarte scurt de inferenta si scor bun.
8. Din punctul de vedere al modelelor proprietare, Sonnet si Haiku au avut performante foarte bine, dar si Gemini 2.5 Flash a surprins cu performantele sale (avand in vedere ca nu mai este de ultima generatie in familia Gemini)

4.4.1.2 Modelele open-source: evaluarea valorii si a potentialului de utilizare

La finalul analizei rezultatelor, putem sumariza performanta modelelor open-source intr-un singur cuvant: surprinzatoare si intr-un sens negativ, dar si intr-unul pozitiv.

A fost neasteptat ca 6 din cele 16 modele sa aiba scor aproximativ 0 in cadrul experimentului, dar nu imposibil. Era de asteptat ca modelele sub 10 miliarde de parametrii sa intampine probleme, motiv pentru care le-am si ales. Dar cele intre 10 si 20 de miliarde, dar si Deepseek R1 au surprins prin neajunsurile lor.

Pe de alta parte am fost foarte placut impresionat de rezultatele Devstral Small 2, Qwen3-Coder si Gemma 4. Acestea au un potential imens, mai ales cele doua din urma care ofera un timp de inferenta foarte bun. Gemma4 este interesant si deoarece are o arhitectura speciala de tipul Mixtral Of Experts.

In final, consider ca folosirea modelelor open-source locale pentru sarcini de dezvoltare software (considerand detinerea unui sistem performant) este motivata de costurile inexistente, controlul asupra datelor, timpul scurt de inferenta dar si performantele bune (care pe masura ce se dezvolta vor creste).

Capitolul 5

DEZVOLTARI VIITOARE

Finalizand analiza post-implementare, impreuna cu experimentele rulate, putem extrage un set de potentiale imbunatatiri pe care le putem aduce aplicatiei in viitor.

5.1 Automatizari, integrari cu alte programe

O zona potentiala de dezvoltare este suportul pentru rularea de sarcini agentice la un interval de timp, sau la un moment in timp predefinit. Astfel de implementari sunt deja regasite in unelte precum Claude Code, prin comanda `/loop`. Aceasta functionalitate permite rularea unui prompt automat la un moment determinat. Un exemplu bun pentru aceasta utilizare in domeniul dezvoltarii software ar fi un prompt de tipul:

```
Realizeaza un rezumat al celor mai importante issue-uri sau PR-uri aparute  
in ultimele 24 de ore pe GitHub, si analizeaza canalele de Slack, Discord,  
Teams la care ai acces. Fa-mi un rezumat complet cu ceea ce gasesti.
```

Acest prompt ar fi foarte util pentru ingineri pentru a fi la curent cu starea proiectelor la care lucreaza si la deciziile ce se intampla cat timp alti colegi lucreaza in alte fusuri orare.

5.2 Integrarea cu mediile de dezvoltare

Desi interfata de tip CLI este foarte versatila, compatibilitatea cu mediile de dezvoltare aduce o granularitate mai mare asupra interactiunii cu agentul. Acesta poate sugera schimbari restranse (linie cu linie), pe care utilizatorul le poate accepta sau rejecta individual.

5.3 Suport nativ pentru Windows

Sistemul de operare Windows nu este la fel de prietenos pentru dezvoltatorii software precum macOS sau Linux din cauza terminalului (si a comenzilor) diferite si a structurii fisierelor, motiv pentru care implementarea initiala are suport doar pentru cele doua sisteme de operare listate anterior.

Acestea fiind spuse, o buna aditie viitoare ar fi suportul nativ pentru Windows, in ciuda diferentelor dintre sistemele de operare, pentru a deschide calea catre o noua audienta de utilizatori.

5.4 Construirea unui graf de cunostinte pentru optimizarea intelegerii proiectelor

O metoda inovativa care are potentialul de a imbunatati contextul unui agent din punctul de vedere al fabricarii legaturilor intre componentele unui proiect, dar mai ales din punctul

de vedere al numarului de tokeni de intrare si al reducerii apelului de unelte este aceea de a construi grafuri ale legaturilor intre diverse parti ale proiectului.

Una dintre solutiile pentru implementarea acestei facilitati este utilizarea Graphify. Acest program construiește grafurile necesare intelegerii proiectului si foloseste modele de limbaj pentru a detalia grafurile din punct de vedere semantic. Acest lucru reduce numarul de apeluri la unelte pentru citirea fisierelor, imbunatatind astfel utilizarea contextului si sporind cunoasterea amanuntita a proiectul pe care se lucreaza.

Integrarea este una simpla, graful se construiește o data (si ulterior automat daca apar schimbari in cod) iar aplicatiile agentice precum Claude Code, pot folosi integrarea oficiala sau pot folosi un server MCP pentru a interactiona cu acesta.

5.5 Suport pentru metode de compactare avansate si SDK pentru agenti

In final, o alta potentiala aditie viitoare ar fi cea legata de tehnicile avansate de compactare bazate pe similaritatea semantica a frazelor ce urmeaza a fi sters din cauza criteriilor de similaritate sau din cauza categorisirii ca fiind neimportante. Aceste tehnici pot preveni stergerea acestora daca sunt considerate informatii vitale.

O ultima schimbare care ar usura munca potentialilor contributori la dezvoltarea programului ar fi dezvoltarea unui SDK prin care acestia ar putea apela cu usurinta metodele programului aducand astfel un plus de valoare.

Capitolul 6

CONCLUZII

În cadrul acestei lucrări am reușit să atingem obiectivele pe care ni le-am impus la început. Am dezvoltat o aplicație competitivă, ce își propune să rivalizeze cu soluțiile existente pe piața prin costurile reduse de operare, impuse doar de utilizarea unor modele proprietare. De asemenea, pe lângă echivalarea capabilităților aplicatilor din industrie și costurile reduse, am reușit să aducem transparența în utilizarea modelelor de limbaj pentru utilizatori ce doresc să studieze cum funcționează acestea în realitate în domeniul dezvoltării software în vederea optimizării și îmbunătățirii sistemelor agentice sau modului în care folosim astfel de aplicații și să extragem un set de recomandări în vederea utilizării aplicațiilor agentice în scopul dezvoltării software în urma experimentelor pe care le-am condus.

În vederea implementării, lucrarea a presupus înainte de toate o planificare exhaustivă, care a început cu motivarea deciziei de a alege această temă, cercetarea lucrărilor de specialitate din domeniu și un studiu al pieței și al capabilităților necesare pentru a fi un produs util și competitiv. Motivarea a fost una simplă: arhitecturile agentice devin din ce în ce mai prezente în viața noastră de zi cu zi, investițiile confirmă faptul că această abordare are un suport puternic, iar domeniul dezvoltării software este cel care poate beneficia cel mai mult de pe urma ajutorului pe care aceste platforme îl pot oferi. Din punctul de vedere al cercetărilor lucrărilor din domeniu, nici acest lucru nu a reprezentat un impediment datorită valului de studii și dezvoltări în această zonă care crește de pe o zi pe alta. În final, pentru studiul pieței am reușit să sondez majoritatea soluțiilor deja existente, folosindu-le în sarcinile de zi cu zi, pentru zone pe care acestea nu le ating. În ciuda diversității și a capabilităților avansate, tuturor acestora le lipsea cu desăvârșire compatibilitatea cu o plajă largă de modele, transparența și observabilitatea, dar și opțiunea de a configura și personaliza în profunzime sistemul.

Trecând la partea practică a lucrării, am întâmpinat un număr de obstacole. Deși planul era făcut, am învățat că rareori acesta rămâne neschimbat pe parcursul implementării. De la decizia de a schimba framework-ul pentru interfata cu utilizatorul, până la alegerea deciziilor de implementare în partea de date de intrare multimodale, am observat că procesul de dezvoltare software este unul dinamic, care se adaptează continuu pe măsura ce produsul se maturizează.

Finalizând primul prototip al aplicației, am continuat procesul de dezvoltare prin îmbunătățiri aduse sistemului. De la lucruri mici, cum ar fi îmbunătățiri de performanță la încărcarea și testarea modelelor, până la implementarea datelor de intrare multimodale care a fost ultima schimbare majoră.

După mai multe runde de îmbunătățiri, am constatat concretizarea proiectului într-un produs consistent. Au urmat apoi partea de gândire a procesului de instalare și finisare a proiectului dar și scrierea de documentație.

În final, având un rezultat finisat, studiul utilizării acestuia putea începe pentru a putea trage concluzii cu privire la eficacitatea acestuia în contextul actual.

Cu datele din aceste studii, am reușit să formulez o serie de recomandări pentru utilizarea eficientă a aplicațiilor agentice în domeniul dezvoltării software.

Însumând toate acestea, în final, prin soluția prezentată am demonstrat valoarea arhitecturilor agentice în domeniul dezvoltării software, care prompt să schimbe complet peisajul acestui domeniu.

BIBLIOGRAFIE

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc.
- [2] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie, and Ji-Rong Wen. A survey of large language models.
- [3] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer.
- [4] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding.
- [5] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. version: 1.
- [6] Ruixiang Zhang, Richard He Bai, Huangjie Zheng, Navdeep Jaitly, Ronan Collobert, and Yizhe Zhang. Embarrassingly simple self-distillation improves code generation.
- [7] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. 36:10088–10115.
- [8] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744. Curran Associates, Inc.
- [9] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. 35:24824–24837.
- [10] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, volume 36, pages 11809–11822. Curran Associates, Inc.
- [11] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models.
- [12] Binfeng Xu, Zhiyuan Peng, Bowen Lei, Subhabrata Mukherjee, Yuchen Liu, and Dongkuan Xu. ReWOO: Decoupling reasoning from observations for efficient augmented language models.

- [13] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. 12:157–173.
- [14] Kiran Vodrahalli, Santiago Ontanon, Nilesch Tripuraneni, Kelvin Xu, Sanil Jain, Rakesh Shivanna, Jeffrey Hui, Nishanth Dikkala, Mehran Kazemi, Bahare Fatemi, Rohan Anil, Ethan Dyer, Siamak Shakeri, Roopali Vij, Harsh Mehta, Vinay Ramasesh, Quoc Le, Ed Chi, Yifeng Lu, Orhan Firat, Angeliki Lazaridou, Jean-Baptiste Lespiau, Nithya Attaluri, and Kate Olszewska. Michelangelo: Long context evaluations beyond haystacks via latent structure queries. version: 2.
- [15] Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Vijay Bharadwaj, Jeff Holm, Raja Aluri, Chen Bo Calvin Zhang, Noah Jacobson, Bing Liu, and Brad Kenstler. SWE-bench pro: Can AI agents solve long-horizon software engineering tasks?
- [16] F. Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain.
- [17] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. 86(11):2278–2324.
- [18] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. 9:1735–1780.
- [19] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners.
- [20] Yichun Xu, Navjot K. Khaira, and Tejinder Singh. KV cache optimization strategies for scalable and efficient LLM inference. version: 1.
- [21] Weian Mao, Xi Lin, Wei Huang, Yuxin Xie, Tianfu Fu, Bohan Zhuang, Song Han, and Yukang Chen. TriAttention: Efficient long reasoning with trigonometric KV compression.
- [22] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding.
- [23] Amir Zandieh, Majid Daliri, Majid Hadian, and Vahab Mirrokni. TurboQuant: Online vector quantization with near-optimal distortion rate.
- [24] Zhongmou He, Yee Man Choi, Kexun Zhang, Ivan Bercovich, Jiabao Ji, Junting Zhou, Dejia Xu, Aidan Zhang, Yixiao Zeng, and Lei Li. HARDTESTGEN: A high-quality RL verifier generation pipeline for LLM algorithmic coding.
- [25] Sander Schulhoff, Michael Ilie, Nishant Balepur, Konstantine Kahadze, Amanda Liu, Chenglei Si, Yinheng Li, Aayush Gupta, Hyojung Han, Sevien Schulhoff, Pranav Sandeep Dulepet, Saurav Vidyadhara, Dayeon Ki, Sweta Agrawal, Chau Pham, Gerson Kroiz, Feileen Li, Hudson Tao, Ashay Srivastava, Hevander Da Costa, Saloni Gupta, Megan L. Rogers, Inna Goncearenco, Giuseppe Sarli, Igor Galynker, Denis Peskoff, Marine Carpuat, Jules

- White, Shyamal Anadkat, Alexander Hoyle, and Philip Resnik. The prompt report: A systematic survey of prompt engineering techniques.
- [26] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan A, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into state-of-the-art pipelines.
 - [27] Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki, editors, *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2609–2634. Association for Computational Linguistics.
 - [28] Dongfang Li, Zixuan Liu, Gang Lin, Baotian Hu, and Min Zhang. LycheeCluster: Efficient long-context inference with structure-aware chunking and hierarchical KV indexing.
 - [29] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. Curran Associates, Inc.
 - [30] Asif Shahriar, Md Nafiu Rahman, Sadif Ahmed, Farig Sadeque, and Md Rizwan Parvez. A survey on agentic security: Applications, threats and defenses.
 - [31] William Hackett, Lewis Birch, Stefan Trawicki, Neeraj Suri, and Peter Garraghan. Bypassing LLM guardrails: An empirical analysis of evasion attacks against prompt injection and jailbreak detection systems. In Leon Derczynski, Jekaterina Novikova, and Muhao Chen, editors, *Proceedings of the The First Workshop on LLM Security (LLMSEC)*, pages 101–114. Association for Computational Linguistics.
 - [32] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, Carol Chen, Catherine Olsson, Christopher Olah, Danny Hernandez, Dawn Drain, Deep Ganguli, Dustin Li, Eli Tran-Johnson, Ethan Perez, Jamie Kerr, Jared Mueller, Jeffrey Ladish, Joshua Landau, Kamal Ndousse, Kamile Lukosuite, Liane Lovitt, Michael Sellitto, Nelson Elhage, Nicholas Schiefer, Noemi Mercado, Nova DasSarma, Robert Lasenby, Robin Larson, Sam Ringer, Scott Johnston, Shauna Kravec, Sheer El Showk, Stanislav Fort, Tamera Lanham, Timothy Telleen-Lawton, Tom Conerly, Tom Henighan, Tristan Hume, Samuel R. Bowman, Zac Hatfield-Dodds, Ben Mann, Dario Amodei, Nicholas Joseph, Sam McCandlish, Tom Brown, and Jared Kaplan. Constitutional AI: Harmlessness from AI feedback.
 - [33] Traian Rebedea, Razvan Dinu, Makesh Narsimhan Sreedhar, Christopher Parisien, and Jonathan Cohen. NeMo guardrails: A toolkit for controllable and safe LLM applications with programmable rails. In Yansong Feng and Els Lefever, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 431–445. Association for Computational Linguistics.
 - [34] Yue Liu, Hongcheng Gao, Shengfang Zhai, Yufei He, Jun Xia, Zhengyu Hu, Yulin Chen, Xihong Yang, Jiaheng Zhang, Stan Z. Li, Hui Xiong, and Bryan Hooi. GuardReasoner: Towards reasoning-based LLM safeguards.